A Greedy PDE Router for Blending Neural Operators and Classical Methods

Anonymous Author(s)
Affiliation
Address
email

Revision note. All text, tables and figures added in this revision are typeset in red (new figures carry a red frame); everything in black is unchanged from the submitted version.

Abstract

When solving PDEs, classical numerical solvers are often computationally expensive, while machine learning methods can suffer from spectral bias, failing to capture high-frequency components. Designing an optimal hybrid iterative solver–where, at each iteration, a solver is selected from an ensemble of solvers to leverage their complementary strengths–poses a challenging combinatorial problem. While greedy selection is desirable for its constant-factor approximation guarantee to the optimal solution under Lipschitz assumptions, it requires knowledge of the true error at each step, which is unavailable in practice. We address this by proposing an approximate greedy router that efficiently mimics a greedy approach to solver selection. Empirical results on the Poisson and convection–diffusion equations show that our method consistently reduces final error and area-under-the-curve (AUC) of the error trajectory relative to single-solver baselines and existing hybrid approaches such as HINTS. In particular, our method reaches comparable error levels in substantially fewer iterations while exhibiting more stable error decay. With a cost-aware instantiation of the greedy rule, a lightweight router, and a band-limited scale-equivariant corrector, these gains carry over to end-to-end wall-clock time on a 128×128 grid: for every one of the ten solver–equation pairings the learned router reaches truncation-level accuracy $238\times$–$1983\times$ faster than the classical solver alone and $2.68\times$–$10.3\times$ faster than HINTS, and it matches or beats HINTS even when the latter's period is tuned per setting.

1 Introduction

Natural phenomena and engineered systems are often governed by ordinary and partial differential equations (PDEs). Solving these equations enables a variety of tasks - predicting the evolution of the system through simulation (e.g., forecasting weather using the Navier-Stokes equations) [Kalnay, 2003, Bauer et al., 2015, Staniforth and Côté, 1991], addressing control problems (e.g., optimizing heat shield design for spacecrafts using heat transfer equations) [Anderson, 1989, Tröltzsch, 2010], and tackling inverse problems based on external measurements (e.g., reconstructing brain activity from EEG data using electrophysiological models) [Baillet et al., 2001, Grech et al., 2008].

Traditionally, PDEs are solved using finite difference methods [Smith, 1985, LeVeque, 2007], which discretize the spatial and/or temporal domain and approximate the solution by solving a system of equations at the discrete grid points. Similarly, finite element methods [Hughes, 2003, Bathe, 2006, Brenner, 2008] construct a system of equations based on local basis functions, and then rely on numerical linear algebra techniques–such as the Jacobi and Gauss-Seidel method [Saad, 2003]–to compute the solution. These iterative methods, however, lack generalization across different

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

initial conditions, boundary conditions, or forcing functions, as even minor changes to any of these parameters require re-solving the entire system of equations from scratch.

These challenges have motivated the development of neural operators [Kovachki et al., 2023]–a class of machine learning models that aim to learn the solution operator directly, enabling fast inference across a range of parameters. Neural operators lift the classical linear layer in neural networks to an operator–typically a kernel integral operator–between function spaces. Despite strong approximation guarantees [Chen and Chen, 1995], neural operators are prone to spectral bias [Liu and Cai, 2021, Liu et al., 2024, You et al., 2024, Khodakarami et al., 2025, Xu et al., 2025], similar to traditional neural networks [Rahaman et al., 2019, Xu et al., 2019, Luo et al., 2019]. As a result, they favor learning low-frequency components of the solution function and often struggle to capture high-frequency features that iterative solvers excel at.

Recognizing this complementarity, Zhang et al. [2024] proposed HINTS, a hybrid solver that interleaves a neural operator pass every τ^{th} iteration of a classical solver, with τ fixed a priori. While HINTS reports significant empirical gains in convergence and accuracy over the standalone neural operator and Jacobi solver, this fixed schedule can be detrimental: a poorly timed neural correction may increase the error and undo recent progress.

Although state-dependent solver schedules are desirable, this naturally suggest a reinforcement learning formulation for learning routing policies. However, in our setting, the structure of iterative PDE solvers allows for a simpler approach: each solver induces a known update, and its effect can be evaluated through error-based signals. Leveraging this, we adopt a greedy rule that selects, at each iteration, the solver with the largest immediate error reduction. Such a rule can be near-optimal when the final error satisfies (weak) supermodularity–so that applying a beneficial solver earlier is at least as valuable as applying it later. To support this approach, we make the following contributions:

- We present a general hybrid PDE solver in which a routing rule chooses, at each iteration, from a set of classical solvers and neural operators. With oracle access to the true error at every step, we show that a greedy routing rule achieves a constant-factor approximation to the optimal strategy for linear PDEs, provided the updates are error-reducing (Lipschitz with constant < 1) and zero-preserving–conditions under which weak supermodularity holds.
- Given that the true error is not observed at test time, a convex surrogate loss is introduced, which, when minimized, enables the learned model to imitate the greedy router without access to true error information. This alignment is guaranteed through Bayes consistency.
- We empirically demonstrate that our approximate greedy solver achieves faster convergence than HINTS and individual solvers, often reaching comparable error levels in fewer iterations on the Poisson and convection-diffusion equations.

2 Background

Consider the following linear PDE:

$$\mathcal{L}_x^a(u) = f, x \in D; \quad \mathcal{B}_x(u) = b, x \in \partial D \tag{1}$$

where $\mathcal{L}_x^a$ is a differential operator with respect to the spatial variable $x \in D$ parameterized by the coefficient function a, f is a forcing function, u is the PDE solution, $\mathcal{B}_x$ is the boundary operator, and b is the boundary term. Such PDEs can be solved numerically using various discretization strategies, such as finite differences [Smith, 1985, LeVeque, 2007], finite element, and spectral methods [Boyd, 2001, Canuto et al., 2006]. Here, we focus on finite differences, which replace derivatives with difference quotients (e.x., $\partial_x u(x) \approx (u(x+h) - u(x))/h$) on a uniform grid.

Formally, let h be the grid size and $G_h(D)$ be a uniform grid with spacing h over D. For a function $g : D \to \mathbb{R}$, we denote its restriction to $G_h(D)$ by $g_h \in \mathbb{R}^N$, where $N = |G_h(D)|$. If $\mathcal{L}_h^a \in \mathbb{R}^{N \times N}$ is the discretized operator, the PDE reduces to a linear system of equations

$$\mathcal{L}_h^a u_h = f_h, \tag{2}$$

with the boundary conditions incorporated in $\mathcal{L}_h^a$. Unlike much of the neural operator literature, which retains function space formulations for discretization invariance, we follow the conventions of classical numerical analysis and represent functions and operators as vectors and matrices. Discretization invariance is not essential here, since we focus on fixed-grid problems.

2.1 Iterative PDE Solvers

Direct methods like Gauss Elimination and Thomas Algorithm can be computationally expensive in high-dimensional domains when solving systems like Equation (2). In contrast, iterative methods like Jacobi and Gauss-Seidel offer computational speedups by iteratively updating the solution, gradually converging to the true solution for the PDE. The general iterative update is

$$u^{(t+1)} = u^{(t)} + C\left(f_h - \mathcal{L}_h^a u^{(t)}\right), \tag{3}$$

where $u^{(t)}$ is the t^{th} iterate of the solution and C is a preconditioning matrix. Jacobi uses $C = D^{-1}$, where D is the diagonal of $\mathcal{L}_h^a$, and Gauss-Seidel uses $C = (D + L)^{-1}$, where L denotes the strict lower triangular part. However, these methods tend to damp low frequency parts of the error slowly.

2.2 Neural Operators

Suppose we observe samples $\{(a_i, f_i, u_i)\}_{i=1}^N$ such that $(a_i, f_i) \sim \mathcal{P}$ are i.i.d. samples, and $u_i = \mathcal{G}^*(a_i, f_i)$ be generated by a deterministic solution operator $\mathcal{G}^*$. A neural operator (NO) $\mathcal{G}_\theta$ seeks to approximate $\mathcal{G}^*$ by minimizing the expected squared $L^2(D)$ error:

$$\mathcal{R}_{\text{NO}}(\mathcal{G}_\theta) = \mathbb{E}_{(a,f)\sim\mathcal{P}}\left[\int_D \|u_i(x) - \mathcal{G}_\theta(a_i, f_i)(x)\|_2^2 \, dx\right]$$

Neural operators use discretization-invariant layers. Prominent instantiations include Fourier Neural Operators (FNO), which use spectral transforms [Li et al., 2020a], Graph Neural Operators, which perform graph-based aggregation over sampled points [Li et al., 2020b], and DeepONets, which employ a trunk–branch decomposition to map input functions to output functions [Lu et al., 2021].

2.3 Greedy Optimization

Consider maximizing $g(S)$ over $S \subseteq \Omega$ with $|S| \leq T$ where $g : 2^\Omega \to \mathbb{R}$ is a set function defined on subsets of a set Ω. An exhaustive search is infeasible, so an efficient alternative is the greedy algorithm. It builds the solution subset iteratively by adding ω^* that yields the largest marginal gain, i.e. $\omega^* = \arg\max_{\omega\in\Omega\setminus S} g(S \cup \omega)$. When g is non-negative, monotone (adding elements never decreases the value), and submodular (diminishing returns), the greedy algorithm achieves a $(1 - e^{-1})$ approximation to the optimal solution [Nemhauser et al., 1978]. Formally, a function g is submodular if $g(A \cup \{\omega\}) - g(A) \geq g(B \cup \{\omega\}) - g(B)$ for all $A \subseteq B \subseteq \Omega$ and $\omega \in \Omega \setminus B$; intuitively, delaying an addition cannot increase its benefit. For set function minimization problems, supermodularity, where the inequality flips, plays an analogous role, and the greedy rule achieves constant-factor approximation guarantees [Liberty and Sviridenko, 2017].

The suboptimality of the greedy algorithm has been extensively studied for both set-function minimization [Bounia and Koriche, 2023] and maximization [Das and Kempe, 2018, Feige et al., 2011, Bian et al., 2017, Harshaw et al., 2019] when standard assumptions–non-negativity, monotonicity, and sub/supermodularity–are weakened. In contrast, results on greedy sequence maximization [Streeter and Golovin, 2008, Alaei et al., 2021, Zhang et al., 2015, Bernardini et al., 2020, Van Over et al., 2024, Tschiatschek et al., 2017]–where the ordering of the elements affects the function–have led to sequential analogues of submodularity and monotonicity. We leverage these tools to analyze the suboptimality of the greedy solution to minimizing the final error.

3 General framework for Hybrid Solvers

To solve Equation (2), consider the following hybrid iterative update:

$$u_h^{(t+1)} = u_h^{(t)} + C_{S_t}\left(f_h - \mathcal{L}_h^a u_h^{(t)}\right) \tag{4}$$

where $\mathcal{C} = \{C_j\}_{j=1}^K$ is a set of preconditioning functions and $S_t \in [K] = \{1, \ldots, K\}$ indexes the function chosen at step t. Here, we use "preconditioning function" broadly to include classical linear updates ($C_j(x) = C_j x$ where C_j is a matrix), and learned models, such as neural operators. This update generalizes the classical iterative update in Equation (3), by allowing C_j to be non-linear and enabling the preconditioning function to be adaptively chosen at every step. $\mathcal{C}$ can also accommodate

parameterized solver families (e.g., different Jacobi relaxation weights), enabling adaptive selection of solver parameters. As the number of solvers K grows, the chance of selecting a more effective update increases. Furthermore, HINTS [Zhang et al., 2024] is a special case of this hybrid solver with $K = 2$, where C_1 is a neural operator and C_2 is a classical preconditioner. Its routing rule is given by $S_t = \mathbf{1}_{t \bmod \tau > 0} + 1$, which selects C_1 every τ steps and C_2 otherwise.

Let $e_h^{(t)} = u_h - u_h^{(t)}$ denote the error at step t. Then,

$$e_h^{(t+1)} = u_h - u_h^{(t)} - C_{S_t}\left(\mathcal{L}_h^a u_h - \mathcal{L}_h^a u_h^{(t)}\right) = \left(I - C_{S_t} \circ \mathcal{L}_h^a\right)\left(e_h^{(t)}\right)$$

where I is the identity map and $I - C_j \circ \mathcal{L}_h^a$ is the error propagation function for solver j. Here, "$\circ$" denotes function composition (matrix multiplication for linear updates). The objective is to select a sequence $S = (S_1, \ldots, S_T)$ that minimizes the error norm after T steps or $\|e_h^{(T)}\|_2^2$:

$$\min_{S_t \in [K], |S| \le T} h(S) := \left\| \left(I - C_{S_{|S|}} \circ \mathcal{L}_h^a\right) \circ \cdots \circ \left(I - C_{S_1} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \tag{5}$$

with compositions applied from right to left, so that the S_1 update acts on the initial error.

4 Greedy Algorithm

Equation (5) defines a combinatorial optimization problem with a search space exponential in T, making exact optimization intractable for large T or K. As a starting point, we consider an "omniscient" greedy algorithm that assumes access to the true initial error $e_h^{(0)}$. This is unrealistic since knowledge of $e_h^{(0)}$ could directly recover the solution via $u_h = u_h^{(0)} + e_h^{(0)}$, but it provides a useful benchmark. In Section 5, we relax this assumption using a practical learning strategy that is Bayes consistent with this omniscient rule, thereby recovering the guarantees shown below.

As discussed in Section 2.3, when supermodularity and monotonicity hold, the greedy rule–such as the one described in Algorithm 1–enjoys constant-factor approximation guarantees. However, classical results focus on *set* functions, with only recent extensions made to sequences. Building on this line, we introduce a sequence-based notion of weak supermodularity.

Algorithm 1 Greedy Algorithm for a Hybrid PDE solver

Require: $\{C_j\}_{j=1}^K, T, \mathcal{L}_h^a, e_h^{(0)}$
  $S^0 \leftarrow \emptyset$
  for $t < T$ **do**
    $S^{t+1} \leftarrow S^t \oplus \arg\min_{j \in [K]} \|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$
    $e_h^{(t+1)} \leftarrow (I - C_{S_{t+1}} \circ \mathcal{L}_h^a)(e_h^{(t)})$
  end for
  return S^T

Let Ω^* denote the space of sequences with elements in Ω. For $S, S' \in \Omega^*$, we denote their concatenation as $S \oplus S'$. S is a prefix of S' or $S \preceq S'$ if $S' = S \oplus L$ for some $L \in \Omega^*$. A function $g : \Omega^* \to \mathbb{R}$ is prefix monotonically non-increasing if $g(S \oplus S') \le g(S)$ for all $S, S' \in \Omega^*$, and postfix monotonically non-increasing if $g(S' \oplus S) \le g(S)$ for all $S, S' \in \Omega^*$. A prefix non-increasing function g is sequence supermodular if, for all $S', S \in \Omega^* : S \preceq S'$, it holds that

$$g(S) - g(S \oplus \omega) \ge g(S') - g(S' \oplus \omega), \quad \forall \omega \in \Omega \tag{6}$$

However, $h(S)$ (defined in Equation (5)) may not satisfy this property in general. Therefore, we introduce weak sequence supermodularity. A prefix non-increasing function g is weakly supermodular with respect to $S' \in \Omega^*$ if, for any $S \in \Omega^{|S'|}$, there exists $\alpha(S') \ge 1$ such that

$$g(S) - g(S \oplus S') \le \alpha(S') \sum_{i \in [|S'|]} g(S) - g(S \oplus S'_i) \tag{7}$$

The parameter $\alpha(S')$ or the supermodularity ratio quantifies deviation from exact sequence supermodularity. Expanding the $g(S) - g(S \oplus S')$ as a telescoping sum $\sum_{i=1}^{|S'|} g(S \oplus (S'_1, \ldots, S'_{i-1})) -$

$g(S \oplus (S'_1, \ldots, S'_i))$ shows that the marginal decrease from appending S'_i after its predecessors is controlled by the effect of appending S'_i directly to S. Thus, postponing the inclusion of S'_i cannot yield a significantly larger benefit than adding it earlier. The supermodularity ratio $\alpha(S)$ quantifies the extent to which future gains from delays may exceed immediate gains.

Having introduced these notions, we now characterize the suboptimality of greedy solutions of weakly supermodular and postfix monotonic sequence functions in Theorem 4.1.

Theorem 4.1. *Let $g : \Omega^* \to \mathbb{R}$ be a weakly supermodular function with respect to the optimal solution $O = \arg\min_{S \in \Omega^T} h(S)$ with a supermodularity ratio of $\alpha(O)$ and postfix monotonicity. Let the greedy solution of length T be S^T. If $\phi_T(\alpha) = \left(1 - \frac{1}{\alpha T}\right)^T$, then*

$$g(\emptyset) - g(S^T) \geq (1 - \phi_T(\alpha(O)))\,(g(\emptyset) - g(O))$$

The proof of Theorem 4.1 appears in Section A.2. As $T \to \infty$, the factor $1 - \phi_T(\alpha(O))$ decreases to $1 - e^{-1/\alpha(O)}$, so the worst-case performance of the greedy rule saturates rather than degrades with horizon. Additionally, larger $\alpha(O)$, which indicates higher reward for delayed inclusions, loosens the suboptimality bound. Theorem 4.1 requires that the sequence objective h be weakly supermodular with respect to the optimal solution and postfix monotone–properties established in Proposition 4.2. We defer the proof to Section A.3.

Proposition 4.2. *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is ρ_j-Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is weakly supermodular with respect to the optimal solution O, with*

$$\alpha(O) = \max\left\{\frac{4}{T - \sum_{i=1}^{T} \rho_{O_i}^2}, 1\right\}$$

Furthermore, if $I - C_j \circ \mathcal{L}_h^a$ is invertible for all $j \in [K]$, h is also postfix non-increasing.

The conditions of Proposition 4.2 are natural. For classical solvers, the Lipschitz constant is $\|I_N - C_j \mathcal{L}_h^a\|$, which is typically less than 1 for well-posed linear elliptic PDEs (i.e., the update is damping errors). For neural networks, Lipschitz continuity can be enforced via weight regularization [Gouk et al., 2021] and a sufficiently trained model should approximate $(\mathcal{L}_h^a)^{-1}$ well enough to make the Lipschitz constant small. The requirement $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$ is both natural and desirable: it precludes spurious updates when the residual $f_h - \mathcal{L}_h^a u_h^{(t)}$ is 0. This holds by design for classical schemes, and it can be enforced for any learned model by excluding bias terms.

The form of $\alpha(O)$ in Proposition 4.2 highlights that the suboptimality factor in Theorem 4.1 is governed by the collective contraction factors of the solvers chosen by the optimal solution: as $\sum_{i=1}^{T} \rho_{O_i}^2 \to T$, $\alpha(O)$ grows, resulting in a weaker bound. Invertibility of the error propagation functions is often satisfied with Jacobi and Gauss-Seidel updates. However, neural networks with dimension changes or ReLU activations may yield non-invertible error propagation maps. Nevertheless, our experiments show that greedy routers remain effective even when invertibility is not met.

Theorem 4.1 thus indicates that the approximation guarantee is strongest when the sequence is nearly supermodular ($\alpha(O) \approx 1$). Generally, if all error propagation maps share an eigenbasis, supermodularity is established. This occurs, for example, for linear, constant-coefficient PDEs with periodic boundary conditions where the solver ensemble includes Jacobi, Gauss-Seidel, and a single-layer linear Fourier Neural Operator. The proof of Theorem 4.3 is deferred to Section A.5.

Proposition 4.3. *Let $\|I_N - C_j \mathcal{L}_h^a\| \leq 1$ for all $j \in [K]$ and $(I - C_j \mathcal{L}_h^a) = P \Lambda_j P^{-1}$. Then, h is supermodular.*

Cost-aware greedy routing. Algorithm 1 treats every update as equally expensive, whereas in practice a neural-operator evaluation and a classical sweep have very different costs, and the quantity of interest is the error reached per unit of wall-clock time. We make the greedy rule cost-aware without changing the analysis: let c_j denote the (measured) cost of one application of C_j, take the cost u of one neural-operator call as the unit, and replace each preconditioner by the *macro-action* $C_j^{(m_j)}$ that applies C_j for $m_j = \max\{1, \mathrm{round}(u/c_j)\}$ consecutive iterations, so that all actions cheaper than the unit cost approximately u; an action dearer than the unit (a multigrid cycle, say)

is applied once and its error is compared per unit of cost, $\|e\|^{u/(m_j c_j)}$, which reduces to the plain comparison when the costs match. For a linear update, m consecutive sweeps are again a stationary iteration, $(I - C_j \circ \mathcal{L}_h^a)^m = I - C_j^{(m)} \mathcal{L}_h^a$ with $C_j^{(m)} = \sum_{i=0}^{m-1} (I - C_j \mathcal{L}_h^a)^i C_j$, and for a nonlinear C_j the macro-action is the m-fold composition of its error propagation map. Hence $\{C_j^{(m_j)}\}_{j=1}^K$ is again a set of preconditioning functions in the sense of Section 4: if $I - C_j \circ \mathcal{L}_h^a$ is ρ_j-Lipschitz, zero-preserving and invertible, then so is its m_j-fold composition (with constant $\rho_j^{m_j} < 1$), and Theorem 4.1, Propositions 4.2 and 4.3, and the consistency result of Section 5 apply verbatim to the macro-action ensemble. Running Algorithm 1 on macro-actions selects, at every decision epoch, the action with the largest error reduction per unit cost, and a horizon of T macro-actions corresponds to a wall-clock budget of $\approx Tu$; Section E uses this cost-aware instantiation throughout.

5 Approximate Greedy Router

The results in Section 4 indicate that the error reduction from the greedy solution closely matches that of the optimal sequence, but this is predicated on having access to the initial error, $e_h^{(0)}$. Inaccurate error estimates can result in poor solver decisions, causing errors to amplify. To remedy this, we learn a router r that selects solvers myopically, as in Algorithm 1, without access to true errors.

We adopt the following learning setup. Let $\mathcal{A}, \mathcal{F}, \mathcal{U} \subseteq \mathbb{R}^N$ denote the spaces of coefficient, forcing, and solution functions on the grid $G_h(D)$. We assume an application-specific data distribution $\mathcal{P}_{\mathcal{A}\times\mathcal{F}}$ over $\mathcal{A} \times \mathcal{F}$ that reflects test time conditions. During training, (a_h, f_h) is drawn from $\mathcal{P}_{\mathcal{A}\times\mathcal{F}}$, and a high-accuracy reference solution u_h is computed, providing the true per-step error. The router r is learned offline on this data; at test time, it operates without access to true errors.

At iteration t, router r selects a solver using the coefficient $a_h \in \mathcal{A}$, forcing $f_h \in \mathcal{F}$, and the current iterate $u_h^{(t)} \in \mathcal{U}$. If $r(a_h, f_h, u_h^{(t)}) = j$, the next iterate is computed with solver j, resulting in an error of $\|(I - C_j \circ \mathcal{L}_h^a)(e_h^{(t)})\|_2^2$. Learning such a router requires minimizing the following loss:

$$l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, u_h\right) = \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right) \right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)})=j} \tag{8}$$

with risk $\mathcal{R}_{\text{route}}(r) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A}\times\mathcal{F}}}[l_{\text{route}}(r, a_h, f_h, u_h^{(t)}, u_h)]$. For analysis, we fix the iterate generation via teacher-forcing [Williams and Zipser, 1989, Lamb et al., 2016]: during training, the iterates fed to the router are produced by an oracle greedy rollout, not by the router's own past choices. Formally, with $u_h^{(0)} = 0_N$, for $t \geq 1$,

$$j_t^* \in \text{argmin}_{j\in[K]} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(e_h^{(t-1)}\right) \right\|_2^2, \quad u_h^{(t)} = u_h^{(t-1)} + C_{j_t^*}\left(f_h - \mathcal{L}_h^a u_h^{(t-1)}\right)$$

Thus, $\{u_h^{(t)}\}$ is a deterministic function of (a_h, f_h). At each t, l_{route} is evaluated on $r(a_h, f_h, u_h^{(t)})$ which takes in the teacher-forced iterate. Finally, the greedy choice j_{t+1}^* is used to advance $u_h^{(t+1)}$. Thus the input distribution seen by r depends on (a_h, f_h), not on the router's predictions. However, full teacher forcing induces a distributional mismatch at test time (exposure bias). In experiments, we mitigate this with scheduled sampling [Bengio et al., 2015]; see Section C for details.

Minimizing $\mathcal{R}_{\text{route}}(r)$ yields the following Bayes-Optimal Router:

$$r^*(a_h, f_h, u_h^{(t)}) \in \text{argmin}_{j\in[K]} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(e_h^{(t)}\right) \right\|_2^2 \tag{9}$$

which aligns with the update in Algorithm 1. Hence, l_{route} is consistent with learning the greedy rule.

5.1 Surrogate Loss

Since Equation (8) is discontinuous and non-convex, direct minimization is intractable in practice. We instead introduce a surrogate loss that (1) is convex, enabling efficient optimization; (2) upper bounds the original loss; and (3) is Bayes consistent, ensuring the Bayes optimal decision of the original loss is preserved upon minimization. Formally, a surrogate ϕ is considered to be Bayes consistent with respect to the loss l if

$$\lim_{n\to\infty} \mathcal{R}_\phi(f_n) - \mathcal{R}_\phi^* \implies \lim_{n\to\infty} \mathcal{R}_l(f_n) - \mathcal{R}_l^*$$

where $\mathcal{R}_l(f) = \mathbb{E}\left[l(f(X), Y)\right]$ and $\mathcal{R}_l^* = \inf_f \mathcal{R}_l(f)$. In other words, in the limit of infinite data, if the risk of a sequence of learned hypotheses $\{f_n\}$ converges to the optimal risk under ϕ, it also converges to the optimal risk with respect to the original loss l.

To define a surrogate loss for the routing problem, consider a set of scoring functions $\mathbf{g} = \{g_j\}_{j=1}^K$ with $g_j : \mathcal{A} \times \mathcal{F} \times \mathcal{U} \to \mathbb{R}$ and we define the router as $r(a, f, u^{(t)}) = \text{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$. For example, $\mathbf{g}$ can be a neural network with K outputs. Then, minimizing the following surrogate loss yields the same decision as minimizing Equation (8):

$$\Psi\left(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h\right) = -\sum_{j=1}^{K}\sum_{k=1}^{K} \tilde{c}_k(a_h, u_h^{(t)}, u_h)\mathbf{1}_{k \neq j} \log\left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{m=1}^{K}\exp\left(g_m(a_h, f_h, u_h^{(t)})\right)}\right) \tag{10}$$

where $\tilde{c}_j(a_h, u_h^{(t)}, u_h) = \|(I - C_j \circ \mathcal{L}_h^a)(u_h - u_h^{(t)})\|_2^2$. The Ψ-risk is denoted by $\mathcal{R}_\Psi(\mathbf{g}) = \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A}\times\mathcal{F}}}[\Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)]$. The convexity of Ψ with respect to $\mathbf{g}$ follows from the convexity of log-softmax function in its inputs. Moreover, Ψ upper bounds l_{route} up to a constant factor, and we refer the reader to Section B.2 for the proof. Finally, Theorem 5.1 shows that Ψ achieves Bayes consistency with respect to l_{route}; the proof can be found in Section B.3.

Theorem 5.1. *Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route}.*

Theorem 5.1 is a cost-sensitive analogue of the classical Bayes-consistency of multiclass cross-entropy for 0-1 loss: in the infinite-sample limit, minimizing cross-entropy recovers the true conditional class probabilities, so the induced decision is Bayes optimal. Our result extends this to cross-entropy with instance-dependent weights $\sum_{k\neq j}^K \tilde{c}_k(a_h, u_h^{(t)}, u_h)$. The uniform upper bound holds when all preconditioning functions are error-damping. It is also reasonable to assume at least one solver cannot annihilate the error in one step, yielding the lower bound. Under these conditions, minimizing Ψ recovers the Bayes-optimal router of Equation (9), i.e., the greedy solution of Equation (5). Following the work of Mao et al. [2024], the proof uses standard conditional-risk calibration: we relate the excess risks of l_{route} and Ψ and take the infinite-sample limit.

6 Related Works

Hybrid PDE Solvers: Early data-driven solvers sought convergence guarantees by predicting parameters–e.g., preconditioning matrices, multi-grid smoothers or restriction matrices–within iterative schemes [Taghibakhshi et al., 2021, Caldana et al., 2024, Kopaničáková and Karniadakis, 2025, Huang et al., 2022, Katrutsa et al., 2020]. These works, however, do not leverage the neural surrogates' ability to generalize across varying coefficients and forcings highlighted in Section 2.2 and thus, offer only modest speedups over classical solvers. HINTS [Zhang et al., 2024] introduced hybrid solvers that interleave a classical method with a pre-trained DeepONet on a fixed schedule (e.g., 24 Jacobi steps, then one DeepONet correction), achieving faster convergence than the standalone numerical solver. Subsequent works extend this idea to new geometries [Kahana et al., 2023] and analyze error mode damping, replacing DeepONet with alternatives such as MIONet [Jin et al., 2022] [Hu and Jin, 2025] or FNO [Li et al., 2020a, Cui et al., 2022]. However, these hybrid solvers are limited in two ways: they, firstly, only combine the trained surrogate with a *single* numerical solver, and secondly rely on a fixed, heuristic schedule. Our proposed method addresses these shortcomings, resulting in significant empirical improvements (Section 7).

Model Routing: Routing [Shnitzer et al., 2023, Hu et al., 2024, Ding et al., 2024, Huang et al., 2025] selects, for each input, a model from a fixed set to optimize a task metric under cost or latency constraints. Simple heuristics–e.g., thresholding a cheap model's uncertainty estimates [Chuang et al., 2024, 2025]–often poorly balance the cost-accuracy tradeoff, motivating many systems to instead learn a router that maps inputs to the best model [Hari and Thomson, 2023, Mohammadshahi et al., 2024, Šakota et al., 2024]. We similarly learn a router, but over numerical and neural solvers for PDEs at the iteration level. Our work is the first to apply routing to the problem of learning hybrid PDE solvers, in contrast to prior approaches that use fixed schedules. Additionally, by exploiting

the algebraic structure of PDE solvers, we derive theoretical guarantees for our routing strategy (Section 4), unlike typical model-routing settings.

Mixture of Experts: Classical mixture-of-experts (MoE) [Jacobs et al., 1991, Jordan and Jacobs, 1994] uses a gating network to combine the outputs of multiple experts, trained jointly with the gate. In modern LLMs, MoE instead performs sparse, token-level routing to a subset of in-layer experts, enabling large capacity without proportional compute, typically with load-balancing mechanisms [Shazeer et al., 2017, Lepikhin et al., 2020, Fedus et al., 2022, Jiang et al., 2024]. Our method can be regarded as gating of a different kind: "experts" (solvers) are not jointly trained with the router, and a single router is reused across iterations, unlike MoE which commonly employs layer-specific gates.

7 Experiments

We empirically demonstrate the fast, uniform convergence of the approximate greedy router on the Poisson and Convection-Diffusion (ConvDiff) equations posed on the unit domain $D = [0, 1]^2$:

$$\forall\, x \in D \qquad \textbf{Poisson: } -\Delta u(\mathbf{x}) = f(\mathbf{x}) \quad \textbf{ConvDiff: } -\Delta u(\mathbf{x}) + 20\frac{\partial}{\partial x_1}u(\mathbf{x}) + 20\frac{\partial}{\partial x_2}u(\mathbf{x}) = f(\mathbf{x})$$

We impose periodic boundary conditions for both equations, and center f to satisfy the compatibility condition $\int_D f(\mathbf{x})d\mathbf{x} = 0$; Dirichlet results are deferred to Section D.8 . The domain D is discretized on a 31×31 uniform grid; results on a finer grid are provided in Section D.7. Data is sampled from a zero-mean Hierarchical Gaussian Random Field on the periodic domain with covariance operator $\alpha(-\Delta + \beta I)^{-\gamma}$, where α, β, γ vary across samples (See Section C). We run two experimental settings. In the first, we restrict to solver pairs ($K = 2$) and compare against HINTS and standalone classical solvers. In the second, we assess performance as the solver ensemble grows; since HINTS does not support this setting, we simply compare against individual solvers. In both cases, we consider a range of iterative methods described below. Performance is reported over 128 test samples via the mean final error $\|e_h^{(T)}\|_2$ and the mean area under the curve (AUC), defined as $\sum_{t=1}^{T} \|e_h^{(t)}\|_2$; the "curve" refers to the error over iterations. AUC characterizes the full convergence behavior, which is critical in practice where solutions must be reached in minimal time.

Paired Solver Experiments: We seek to optimally solve the aforementioned PDEs with access to a single numerical scheme and a trained DeepONet. In particular, we consider settings coupling the DeepONet with Jacobi, Gauss-Seidel (GS), and Symmetric Gauss-Seidel (SymGS) solvers. For each setup, we compare our learned router against the single-solver schedules (Jacobi only, GS only, and SymGS only) and HINTS variants (HINTS-Jacobi, HINTS-GS, HINTS-SymGS), where the DeepONet correction are applied every 24 iterations. We train LSTM-based routers using the loss in Equation (10) separately for each pairwise settings. The LSTM captures the sequential nature of routing decisions where the benefit of each action depends on the error trajectory and past choices. Training details of the DeepONet and the routers can be found in Section C. To ensure fairness, comparisons are restricted to methods with identical solver access, and are grouped by solver family (e.g., Jacobi-related methods). In addition, we include a *true greedy* oracle, which selects at each iteration the solver that yields the largest immediate error reduction. This oracle has access to the ground-truth error and is therefore not implementable in practice, but helps evaluate how well the learned router approximates the optimal policy. Each method is run for 300 iterations.

As shown in Table 1, the greedy router achieves the lowest final error and AUC across all solver families, outperforming their single-solver and HINTS counterparts. Moreover, its performance closely tracks that of the true greedy oracle, indicating that our surrogate-based training procedure effectively approximates the greedy strategy. This can be further seen in the routing visualizations in Section D.6, which demonstrate that the router learns qualitatively similar strategies to the oracle. The improvements over HINTS are particularly notable: while HINTS can improve over single-solver schedules, it exhibits oscillatory error behavior (See Figure 1) due to invoking the DeepONet at suboptimal times. In contrast, the greedy routers selectively applies updates that reduce error, resulting in near-monotone convergence. These gains arise from adapting routing decisions across *both* samples for a given PDE and PDEs. For instance, the neural operator is used more frequently in the convection–diffusion setting than in Poisson (Figure 2), a behavior that must be manually specified in HINTS but is learned automatically by our method. This shows that improvements stem from selective use of the learned component, rather than frequent invocation as in HINTS.

Table 1: Final error and AUC of L_2^2 error (lower is better). Values are mean ($\pm$ standard error) over 128 test instances; Error is reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Bold indicates the best method within each solver family. True-Greedy denotes an oracle policy with access to exact error information

Equation	Poisson		ConvDiff	
Methods	$\|e_h^{(T)}\| \times 10^3$	AUC	$\|e_h^{(T)}\| \times 10^3$	AUC
Jacobi-related Solvers				
Jacobi Only	0.383 (1.029)	0.821 (2.154)	0.136 (0.364)	0.312 (0.788)
HINTS-Jacobi	0.759 (0.016)	0.393 (0.590)	0.447 (0.013)	0.202 (0.256)
Learned Greedy-Jacobi	**0.054 (0.142)**	**0.165 (0.368)**	**0.033 (0.097)**	**0.098 (0.239)**
True-Greedy-Jacobi	*0.021 (0.018)*	*0.094 (0.170)*	*0.012 (0.012)*	*0.049 (0.091)*
GS-related Solvers				
GS only	0.019 (0.051)	0.435 (1.141)	$< \mathbf{10^{-3}}$	0.088 (0.219)
HINTS-GS	0.724 (0.000)	0.325 (0.488)	0.456 (0.000)	0.130 (0.154)
Learned Greedy-GS	**0.002 (0.004)**	**0.083 (0.152)**	$< \mathbf{10^{-3}}$	**0.031 (0.078)**
True-Greedy-GS	*0.001 (0.001)*	*0.057 (0.107)*	*$< 10^{-3}$*	*0.019 (0.038)*
SymGS-related Solvers				
SymGS only	$< \mathbf{10^{-3}}$	0.227 (0.593)	$< \mathbf{10^{-3}}$	0.071 (0.178)
HINTS-SymGS	0.709 (0.000)	0.252 (0.380)	0.463 (0.000)	0.116 (0.131)
Learned Greedy-SymGS	$< \mathbf{10^{-3}}$	**0.052 (0.102)**	$< \mathbf{10^{-3}}$	**0.022 (0.039)**
True-Greedy-SymGS	*$< 10^{-3}$*	*0.034 (0.066)*	*$< 10^{-3}$*	*0.016 (0.032)*

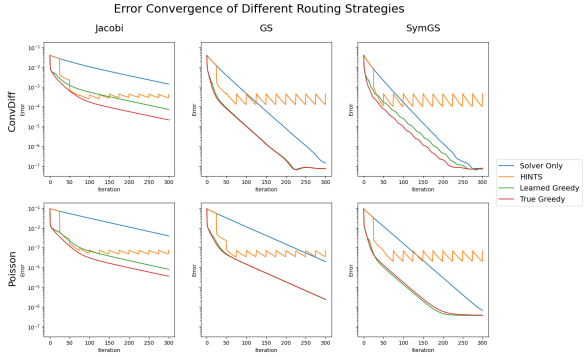

Figure 1: Convergence histories for representative test instances. Rows: Convection Diffusion (top) and Poisson (bottom). Columns: Jacobi, Gauss–Seidel (GS), and Symmetric Gauss-Seidel (SymGS). Greedy yields near-monotone decay and the lowest errors, whereas HINTS shows sawtooth behaviors.

Solver Ensembles Experiments: We consider routing over a collection of more than two solvers. Specifically, we study ensembles $\mathcal{W}$ of increasing size, consisting of a trained DeepONet and a diverse set of numerical solvers, including Jacobi, GS, SymGS, Damped Jacobi ($\omega = 0.67$) and Successive Over-relaxation (SOR) with $\omega = 1.5$. Our goal is to evaluate whether enlarging the solver set yields gains beyond pairwise configurations. To this end, we compare $\mathrm{Router}(\mathrm{NO} \cup \mathcal{W})$ against the best pairwise version of our method, $\mathrm{Router}(\mathrm{NO} \cup \{\text{solver}\})$ for solver $\in \mathcal{W}$. As shown in Table 3, larger ensembles often yield improvements over both final error and AUC relative to

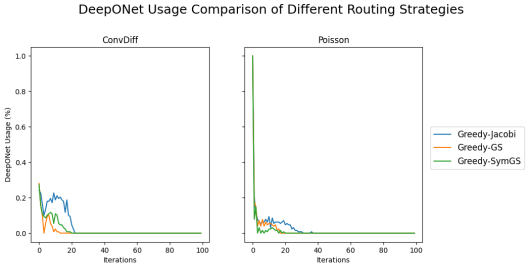

Figure 2: DeepONet usage across iterations for different greedy routing strategies on convection–diffusion (left) and Poisson (right). The variability across iterations highlights the non-uniform, sample-dependent nature of the learned routing strategy. See Section D.6 for detailed routing trajectories. For readability, we display the first 100 iterations (out of 300 total).

the best pairwise configuration, indicating that additional solver diversity can be leveraged. The router achieves these gains by learning to route to each member of the ensemble for a non-trivial proportion of the iterates, as measured in Section D.6. Finally, despite the increased size of the action space, the learned router continues to produce improvements, demonstrating robustness of both the training procedure and the convex surrogate of Section 5.1; true greedy comparisons are deferred to Section D.1. These results show that our method can scale to richer solver sets, enabling improved performance without sacrificing learnability.

Wall-clock time-to-solution: Iteration counts favor methods whose iterations are expensive, so we complement the results above with an end-to-end wall-clock study on a 128×128 grid (128^2 unknowns) for all five classical solvers and both PDEs, plus a third, harder PDE—anisotropic diffusion $-\epsilon\,\partial^2_{x_1} u - \partial^2_{x_2} u = f$ with $\epsilon = 10^{-2}$ on the same periodic domain and data, the standard case in which point smoothers and multigrid with point smoothing degrade; Section E details the protocol. Three ingredients make the comparison meaningful and the router deployable: (i) all classical solvers are implemented as $O(N^2)$ stencil or cached sparse-triangular sweeps (matching the dense implementations used elsewhere to machine precision), so the classical baseline is not artificially slowed; (ii) the DeepONet corrector acts on the residual through a 64×64 sensor grid with a Fourier trunk basis and is applied scale-equivariantly, so one call solves the smooth half of the spectrum to $\approx 10^{-6}$ at the cost of about 12 Jacobi or 3 Gauss–Seidel sweeps and never injects high-frequency error; and (iii) the router is made *cost-aware* through the cost-equalised macro-actions of Section 4, and the LSTM is replaced by a router over $6+K$ scalar features (log residual, its recent decay rate, the previous action, and call counts) with a two-layer MLP whose decision costs $\approx 5\,\mu$s, trained with the surrogate of Section 5.1 to imitate the cost-aware greedy oracle. Table 2 reports the median wall-clock time to reach truncation-level relative error $\varepsilon = h^2$, the accuracy beyond which further algebraic refinement is not meaningful on an $N \times N$ grid. The learned router reaches it 238×–1983×faster than the classical solver alone and 2.68×–10.3×faster than HINTS with the published period τ=25, in every one of the 10 solver–equation pairings (Table 2); it is also never slower than HINTS with the best period $\tau \in \{5, 10, 25, 50\}$ chosen *a posteriori* on the test set for that pairing (1.21×–3.22×; Table 17), whereas the best τ itself varies across pairings and tolerances. The router tracks the cost-aware oracle within 1.06×in median time, and the iteration-based metrics of Table 1 carry over to the finer grid (Table 18). Figure 3 shows where the gain comes from: the router calls the corrector once at the very start (where the error is smooth and the corrector removes four orders of magnitude at once), spends a solver-dependent number of sweeps on the high-frequency band, and calls it again only if the smooth error re-emerges—between one and three calls in total—while HINTS spends $\tau-1$ sweeps before its first call and keeps calling the corrector when it no longer helps. At tolerances far below h^2 the undamped-Jacobi pairing is limited by the near-Nyquist modes that neither component damps (Table 21), but the router degrades gracefully there because it stops selecting the corrector, whereas fixed schedules keep paying for it (Section E). Against the classical methods a practitioner would use on these problems (Table 25), the router with its best stationary pairing is 3.62×–3.71× faster than geometric multigrid alone, 3.85×–5.31× faster than the best multigrid-preconditioned Krylov method, and only the FFT direct solve—a method specific to constant-coefficient periodic problems, which we use to compute ground truth—is faster, by 2.87×–2.91×; routing over $\{\mathrm{NO}, \text{multigrid}\}$ is itself 1.93×–2.03× faster than multigrid alone. The gains persist on 256^2 and 512^2 grids (Table 26) and all comparisons are significant at $p < 0.01$ (paired Wilcoxon over instances) and stable across five independent router-training seeds (Section E.5). The same construction extends to solver ensembles (Table 23): over the four ensembles of Table 3 and both PDEs, the router over $\mathrm{NO} \cup \mathcal{W}$ matches the best pairwise router of its members to within $\pm 15\%$ (ratio of medians 0.82×–1.14×) without knowing in advance which member is fastest, and is 380×–1512× faster than the best member solver alone; the remaining gap to the ensemble oracle, which is faster than every pairwise router, is imitation error in choosing among near-equivalent smoothers (Section E).

Additional experiments are provided in Section D.

8 Discussion

We have introduced an adaptive method for selecting solvers that efficiently minimizes the final error of a PDE in iterative methods, opening many directions for future work. Our current DeepONet is trained without accounting for its downstream role in correction term prediction; jointly training the

Table 2: Median wall-clock time to truncation-level relative error $\varepsilon = h^2$ on the 128×128 grid, 64 test instances per cell (paired median speedup over the solver-only baseline in parentheses). "HINTS (best τ)" is the best of $\tau \in \{5, 10, 25, 50\}$ chosen per cell on the test set itself. The greedy oracle is the cost-agnostic Algorithm 1, the cost-aware oracle is Algorithm 1 on cost-equalised macro-actions; both use the true error and are not deployable. Bold marks the fastest deployable method in each row; the learned router pays for all of its features and decisions.

Equation	Solver	Solver only	HINTS (τ=25)	HINTS (best τ)	Greedy oracle	Cost-aware oracle	Learned router (ours)
Poisson	Jacobi	1.15 s	2.65 ms (412×)	1.10 ms (984×; τ=5)	854 μs (1283×)	*942 μs (1199×)*	**987 μs (1154×)**
	Jacobi (0.67)	1.62 s	2.71 ms (621×)	1.13 ms (1464×; τ=5)	915 μs (1904×)	*922 μs (1836×)*	**912 μs (1863×)**
	GS	1.83 s	6.65 ms (277×)	1.99 ms (906×; τ=5)	960 μs (1872×)	*953 μs (1892×)*	**1.01 ms (1809×)**
	SymGS	1.52 s	10.3 ms (149×)	2.60 ms (585×; τ=5)	1.09 ms (1395×)	*1.13 ms (1330×)*	**1.13 ms (1331×)**
	SOR (1.5)	600.9 ms	12.2 ms (49.4×)	3.37 ms (180×; τ=5)	1.03 ms (583×)	*1.02 ms (582×)*	**1.02 ms (576×)**
ConvDiff	Jacobi	1.51 s	3.44 ms (436×)	1.28 ms (1145×; τ=5)	992 μs (1532×)	*1.15 ms (1319×)*	**1.14 ms (1317×)**
	Jacobi (0.67)	2.22 s	3.40 ms (656×)	1.31 ms (1670×; τ=5)	1.10 ms (2036×)	*1.08 ms (1974×)*	**1.10 ms (1983×)**
	GS	1.63 s	8.27 ms (198×)	3.60 ms (453×; τ=5)	1.23 ms (1341×)	*1.32 ms (1222×)*	**1.35 ms (1243×)**
	SymGS	1.56 s	11.1 ms (141×)	2.96 ms (529×; τ=5)	1.25 ms (1254×)	*1.27 ms (1236×)*	**1.28 ms (1211×)**
	SOR (1.5)	383.6 ms	13.6 ms (28.0×)	3.64 ms (104×; τ=5)	1.39 ms (273×)	*1.63 ms (240×)*	**1.59 ms (238×)**

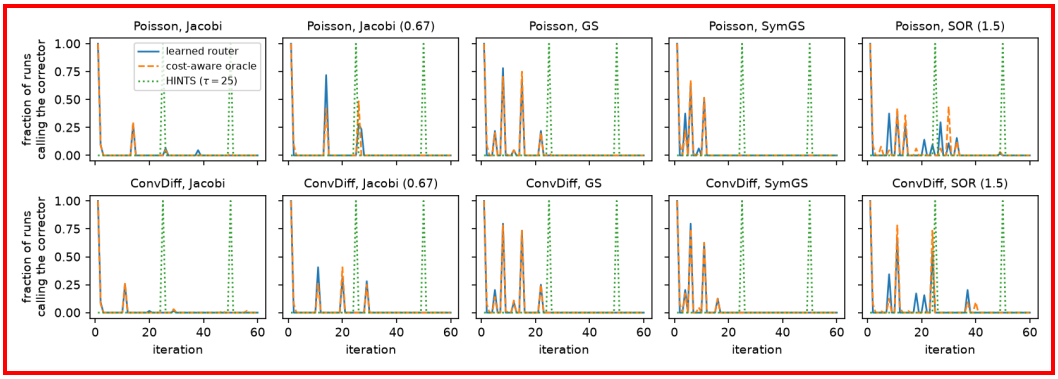

Figure 3: Corrector usage on the 128×128 grid: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS (τ=25), for every solver pairing (columns) and both equations (rows).

ML solver and router could yield larger gains. Another avenue is to use reinforcement learning to learn a cost-aware routing strategy that optimizes error under compute budgets; Sections 4 and E show that a myopic cost-aware variant of the greedy rule—Algorithm 1 on cost-equalised macro-actions—already delivers uniform wall-clock gains, and RL could extend it to non-myopic budgeted routing. RL would additionally enable extending the ensemble routing to *continuous* parameterization of solvers. Finally, framing routing as an online learning problem could enable adaptation to unknown deployment conditions.

References

Eugenia Kalnay. *Atmospheric modeling, data assimilation and predictability*. Cambridge university press, 2003.

Peter Bauer, Alan Thorpe, and Gilbert Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525(7567):47–55, 2015.

Andrew Staniforth and Jean Côté. Semi-lagrangian integration schemes for atmospheric models—a review. *Monthly weather review*, 119(9):2206–2223, 1991.

John David Anderson. *Hypersonic and high temperature gas dynamics*. Aiaa, 1989.

Fredi Tröltzsch. *Optimal control of partial differential equations: theory, methods, and applications*, volume 112. American Mathematical Soc., 2010.

Sylvain Baillet, John C Mosher, and Richard M Leahy. Electromagnetic brain mapping. *IEEE Signal processing magazine*, 18(6):14–30, 2001.

Table 3: Final error and AUC of squared L^2 error for ensembles of increasing size. $\mathcal{W}$ denotes the set of numerical solvers and NO the neural operator (DeepONet). Each $\mathrm{Router}(\mathrm{NO} \cup \mathcal{W})$ is compared against the best pairwise routing using our method, i.e., $\mathrm{Router}(\mathrm{NO} \cup \{\text{solver}\})$ for solver $\in \mathcal{W}$. Values are mean ($\pm$ s.e.) over 128 test instances; Error is reported in $\times 10^{-3}$. p-values from paired one-sided t-tests comparing the AUC of the ensemble to that of the best pairwise router.

$\mathcal{W}$	**Method**	**Poisson**			**ConvDiff**		
		$\|e_h^{(T)}\| \times 10^3$	AUC	p-value (vs pairwise)	$\|e_h^{(T)}\| \times 10^3$	AUC	p-value (vs pairwise)
{Jacobi, GS}	Best $\mathrm{Router}(\mathrm{NO} \cup \{\text{solver}\})$	0.002 (0.004)	0.083 (0.152)	-	$< 10^{-3}$	0.031 (0.078)	-
	$\mathrm{Router}(\mathrm{NO} \cup \mathcal{W})$	0.002 (0.002)	0.078 (0.132)	$< 10^{-3}$	$< 10^{-3}$	0.021 (0.040)	0.004
{Jacobi, GS, SymGS}	Best $\mathrm{Router}(\mathrm{NO} \cup \{\text{solver}\})$	$< 10^{-3}$	0.052 (0.102)	-	$< 10^{-3}$	0.022 (0.039)	-
	$\mathrm{Router}(\mathrm{NO} \cup \mathcal{W})$	$< 10^{-3}$	0.041 (0.091)	-$< 10^{-3}$	$< 10^{-3}$	0.018 (0.035)	$< 10^{-3}$
{Jacobi, GS, SymGS, Jacobi (0.67)}	Best $\mathrm{Router}(\mathrm{NO} \cup \{\text{solver}\})$	$< 10^{-3}$	0.052 (0.102)	-	$< 10^{-3}$	0.022 (0.039)	-
	$\mathrm{Router}(\mathrm{NO} \cup \mathcal{W})$	0.002 (0.003)	0.046 (0.083)	0.004	$< 10^{-3}$	0.019 (0.037)	0.002
{Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	Best $\mathrm{Router}(\mathrm{NO} \cup \{\text{solver}\})$	$< 10^{-3}$	0.052 (0.143)	-	$< 10^{-3}$	0.011 (0.030)	-
	$\mathrm{Router}(\mathrm{NO} \cup \mathcal{W})$	$< 10^{-3}$	0.043 (0.082)	0.03	$< 10^{-3}$	0.008 (0.019)	0.004

Roberta Grech, Tracey Cassar, Joseph Muscat, Kenneth P Camilleri, Simon G Fabri, Michalis Zervakis, Petros Xanthopoulos, Vangelis Sakkalis, and Bart Vanrumste. Review on solving the inverse problem in eeg source analysis. *Journal of neuroengineering and rehabilitation*, 5:1–33, 2008.

Gordon D Smith. *Numerical solution of partial differential equations: finite difference methods*. Oxford university press, 1985.

Randall J LeVeque. *Finite difference methods for ordinary and partial differential equations: steady-state and time-dependent problems*. SIAM, 2007.

Thomas JR Hughes. *The finite element method: linear static and dynamic finite element analysis*. Courier Corporation, 2003.

Klaus-Jürgen Bathe. *Finite element procedures*. Klaus-Jurgen Bathe, 2006.

Susanne C Brenner. *The mathematical theory of finite element methods*. Springer, 2008.

Yousef Saad. *Iterative methods for sparse linear systems*. SIAM, 2003.

Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

Tianping Chen and Hong Chen. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. *IEEE transactions on neural networks*, 6(4):911–917, 1995.

Lizuo Liu and Wei Cai. Multiscale deeponet for nonlinear operators in oscillatory function spaces for building seismic wave responses. *arXiv preprint arXiv:2111.04860*, 2021.

Xinliang Liu, Bo Xu, Shuhao Cao, and Lei Zhang. Mitigating spectral bias for the multiscale operator learning. *Journal of Computational Physics*, 506:112944, 2024.

Zhilin You, Zhenli Xu, and Wei Cai. Mscalefno: Multi-scale fourier neural operator learning for oscillatory function spaces. *arXiv preprint arXiv:2412.20183*, 2024.

Siavash Khodakarami, Vivek Oommen, Aniruddha Bora, and George Em Karniadakis. Mitigating spectral bias in neural operators via high-frequency scaling for physical systems. *arXiv preprint arXiv:2503.13695*, 2025.

Zhi-Qin John Xu, Lulu Zhang, and Wei Cai. On understanding and overcoming spectral biases of deep neural network learning methods for solving pdes. *arXiv preprint arXiv:2501.09987*, 2025.

Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred Hamprecht, Yoshua Bengio, and Aaron Courville. On the spectral bias of neural networks. In *International conference on machine learning*, pages 5301–5310. PMLR, 2019.

Zhi-Qin John Xu, Yaoyu Zhang, Tao Luo, Yanyang Xiao, and Zheng Ma. Frequency principle: Fourier analysis sheds light on deep neural networks. *arXiv preprint arXiv:1901.06523*, 2019.

Tao Luo, Zheng Ma, Zhi-Qin John Xu, and Yaoyu Zhang. Theory of the frequency principle for general deep neural networks. *arXiv preprint arXiv:1906.09235*, 2019.

Enrui Zhang, Adar Kahana, Alena Kopaničáková, Eli Turkel, Rishikesh Ranade, Jay Pathak, and George Em Karniadakis. Blending neural operators and relaxation methods in pde numerical solvers. *Nature Machine Intelligence*, pages 1–11, 2024.

John P Boyd. *Chebyshev and Fourier spectral methods*. Courier Corporation, 2001.

Claudio Canuto, M. Yousuff Hussaini, Alfio Quarteroni, and Thomas A. Zang. *Spectral Methods: Fundamentals in Single Domains*. Springer, 2006. ISBN 9783540307264.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020a.

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Andrew Stuart, Kaushik Bhattacharya, and Anima Anandkumar. Multipole graph neural operator for parametric partial differential equations. *Advances in Neural Information Processing Systems*, 33:6755–6766, 2020b.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.

George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical programming*, 14(1):265–294, 1978.

Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2017)*, pages 19–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

Louenas Bounia and Frederic Koriche. Approximating probabilistic explanations via supermodular minimization. In *Uncertainty in Artificial Intelligence*, pages 216–225. PMLR, 2023.

Abhimanyu Das and David Kempe. Approximate submodularity and its applications: Subset selection, sparse approximation and dictionary selection. *Journal of Machine Learning Research*, 19(3): 1–34, 2018.

Uriel Feige, Vahab S Mirrokni, and Jan Vondrák. Maximizing non-monotone submodular functions. *SIAM Journal on Computing*, 40(4):1133–1153, 2011.

Andrew An Bian, Joachim M Buhmann, Andreas Krause, and Sebastian Tschiatschek. Guarantees for greedy maximization of non-submodular functions with applications. In *International conference on machine learning*, pages 498–507. PMLR, 2017.

Chris Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. Submodular maximization beyond non-negativity: Guarantees, fast algorithms, and applications. In *International Conference on Machine Learning*, pages 2634–2643. PMLR, 2019.

Matthew Streeter and Daniel Golovin. An online algorithm for maximizing submodular functions. *Advances in Neural Information Processing Systems*, 21, 2008.

Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. Maximizing sequence-submodular functions and its application to online advertising. *Management Science*, 67(10):6030–6054, 2021.

Zhenliang Zhang, Edwin KP Chong, Ali Pezeshki, and William Moran. String submodular functions with curvature constraints. *IEEE Transactions on Automatic Control*, 61(3):601–616, 2015.

Sara Bernardini, Fabio Fagnani, and Chiara Piacentini. Through the lens of sequence submodularity. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30, pages 38–47, 2020.

Brandon Van Over, Bowen Li, Edwin KP Chong, and Ali Pezeshki. A performance bound for the greedy algorithm in a generalized class of string optimization problems. *arXiv preprint arXiv:2409.05020*, 2024.

Sebastian Tschiatschek, Adish Singla, and Andreas Krause. Selecting sequences of items via submodular maximization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.

Henry Gouk, Eibe Frank, Bernhard Pfahringer, and Michael J Cree. Regularisation of neural networks by enforcing lipschitz continuity. *Machine Learning*, 110(2):393–416, 2021.

Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural computation*, 1(2):270–280, 1989.

Alex M Lamb, Anirudh Goyal ALIAS PARTH GOYAL, Ying Zhang, Saizheng Zhang, Aaron C Courville, and Yoshua Bengio. Professor forcing: A new algorithm for training recurrent networks. *Advances in neural information processing systems*, 29, 2016.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. *Advances in neural information processing systems*, 28, 2015.

Anqi Mao, Mehryar Mohri, and Yutao Zhong. Regression with multi-expert deferral. In *International Conference on Machine Learning*, pages 34738–34759. PMLR, 2024.

Ali Taghibakhshi, Scott MacLachlan, Luke Olson, and Matthew West. Optimization-based algebraic multigrid coarsening using reinforcement learning. *Advances in neural information processing systems*, 34:12129–12140, 2021.

Matteo Caldana, Paola F Antonietti, et al. A deep learning algorithm to accelerate algebraic multigrid methods in finite element solvers of 3d elliptic pdes. *Computers & Mathematics with Applications*, 167:217–231, 2024.

Alena Kopaničáková and George Em Karniadakis. Deeponet based preconditioning strategies for solving parametric linear systems of equations. *SIAM Journal on Scientific Computing*, 47(1): C151–C181, 2025.

Ru Huang, Ruipeng Li, and Yuanzhe Xi. Learning optimal multigrid smoothers via neural networks. *SIAM Journal on Scientific Computing*, 45(3):S199–S225, 2022.

Alexandr Katrutsa, Talgat Daulbaev, and Ivan Oseledets. Black-box learning of multigrid parameters. *Journal of Computational and Applied Mathematics*, 368:112524, 2020.

Adar Kahana, Enrui Zhang, Somdatta Goswami, George Karniadakis, Rishikesh Ranade, and Jay Pathak. On the geometry transferability of the hybrid iterative numerical solver for differential equations. *Computational Mechanics*, 72(3):471–484, 2023.

Pengzhan Jin, Shuai Meng, and Lu Lu. Mionet: Learning multiple-input operators via tensor product. *SIAM Journal on Scientific Computing*, 44(6):A3490–A3514, 2022.

Jun Hu and Pengzhan Jin. A hybrid iterative method based on mionet for pdes: Theory and numerical examples. *Mathematics of Computation*, 2025.

Chen Cui, Kai Jiang, Yun Liu, and Shi Shu. Fourier neural solver for large sparse linear algebraic systems. *Mathematics*, 10(21):4014, 2022.

Tal Shnitzer, Anthony Ou, Mírian Silva, Kate Soule, Yuekai Sun, Justin Solomon, Neil Thompson, and Mikhail Yurochkin. Large language model routing with benchmark datasets. *arXiv preprint arXiv:2309.15789*, 2023.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. Routerbench: A benchmark for multi-llm routing system. *arXiv preprint arXiv:2403.12031*, 2024.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks VS Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query routing. *arXiv preprint arXiv:2404.14618*, 2024.

Zhongzhan Huang, Guoming Ling, Yupei Lin, Yandong Chen, Shanshan Zhong, Hefeng Wu, and Liang Lin. Routereval: A comprehensive benchmark for routing llms to explore model-level scaling up in llms. *arXiv preprint arXiv:2503.10657*, 2025.

Yu-Neng Chuang, Prathusha Kameswara Sarma, Parikshit Gopalan, John Boccio, Sara Bolouki, Xia Hu, and Helen Zhou. Learning to route llms with confidence tokens. *arXiv preprint arXiv:2410.13284*, 2024.

Yu-Neng Chuang, Leisheng Yu, Guanchu Wang, Lizhe Zhang, Zirui Liu, Xuanting Cai, Yang Sui, Vladimir Braverman, and Xia Hu. Confident or seek stronger: Exploring uncertainty-based on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*, 2025.

Surya Narayanan Hari and Matt Thomson. Tryage: Real-time, intelligent routing of user prompts to large language models. *arXiv preprint arXiv:2308.11601*, 2023.

Alireza Mohammadshahi, Arshad Rafiq Shaikh, and Majid Yazdani. Routoo: Learning to route to large language models effectively. *arXiv preprint arXiv:2401.13979*, 2024.

Marija Šakota, Maxime Peyrard, and Robert West. Fly-swat or cannon? cost-effective language model choice via meta-modeling. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, pages 606–615, 2024.

Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.

Michael I Jordan and Robert A Jacobs. Hierarchical mixtures of experts and the em algorithm. *Neural computation*, 6(2):181–214, 1994.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and applications. In *International conference on Machine learning*, pages 23803–23828. pmlr, 2023.

Michael C Mozer. A focused backpropagation algorithm for temporal pattern recognition. In *Backpropagation*, pages 137–169. Psychology Press, 2013.

Anthony J Robinson and Frank Fallside. *The utility driven dynamic error propagation network*, volume 11. University of Cambridge Department of Engineering Cambridge, 1987.

Paul J Werbos. Generalization of backpropagation with application to a recurrent gas market model. *Neural networks*, 1(4):339–356, 1988.

Lu Lu, Xuhui Meng, Shengze Cai, Zhiping Mao, Somdatta Goswami, Zhongqiang Zhang, and George Em Karniadakis. A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data. *Computer Methods in Applied Mechanics and Engineering*, 393:114778, 2022.

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635. JMLR Workshop and Conference Proceedings, 2011.

A Proofs for Section 4

A.1 Proof of Proposition A.1

Proposition A.1. *Any prefix monotonically non-increasing sequence supermodular function g is weakly supermodular with respect to all sequences $S \in \Omega^*$ with $\alpha(S) = 1$*

Proof. This proof is adapted from Liberty and Sviridenko [2017].

If g is sequence supermodular then,

$$\begin{aligned}
&g(S) - g(S \oplus S') \\
&= \sum_{i=1}^{|S'|} g(S \oplus (S'_1, \ldots, S'_{i-1})) - g(S \oplus (S'_1, \ldots, S'_i)) \\
&\overset{(a)}{\leq} \sum_{i=1}^{|S'|} g(S) - g(S \oplus S'_i) \\
&\leq |S'| \max_{i \in [|S'|]} g(S) - g(S \oplus S'_i)
\end{aligned}$$

(a) by supermodularity

□

A.2 Proof of Theorem 4.1

Theorem 4.1. *Let $g : \Omega^* \to \mathbb{R}$ be a weakly supermodular function with respect to the optimal solution $O = \arg\min_{S \in \Omega^T} h(S)$ with a supermodularity ratio of $\alpha(O)$ and postfix monotonicity. Let the greedy solution of length T be S^T. If $\phi_T(\alpha) = \left(1 - \frac{1}{\alpha T}\right)^T$, then*

$$g(S^T) \leq \left(1 - \phi_T(\alpha(O))\right) g(O) + \phi_T(\alpha(O)) g(\emptyset)$$

Proof. This proof strategy is inspired by Streeter and Golovin [2008] and Liberty and Sviridenko [2017]

$$\begin{aligned}
g(S^t) - g(O) &\overset{(a)}{\leq} g(S^t) - g(S^t \oplus O) \\
&= \sum_{i=1}^{|O|} g(S^t \oplus (o_1, \ldots o_{i-1})) - g(S^t \oplus (o_1, \ldots, o_i)) \\
&\overset{(b)}{\leq} \alpha(O) \sum_{i=1}^{|O|} g(S^t) - g(S^t \oplus o_i) \\
&\leq \alpha(O)|O| \max_{i \in [|O|]} g(S^t) - g(S^t \oplus o_i) \\
&\leq \alpha(O)T g(S^t) - \alpha(O)T \min_{\omega \in \Omega} g(S^t \oplus \omega) \\
&= \alpha(O)T g(S^t) - \alpha(O)T g(S^{t+1})
\end{aligned}$$

(a) by μ- postfix monotonicity, (b) by supermodularity.

After rearranging the inequality, we get:

$$\begin{aligned}
g(S^{t+1}) &\leq \frac{1}{\alpha(O)T} \left(g(O) - (\alpha(O)T - 1)\, g(S^t)\right) \\
&= \frac{1}{\alpha(O)T} g(O) + \left(1 - \frac{1}{\alpha(O)T}\right) g(S^t)
\end{aligned}$$

When recursively applying this inequality, we get:

$$
\begin{aligned}
g(S^T) &\leq \frac{1}{\alpha(O)T} g(O) \sum_{i=0}^{T-1} \left(1 - \frac{1}{\alpha(O)T}\right)^i + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset) \\
&= \left(1 - \left(1 - \frac{1}{\alpha(O)T}\right)^T\right) g(O) + \left(1 - \frac{1}{\alpha(O)T}\right)^T g(\emptyset)
\end{aligned}
$$

□

A.3 Proof of Proposition 4.2

Proposition 4.2. *Suppose that for all $j \in [K]$, the error propagation function $I - C_j \circ \mathcal{L}_h^a$ is ρ_j-Lipschitz continuous with $\rho_j < 1$, and that $(I - C_j \circ \mathcal{L}_h^a)(0_N) = 0_N$. Then, the function h is weakly supermodular with respect to the optimal solution O, with*

$$
\alpha(O) = \max\left\{\frac{4}{T - \sum_{i=1}^T \rho_{O_i}^2}, 1\right\}
$$

Furthermore, if $I - C_j \circ \mathcal{L}_h^a$ is invertible for all $j \in [K]$, h is also postfix monotonically non-increasing.

Proof. For brevity, we use the notation $(g_1 \circ \cdots \circ g_T)(x) = \circ_{t=1}^T g_t(x)$, where composition is applied from right to left so that g_T acts first. In this proof, we use a few properties of Lipschitz continuous functions:

- **Property 1:** If g is ρ-Lipschitz continuous and $g(0) = 0$, then $\|g(x)\|_2 = \|g(x) - 0\|_2 = \|g(x) - g(0)\|_2 \leq \rho\|x - 0\|_2 = \rho\|x\|_2$
- **Property 2:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1 and ρ_2 respectively, the Lipschitz constant of $g_1 + g_2$ and $g_1 - g_2$ is $\rho_1 + \rho_2$.
- **Property 3:** If g_1 and g_2 are Lipschitz continuous functions with Lipschitz constants of ρ_1 and ρ_2 respectively, the Lipschitz constant of $g_1 \circ g_2$ is $\rho_1\rho_2$.

In order to prove weakly-α-supermodularity, we must first prove prefix monotonicity.

Prefix monotonicity: Let $S \preceq S'$ where $S' = S \oplus N$.

$$
\begin{aligned}
h(S') &= \left\| \circ_{t=|S'|}^1 \left(I - C_{S'_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&= \left\| \circ_{t=|S'|}^{|S|+1} \left(I - C_{S'_t} \circ \mathcal{L}_h^a\right) \circ \circ_{t=|S|}^1 \left(I - C_{S'_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&\overset{(a)}{\leq} \left(\prod_{t=|S|+1}^{|S'|} \rho_{S'_t}^2 \right) \left\| \circ_{t=|S|}^1 \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&\leq h(S)
\end{aligned}
$$

(a) by Property 1

Weak supermodularity: We will upper bound $\alpha(O)$ by providing an upper bound for $h(S) - h(S \oplus O)$ and a lower bound for $\sum_{i=1}^{T} h(S) - h(S \oplus O_i)$.

$$
\begin{aligned}
h(S) - h(S \oplus O) &= \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 - \left\| \circ_{t=|O|}^{1} \left(I - C_{O_t} \circ \mathcal{L}_h^a\right) \circ \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&\overset{(a)}{\leq} \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) - \circ_{t=|O|}^{1} \left(I - C_{O_t} \circ \mathcal{L}_h^a\right) \circ \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&= \left\| \left(I - \circ_{t=|O|}^{1} \left(I - C_{O_t} \circ \mathcal{L}_h^a\right)\right) \circ \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&\overset{(b)}{\leq} \left(1 + \prod_{t=1}^{|O|} \rho_{O_t}\right)^2 \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&\overset{(c)}{\leq} 4 \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2
\end{aligned}
$$

(a) by reverse triangle property and $h(S) - h(S \oplus S') > 0$ by prefix monotonicity, (b) since the Lipschitz constant of $I - \circ_{t=|S'|}^{1} \left(I - C_{S'_t} \circ \mathcal{L}_h^a\right)$ is $1 + \prod_{t=1}^{|S'|} \rho_{S'_t}$ by Property 2 and 3, (c) since $\rho_j < 1$

To lower bound $\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i)$

$$
\begin{aligned}
\sum_{i=1}^{|O|} h(S) - h(S \oplus O_i) &= \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 - \left\| \left(I - C_{O_i} \circ \mathcal{L}_h^a\right) \circ \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&\geq \sum_{i=1}^{|O|} \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 - \rho_{O_i}^2 \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \\
&= \left\| \circ_{t=|S|}^{1} \left(I_N - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \left(T - \sum_{i=1}^{T} \rho_{O_i}^2\right)
\end{aligned}
$$

Finally,

$$
\begin{aligned}
\frac{h(S) - h(S \oplus O)}{T \max_i h(S) - h(S \oplus O_i)} &\leq \max \left\{ \frac{4 \left\| \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2}{\left\| \circ_{t=|S|}^{1} \left(I_N - C_{S_t} \circ \mathcal{L}_h^a\right) \left(e_h^{(0)}\right) \right\|_2^2 \left(T - \sum_{i=1}^{T} \rho_{O_i}^2\right)}, 1 \right\} \\
&= \max \left\{ \frac{4}{T - \sum_{i=1}^{T} \rho_{O_i}^2}, 1 \right\}
\end{aligned}
$$

Postfix monotonicity: Let $S' = S \oplus N$.

$$
\begin{aligned}
h(S') &= \left\| \circ_{t=|S'|}^{1} \left(I - C_{S'_t} \circ \mathcal{L}_h\right) e_h^{(0)} \right\|_2^2 \\
&= \left\| \circ_{t=|N|}^{1} \left(I - C_{N_t} \circ \mathcal{L}_h\right) \circ \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h\right) e_h^{(0)} \right\|_2^2 \\
&\overset{(a)}{=} \left\| \circ_{t=|N|}^{1} \left(I - C_{N_t} \circ \mathcal{L}_h\right) \circ \circ_{t=|S|}^{1} \left(I - C_{S_t} \circ \mathcal{L}_h\right) \circ \left(\circ_{t=|N|}^{1} \left(I - C_{N_t} \circ \mathcal{L}_h\right)\right)^{-1} \circ \circ_{t=|N|}^{1} \left(I - C_{N_t} \circ \mathcal{L}_h\right) e_h^{(0)} \right\|_2^2 \\
&\leq \prod_{t=1}^{|N|} \rho_{N_t}^2 \prod_{t=1}^{|S|} \rho_{S_t}^2 \prod_{t=1}^{|N|} \rho_{N_t}^{-2} \left\| \circ_{t=|N|}^{1} \left(I - C_{N_t} \circ \mathcal{L}_h\right) e_h^{(0)} \right\|_2^2 \\
&\leq h(N)
\end{aligned}
$$

(a) due the invertibility of $I_N - C_j \mathcal{L}_h$

□

A.4 Proof of Theorem A.2

Lemma A.2. *Let* $(I - C_j\mathcal{L}_h^a) = P\Lambda_j P^{-1}$ *where* P *is an orthogonal matrix and* $\Lambda_j = \mathrm{diag}(\lambda_{j1}, \ldots, \lambda_{jN})$. *If* $P^{-1}e_h^{(0)} = z$, *then the following equality holds:*

$$h(S) = \sum_{i=1}^{N} z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S)} \tag{11}$$

where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$

Proof.

$$\begin{aligned}
h(S) &= \left\| \prod_{t=|S|}^{1} \left(I_N - C_{S_t}\mathcal{L}_h^a\right) e_h^{(0)} \right\|^2 \\
&= \left\| \prod_{t=|S|}^{1} \left(P\Lambda_{S_t}P^{-1}\right) e_h^{(0)} \right\|^2 \\
&= \left\| P \prod_{t=|S|}^{1} \Lambda_{S_t}P^{-1} e_h^{(0)} \right\|^2 \\
&= \left\| \prod_{t=|S|}^{1} \Lambda_{S_t}P^{-1} e_h^{(0)} \right\|^2 \\
&= \sum_{i=1}^{N} z_i^2 \prod_{t=T}^{1} \lambda_{S_t i}^2 \\
&= \sum_{i=1}^{N} z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

□

A.5 Proof of Theorem 4.3

Proposition 4.3. *Let* $\|I_N - C_j\mathcal{L}_h^a\| \leq 1$ *for all* $j \in [K]$ *and* $(I - C_j\mathcal{L}_h^a) = P\Lambda_j P^{-1}$. *Then,* h *is supermodular.*

Proof. Let $S \preceq S'$ where $S' = S \oplus B$. By Theorem A.2,

$$h(S) = \sum_{i=1}^{N} z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S)}$$

where $m_j(S) = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=j}$ be the number of times a sequence S calls the solver j. Recall that h is considered sequence supermodular if $\forall S', S \in \Omega^*$ such that $S \preceq S'$, it holds that

$$h(S) - h(S \oplus \omega) \geq h(S') - h(S' \oplus \omega)$$

$$\begin{aligned}
h(S) - h(S \oplus \omega) &= \sum_{i=1}^{N} z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S)} - \sum_{i=1}^{N} z_i^2 \lambda_{\omega i}^2 \prod_{k=1}^{K} \lambda_{ji}^{2m_j(S)} \\
&= \sum_{i=1}^{N} \left(1 - \lambda_{\omega i}^2\right) z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S)}
\end{aligned}$$

Similarly,

$$
\begin{aligned}
h(S') - h(S \oplus \omega) &= \sum_{i=1}^{N} \left(1 - \lambda_{\omega i}^2\right) z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S')} \\
&\overset{(a)}{=} \sum_{i=1}^{N} \left(1 - \lambda_{\omega i}^2\right) z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2(m_j(S)+m_j(B))} \\
&\overset{(b)}{\leq} \sum_{i=1}^{N} \left(1 - \lambda_{\omega i}^2\right) z_i^2 \prod_{j=1}^{K} \lambda_{ji}^{2m_j(S)} \\
&= h(S) - h(S \oplus \omega)
\end{aligned}
$$

(a) Since $m_j(S') = \sum_{t=1}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S'_t=j} + \sum_{t=|S|}^{|S'|} \mathbf{1}_{S'_t=j} = \sum_{t=1}^{|S|} \mathbf{1}_{S_t=k} + \sum_{t=1}^{|B|} 1_{B_t=j} = m_j(S) + m_j(B)$, (b) since $\rho(I_N - C_j \mathcal{L}_h^a) < 1$ and $m_j(B) \geq 0$ for all $j \in [K]$

□

B Proofs for Section 5

B.1 Proof of Theorem B.1

Lemma B.1. *For any set of preconditioning functions $\mathcal{C}$, any discrete operator $\mathcal{L}_h^a$, any router r, any $a_h, f_h, u_h^{(t)}, u_h \in \mathcal{A} \times \mathcal{F} \times \mathcal{U} \times \mathcal{U}$, the following equality holds true:*

$$
\begin{aligned}
l_{\textit{route}}\left(r, a_h, f_h, u_h^{(t)}, u_h\right) = &\sum_{j=1}^{K} \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&- (K-2) \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2
\end{aligned}
$$

Proof. Note that $\sum_{j=1}^{K} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} = K - 1$

$$
\begin{aligned}
l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, u_h\right) &= \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) = j} \\
&= \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 - \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&= \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 - \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&+ (K-1) \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 - (K-1) \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \\
&= \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 - \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&+ \sum_{j=1}^{K} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \\
&- (K-1) \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2
\end{aligned}
$$

$$
\begin{aligned}
&= \sum_{j=1}^{K} \left(\sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 - \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \right) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&- (K-2) \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&= \sum_{j=1}^{K} \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&- (K-2) \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2
\end{aligned}
$$

□

B.2 Proof of Theorem B.2

Proposition B.2. *For any router r defined by $r(a, f, u^{(t)}) = \text{argmax}_{j \in [K]} g_j(a, f, u^{(t)})$, any $a_h \in \mathcal{A}$, $f_h \in \mathcal{F}$, and $u_h^{(t)}, u_h \in \mathcal{U}$, the routing loss l_{route} satisfies:*

$$
\log(2) l_{route} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) \leq \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)
$$

Proof. By Theorem B.1, we know that

$$
\begin{aligned}
\log(2) l_{\text{route}} \left(r, a_h, f_h, u_h^{(t)}, u_h \right) &= \log(2) \sum_{j=1}^{K} \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\quad - \log(2) (K-2) \sum_{j=1}^{K} \left\| (I - C_j \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \\
&\leq \log(2) \sum_{j=1}^{K} \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} \\
&\overset{(a)}{\leq} - \sum_{j=1}^{K} \sum_{k=1}^{K} \left\| (I - C_k \circ \mathcal{L}_h^a) \left(u_h - u_h^{(t)} \right) \right\|_2^2 \mathbf{1}_{k \neq j} \log \left(\frac{\exp\left(g_j(a, f, u^{(t)}) \right)}{\sum_{k=1}^{K} \exp\left(g_k(a, f, u^{(t)}) \right)} \right) \\
&= \Psi(\mathbf{g}, a_h, f_h, u_h^{(t)}, u_h)
\end{aligned}
$$

(a) if $r(a_h, f_h, u_h^{(t)}) \neq j$, $\frac{\exp\left(g_j(a, f, u^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a, f, u^{(t)})\right)} < 0.5$ which implies that $-\log\left(\frac{\exp\left(g_j(a, f, u^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a, f, u^{(t)})\right)} \right) \geq \log(2) \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j}$

□

B.3 Proof of Theorem 5.1

Theorem 5.1. *Let $\tilde{c}_j(a_h, u_h^{(t)}, u_h) < \bar{E} < \infty$ for all $j \in [K]$. If there exists $j \in [K]$ such that $\tilde{c}_j(a_h, u_h^{(t)}, u_h) > E_{min} > 0$, then, for any collection of solvers $\{C_j\}_{j=1}^K$ and linear discrete operator $\mathcal{L}_h^a$, Ψ is Bayes consistent surrogate for l_{route}.*

Proof. For a given a_h, f_h, let u_h be $\mathcal{G}_h(a_h, f_h)$ where $\mathcal{G}_h$ denotes the solution operator acting on the grid G_h. Furthermore, let's consider routers of the form

$$
r(a, f, u^{(t)}) = \text{argmax}_{j \in [K]} g_j(a, f, u^{(t)})
$$

For a given $a_h, f_h, u_h^{(t)} \in \mathcal{A} \times \mathcal{F} \times \mathcal{U}$, let the optimal loss under l_{route} be $l^*_{\text{route}}\left(a_h, f_h, u_h^{(t)}\right) = \inf_{\tilde{r}} l_{\text{route}}\left(\tilde{r}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right)$. Similarly, let the optimal loss under Ψ be $\Psi^*\left(a_h, f_h, u_h^{(t)}\right) = \inf_{\tilde{\mathbf{g}}} \Psi\left(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right)$. Let $B_j(a_h, f_h, u_h^{(t)}) = \sum_{k=1}^K \left\|(I - C_k \circ \mathcal{L}_h^a)\left(\mathcal{G}_h\left(a_h, f_h\right) - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{k \neq j}$.

$$
\begin{aligned}
&l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right) - l^*_{\text{route}}\left(a_h, f_h, u_h^{(t)}\right) \\
&\overset{(a)}{=} \sum_{j=1}^K \sum_{k=1}^K \left\|(I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{k\neq j}\mathbf{1}_{r(a_h,f_h,u_h^{(t)})\neq j} - (K-2)\sum_{j=1}^K \left\|(I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \\
&\quad - \inf_{\tilde{r}} \sum_{j=1}^K \sum_{k=1}^K \left\|(I - C_k \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \mathbf{1}_{k\neq j}\mathbf{1}_{\tilde{r}(a_h,f_h,u_h^{(t)})\neq j} + (K-2)\sum_{j=1}^K \left\|(I - C_j \circ \mathcal{L}_h^a)\left(u_h - u_h^{(t)}\right)\right\|_2^2 \\
&= \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)})\mathbf{1}_{r(a_h,f_h,u_h^{(t)})\neq j} - \inf_{\tilde{r}} \sum_{j=1}^K B_j(a_h, f_h, u_h^{(t)})\mathbf{1}_{\tilde{r}(a_h,f_h,u_h^{(t)})\neq j} \\
&= \sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}\mathbf{1}_{r(a_h,f_h,u_h^{(t)})\neq j} - \inf_{\tilde{r}} \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}\mathbf{1}_{\tilde{r}(a_h,f_h,u_h^{(t)})\neq j}\right)
\end{aligned}
$$

(a) by Theorem B.1

Let $\mathcal{X} = \mathcal{A} \times \mathcal{F} \times \mathcal{U}$ and $\mathcal{Y} = [K]$. Let $\mathcal{P}_{\mathcal{X}}$ denote the degenerate distribution suported at the point $(a_h, f_h, u_h^{(t)})$. We define the conditional distribtion - $P(Y = j \mid X = (a_h, f_h, u_h^{(t)})) = \frac{B_j(a_h,f_h,u_h^{(t)})}{\sum_{k=1}^K B_k(a_h,f_h,u_h^{(t)})}$ for $j \in [K]$. The risk and optimal risk of $0-1$ loss under this distribution can be written as:

$$
\begin{aligned}
\mathcal{R}_{0-1}(r) &= \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}\mathbf{1}_{r(a_h,f_h,u_h^{(t)})\neq j} \\
\mathcal{R}^*_{0-1} &= \inf_r \sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})}\mathbf{1}_{r(a_h,f_h,u_h^{(t)})\neq j}
\end{aligned}
$$

If $r(a_h, f_h, u_h^{(t)}) = \arg\max_{j\in[k]} g_j(a_h, f_h, u_h^{(t)})$ for all $x \in \mathcal{X}$, then the he risk and optimal risk of cross entropy loss ($l_{ce}(\mathbf{g}, x, y) - \log\left(\frac{\exp(g_y(x))}{\sum_{k=1}^K \exp(g_k(x))}\right)$) under this distribution can be written as:

$$
\begin{aligned}
\mathcal{R}_{ce}(\mathbf{g}) &= -\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log\left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^K \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)}\right) \\
\mathcal{R}^*_{ce} &= \inf_{\mathbf{g}} -\sum_{j=1}^K \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^K B_k(a_h, f_h, u_h^{(t)})} \log\left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^K \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)}\right)
\end{aligned}
$$

From Theorem 3.1 of [Mao et al., 2023], $\mathcal{R}_{0-1}(r) - \mathcal{R}^*_{0-1} \leq \Gamma^{-1}\left(\mathcal{R}_{ce}(\mathbf{g}) - \mathcal{R}^*_{ce}\right)$ if $r(a_h, f_h, u_h^{(t)}) = \arg\max_{j\in[k]} g_j(a_h, f_h, u_h^{(t)})$ where $\Gamma(z) = \frac{1+z}{2}\log(1+z) + \frac{1-z}{2}\log(1-z)$. Then,

$$
\begin{aligned}
&l_{\text{route}}\left(r, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right) - l^*_{\text{route}}\left(a_h, f_h, u_h^{(t)}\right) \\
&= \sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)}) \left(\sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{r(a_h, f_h, u_h^{(t)}) \neq j} - \inf_{\tilde{r}} \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)})} \mathbf{1}_{\tilde{r}(a_h, f_h, u_h^{(t)}) \neq j} \right) \\
&\leq \sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)}) \Gamma^{-1} \left(- \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)} \right) \right. \\
&\qquad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)} \right) \right) \\
&\overset{(a)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)} \right) \right. \\
&\qquad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{\sum_{k=1}^{K} B_k(a_h, f_h, u_h^{(t)})} \log \left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)} \right) \right) \\
&\overset{(b)}{\leq} \bar{E} K (K-1) \Gamma^{-1} \left(- \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1) E_{min}} \log \left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)} \right) \right. \\
&\qquad \left. - \inf_{\mathbf{g}} - \sum_{j=1}^{K} \frac{B_j(a_h, f_h, u_h^{(t)})}{(K-1) E_{min}} \log \left(\frac{\exp\left(g_j(a_h, f_h, u_h^{(t)})\right)}{\sum_{k=1}^{K} \exp\left(g_k(a_h, f_h, u_h^{(t)})\right)} \right) \right) \\
&= \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi\left(\mathbf{g}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right) - \Psi^*\left(a_h, f_h, u_h^{(t)}\right)}{(K-1) E_{min}} \right)
\end{aligned}
$$

(a) since $\left\| (I - C_j \circ \mathcal{L}_h^a)\left(e_h^{(t)}\right) \right\|_2^2 < \bar{E}$ for all $j \in [K]$, (b) since Γ^{-1} is non-decreasing and $\exists j \in [K]$ such that $\left\| (I - C_j \circ \mathcal{L}_h^a)\left(e_h^{(t)}\right) \right\|_2^2 > E_{min}$

Finally,

$$
\begin{aligned}
&\lim_{n\to\infty} \mathcal{R}_{\text{route}}\left(r_n\right) - \mathcal{R}^*_{\text{route}} \\
&\overset{(a)}{=} \lim_{n\to\infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A}\times\mathcal{F}}} \left[l_{\text{route}}\left(r_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right) - l^*_{\text{route}}\left(a_h, f_h, u_h^{(t)}\right) \right] \\
&\leq \lim_{n\to\infty} \mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A}\times\mathcal{F}}} \left[\bar{E} K (K-1) \Gamma^{-1} \left(\frac{\Psi\left(\tilde{\mathbf{g}}, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right) - \Psi^*\left(a_h, f_h, u_h^{(t)}\right)}{(K-1) E_{min}} \right) \right] \\
&\overset{(b)}{\leq} \lim_{n\to\infty} \bar{E} K (K-1) \Gamma^{-1} \left(\frac{\mathbb{E}_{a_h, f_h \sim \mathcal{P}_{\mathcal{A}\times\mathcal{F}}} \left[\Psi\left(\mathbf{g}_n, a_h, f_h, u_h^{(t)}, \mathcal{G}_h\left(a_h, f_h\right)\right) - \Psi^*\left(a_h, f_h, u_h^{(t)}\right) \right]}{(K-1) E_{min}} \right)
\end{aligned}
$$

$$
\begin{aligned}
&= \lim_{n\to\infty} \bar{E}K\,(K-1)\,\Gamma^{-1}\left(\frac{\mathcal{R}_\Psi\left(\mathbf{g}_n\right)-\mathcal{R}_\Psi^*}{(K-1)E_{min}}\right)\\
&\overset{(c)}{=} \bar{E}K\,(K-1)\,\Gamma^{-1}\left(\frac{\lim_{n\to\infty}\mathcal{R}_\Psi\left(\mathbf{g}_n\right)-\mathcal{R}_\Psi^*}{(K-1)E_{min}}\right)\\
&= \bar{E}K\,(K-1)\,\Gamma^{-1}\left(0\right)\\
&\overset{(d)}{=} 0
\end{aligned}
$$

(a) $\mathcal{R}^*_{\text{route}} = \mathbb{E}_{a_h,f_h\sim\mathcal{P}_{\mathcal{A}\times\mathcal{F}}}\left[l^*_{\text{route}}\left(a_h, f_h, u_h^{(t)}\right)\right]$ since the infimum is taken over all measurable functions, (b) by Jensen's inequality since Γ^{-1} is concave, (c) by continuity of Γ^{-1} at 0, (d) $\Gamma^{-1}(0) = 0$

□

C Training Details

Data for both DeepONet and the routers is sampled from a zero-mean Hierarchical Gaussian Random Field on the periodic domain with covariance operator $\alpha(-\Delta+\beta I)^{-\gamma}$, where α, β, γ are sampled as:

$$
\begin{aligned}
\alpha &\sim \text{Log-Uniform}(0.01, 100)\\
\beta &\sim \text{Log-Uniform}(0.1, 1000)\\
\gamma &\sim \text{Uniform}\left(\{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0\}\right)
\end{aligned}
$$

Given (α, β, γ), samples are generated in the Fourier space. For each non-zero frequency mode k, we draw an independent complex coefficient from a Gaussian distribution with mean 0 and variance $\alpha(4\pi^2\|k\|_2^2+\beta)^{-\gamma}$. We enforce a Hermitian symmetry to obtain a real-valued field, set the zero-frequency (DC) mode to 0 to obtain a zero mean field, and apply the inverse Discrete Fourier Transform to obtain the field in physical space.

For each sample, we compute reference solutions with a least squares solver and treat them as ground truth.

This data is used to trained our DeepONet models and LSTM routers. All the models were implemented using PyTorch and all the models were trained on one Nvidia A40 GPU.

Table 4 contains all hyperparameter details for the DeepONet. DeepONet took 30 minutes to train. We then use the model with the best validation loss.

Table 4: Hyperparameter settings for DeepONet

Hyperparameter	Value
Learning rate	1e-3
Branch Dimension	128
Hidden dimension for branch net	256
No. of hidden layers in branch net	4
Hidden dimension for trunk net	256
No. of hidden layers in trunk net	4
Gradient Clipping Norm	1.0
Weight Decay	0.005
Batch size	256
Training samples	10000
Validation samples	2000
Epochs	1000

The routers are LSTM models. Along with the inputs specified in Section 5, namely $a_h, f_h, u_h^{(t)}$, we also supply the routers with the iteration index t, current residual $f_h - \mathcal{L}_h^a u_h^{(t)}$, and the routing

decisions from the previous iteration. While the iteration index and residual are deterministic functions of $(a_h, f_h, u_h^{(t)})$ and align with the theory presented in Section 5, incorporating previous routing decisions is a mild deviations from the idealized setting. Nevertheless, this modification is motivated by the same considerations that justify the use of scheduled sampling: under deployment, routing decisions are predicted autoregressivelt and influence future inputs, whereas training under teacher forcing assumes independence from past predictions. Including previous routing decisions bridges the gap induced by exposure bias by providing the model with trajectory-level information.

All the routers are trained with scheduled sampling. We use a warm-up of e_w epochs with teacher-forcing probability $p_{tf}(e) = ss_{\text{start}}$. After the warm-up, the p_{tf} decays geomterically by a factor of $\gamma_{tf} < 1$ per epoch and is floored by s_{end}:

$$p_{tf}(e) = \begin{cases} ss_{\text{start}} & e \le e_w \\ \max(ss_{start}\gamma_{tf}^{e-e_w}, ss_{end}) & e > e_w \end{cases}$$

At each time step, with probability $p_{tf}(e)$, we feed the teacher-forced greedy iterate; otherwise, we feed the router's own predicted iterate.

Since LSTMs on long rollouts can suffer from exploding/vanishing gradients, we use truncated backpropagation through time (TBPTT) [Mozer, 2013, Robinson and Fallside, 1987, Werbos, 1988]: the forward pass unrolls the entire trajectory, but gradients are propagated only through the most recent $w_{\text{bptt}}(e)$ steps at epoch e. Hidden states are passed forward between segments, while earlier segments are treated as stop-gradient.

We employ a curriculum learning approach analogous to scheduled sampling. Let $T_{\max}$ be the horizon (300 iterations). With a warm-up of e_w epochs,

$$w_{\text{bptt}}(e) = \begin{cases} w_{\text{start}} & e \le e_w \\ \min\left(T_{\max}, w_{\text{start}}\gamma_{\text{bptt}}^{\left\lfloor \frac{e-e_w}{f_{\text{bptt}}} \right\rfloor}\right) & e > e_w \end{cases} \tag{12}$$

so the window grows geometrically by a factor of $\gamma_{\text{bptt}} > 1$ every f_{bptt} epochs and is capped at the full trajectory length.

Table 5 contains all hyperparameter details for the LSTM routers. The routers for the Paired Solver experiment took a maximum of 4 hours and 30 minutes to train while the routers for the Solver Ensemble experiment took a maximum of 6 hours to train. We then use the model with the best validation loss for testing. The architecture and dataset size is larger for the Solver ensemble experiments to encourage the model to learn some of the nuanced differences between the classes.

Table 5: Hyperparameter settings for routers in Paired solver and Solver Ensemble Experiments

Hyperparameter	Paired Solver	Solver Ensemble
Learning rate	1e-3	1e-3
Hidden dimension	256	512
No. of hidden layers	4	3
Gradient Clipping Norm	1.0	1.0
Weight Decay	0.005	0.005
Batch size	32	32
Training samples	256	1024
Validation samples	32	128
Epochs	200	200
ss_{start}	1.0	1.0
γ_{tf}	0.95 0.95	
ss_{end}	0.0	0.0
w_{start}	50	50
γ_{bptt}	1.25	1.25
e_w	10	10
f_{bptt}	1	1

D Additional Experimental Results

We present additional visualizations and tables that further highlight the strengths of our method. In the paired solver experiments, we include Damped Jacobi with $\omega = 0.67$ and Successive Over-Relaxation (SOR) with $\omega = 1.5$. We also compare against a True Greedy oracle, which has access to true error at each step (See Section 4). This comparison demonstrates that the learned router closely approximates the behavior of the greedy policy.

D.1 Error Comparisons with Paired t-Test Results

We supplement the paired-solver experiments in Table 1 with additional solver families (Jacobi (0.67) and SOR (1.5)) and statistical significance analysis. For each solver family, we compare the learned greedy router against baseline methods (single-solver only and HINTS) using paired t-tests with a one-sided alternative hypothesis that the learned router achieves lower error or AUC. Results are reported in Table 6.

Across all five solver families, the resulting p-values indicate statistically significant improvements of the learned greedy router over both single-solver and HINTS baselines.

We support the Large Solver ensemble experiments in Table 3 with a statistical significance analysis. We use paired t-tests across the 128 test instances. For each baseline solver j (e.g., Jacobi, GS, SymGS, etc.), we compare the performance of the solver ensemble $\mathcal{W}$ against the corresponding pairwise learned router $\mathrm{Router}(\mathrm{NO} \cup \{j\})$ if $j \in \mathcal{W}$. The test is performed on per-instance differences in final error (and AUC), using a one-sided alternative hypothesis that the solver ensemble achieves lower error than the baseline.

Across both Poisson and convection–diffusion problems, the solver ensembles yield statistically significant improvements over all pairwise baselines in nearly every setting, with p-values typically well below 0.01. The gains are enhanced as the ensemble size increases, while the performance of the learned router remains close to that of the true greedy oracle. These results support the claim that enlarging the solver set provides measurable benefits and that our learned policy effectively captures the resulting improvements. Results are reported in Table 7.

Table 6: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t-tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p-values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\|e_h^{(T)}\| \times 10^3$	$\|e_h^{(T)}\|$ p-value	AUC	AUC p-value	$\|e_h^{(T)}\| \times 10^3$	$\|e_h^{(T)}\|$ p-value	AUC	AUC p-value
Jacobi-related solvers								
Jacobi Only	0.383 (1.029)	$< 10^{-3}$	0.821 (2.154)	$< 10^{-3}$	0.136 (0.364)	$< 10^{-3}$	0.312 (0.788)	$< 10^{-3}$
HINTS-Jacobi	0.759 (0.016)	$< 10^{-3}$	0.393 (0.590)	$< 10^{-3}$	0.447 (0.013)	$< 10^{-3}$	0.202 (0.256)	$< 10^{-3}$
Learned Greedy-Jacobi	0.054 (0.142)	-	0.165 (0.368)	-	0.033 (0.097)	-	0.098 (0.239)	-
True-Greedy-Jacobi	0.021 (0.018)	-	0.094 (0.170)	-	0.012 (0.012)	-	0.049 (0.091)	-
GS-related solvers								
GS only	0.019 (0.051)	$< 10^{-3}$	0.435 (1.141)	$< 10^{-3}$	$< 10^{-3}$	0.006	0.088 (0.219)	$< 10^{-3}$
HINTS-GS	0.724 (0.000)	$< 10^{-3}$	0.325 (0.488)	$< 10^{-3}$	0.456 (0.000)	$< 10^{-3}$	0.130 (0.154)	0.0
Learned Greedy-GS	0.002 (0.004)	-	0.083 (0.152)	-	$< 10^{-3}$	-	0.031 (0.078)	-
True-Greedy-GS	0.001 (0.001)	-	0.057 (0.107)	-	$< 10^{-3}$	-	0.019 (0.038)	-
SymGS-related solvers								
SymGS only	$< 10^{-3}$	$< 10^{-3}$	0.227 (0.593)	$< 10^{-3}$	$< 10^{-3}$	0.028	0.071 (0.178)	$< 10^{-3}$
HINTS-SymGS	0.709 (0.000)	$< 10^{-3}$	0.252 (0.380)	$< 10^{-3}$	0.463 (0.000)	$< 10^{-3}$	0.116 (0.131)	$< 10^{-3}$
Learned Greedy-SymGS	$< 10^{-3}$	-	0.052 (0.102)	-	$< 10^{-3}$	-	0.022 (0.039)	-
True-Greedy-SymGS	$< 10^{-3}$	-	0.034 (0.066)	-	$< 10^{-3}$	-	0.016 (0.032)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	1.066 (2.861)	$< 10^{-3}$	1.128 (2.954)	$< 10^{-3}$	0.349 (0.931)	$< 10^{-3}$	0.428 (1.077)	$< 10^{-3}$
HINTS-Jacobi (0.67)	0.789 (0.000)	$< 10^{-3}$	0.429 (0.642)	0.005	0.473 (0.000)	$< 10^{-3}$	0.223 (0.284)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.232 (0.654)	-	0.309 (0.791)	-	0.069 (0.102)	-	0.117 (0.210)	-
True-Greedy-Jacobi (0.67)	0.057 (0.041)	-	0.131 (0.233)	-	0.026 (0.021)	-	0.065 (0.117)	-
SOR (1.5)-related solvers								
SOR (1.5) only	$< 10^{-3}$	0.057	0.156 (0.409)	$< 10^{-3}$	$< 10^{-3}$	0.086	0.008 (0.020)	0.988
HINTS-SOR (1.5)	0.700 (0.000)	$< 10^{-3}$	0.207 (0.316)	$< 10^{-3}$	0.462 (0.000)	$< 10^{-3}$	0.021 (0.020)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	$< 10^{-3}$	-	0.052 (0.143)	-	$< 10^{-3}$	-	0.011 (0.030)	-
True-Greedy-SOR (1.5)	$< 10^{-3}$	-	0.032 (0.071)	-	$< 10^{-3}$	-	0.007 (0.019)	-

Table 7: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t-tests comparing each solver ensemble against the corresponding pairwise learned router (e.g., NO + Jacobi, NO + GS), using a one-sided alternative that the ensemble achieves lower error/AUC. Reported p-values correspond to these tests.

$\mathcal{W}$	$\|e_h^{(T)}\| \times 10^3$	AUC	Jacobi p-value	GS p-value	SymGS p-value	Jacobi (0.67) p-value	SOR (1.5) p-value
			Poisson				
Learned Greedy {Jacobi, GS}	0.003 (0.002)	0.068 (0.131)	$< 10^{-3}$	$< 10^{-3}$	-	-	-
True Greedy {Jacobi, GS}	0.001 (0.001)	0.057 (0.107)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.038 (0.071)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	-	-
True Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.034 (0.066)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.042 (0.074)	$< 10^{-3}$	$< 10^{-3}$	0.004	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.034 (0.066)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.039 (0.076)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.04
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.031 (0.069)	-	-	-	-	-
			ConvDiff				
Learned Greedy {Jacobi, GS}	$< 10^{-3}$	0.021 (0.040)	$< 10^{-3}$	0.005	-	-	-
True Greedy{Jacobi, GS}	$< 10^{-3}$	0.019 (0.038)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.020 (0.036)	$< 10^{-3}$	0.004	$< 10^{-3}$	-	-
True Greedy {Jacobi, GS, SymGS}	$< 10^{-3}$	0.016 (0.032)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.020 (0.039)	$< 10^{-3}$	0.003	0.003	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	$< 10^{-3}$	0.016 (0.032)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.008 (0.019)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.004
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.007 (0.018)	-	-	-	-	-

D.2 Convergence Histories

See Figure 4 for convergence histories for the Jacobi (0.67)-related solvers and Successive Over-relaxation (1.5)-related solvers.

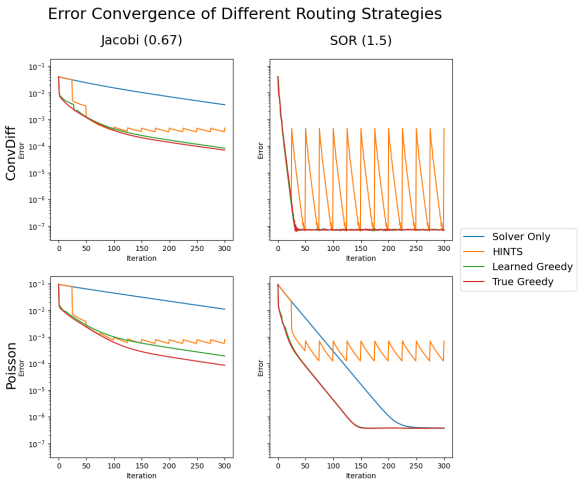

Figure 4: Convergence histories for representative test instances. Rows: ConvDiff (top) and Poisson (bottom). Columns: Jacobi (0.67) and Successive Over-relaxation (1.5). Greedy yields near-monotone decay and the lowest errors, whereas HINTS shows sawtooth behaviors.

D.3 Prediction and Error Visualizations

Figures 5 and 6 provide qualitative visualizations of sample predictions across routing strategies and numerical solvers for both equations. While these prediction may appear identical to the ground truth, Figures 7 and 8 reveals distinct error patterns with varying magnitudes across all predictions. In particular, our learned method consistently achieves the lower-magnitude errors compared to its respective baselines.

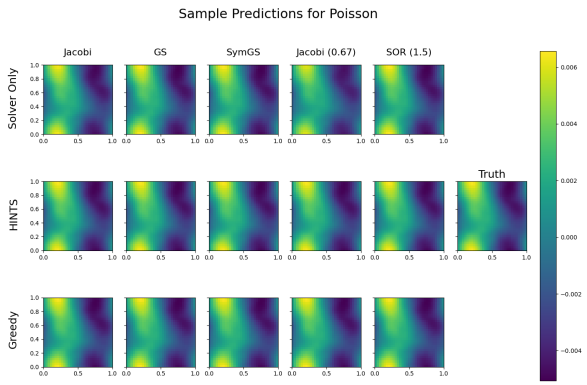

Figure 5: Sample predictions for the Poisson equation across different solver pairings. Each column corresponds to a numerical solver (Jacobi, Gauss–Seidel (GS), Symmetric GS, Damped Jacobi, and Successive Over-relaxation), while rows show predictions from the corresponding solver-only baseline (top), HINTS (middle), and the learned greedy router (bottom). The ground truth solution is shown on the right.

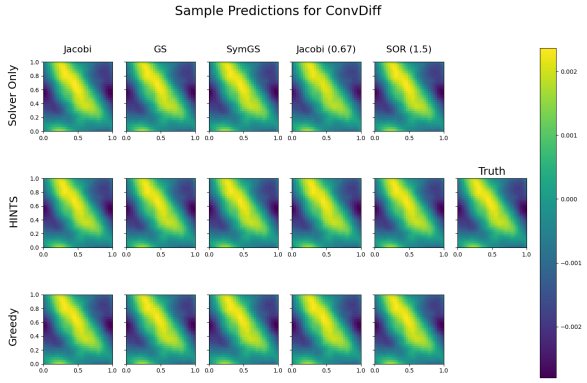

Figure 6: Sample predictions for the convection diffusion equation across different solver pairings. Each column corresponds to a numerical solver (Jacobi, Gauss–Seidel (GS), Symmetric GS, Damped Jacobi, and Successive Over-relaxation), while rows show predictions from the corresponding solver-only baseline (top), HINTS (middle), and the learned greedy router (bottom). The ground truth solution is shown on the right.

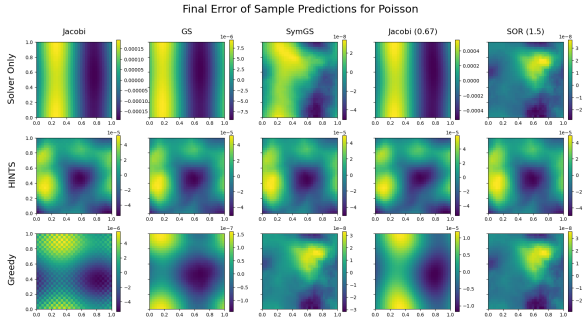

Figure 7: Pointwise error maps for Poisson sample predictions across solver families. Columns correspond to different numerical solvers, while rows show solver-only (top), HINTS (middle), and learned greedy router (bottom). Errors are computed as $u_h - u_h^{(T)} = e_h^{(T)}$. The learned greedy router consistently yields lower-magnitude errors compared to both solver-only and HINTS baselines.

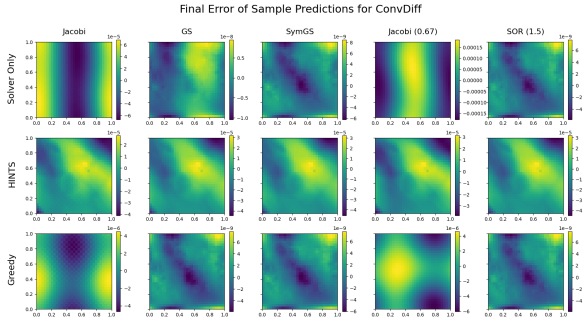

Figure 8: Pointwise error maps for convection diffusion sample predictions across solver families. Columns correspond to different numerical solvers, while rows show solver-only (top), HINTS (middle), and learned greedy router (bottom). Errors are computed as $u_h - u_h^{(T)} = e_h^{(T)}$. The learned greedy router consistently yields lower-magnitude errors compared to both solver-only and HINTS baselines.

D.4 Residual Comparison

Table 8: Final residual $\|\mathcal{L}_h^a e_h^{(T)}\|$ and AUC of residuals over iterations (lower is better). Values are reported as mean ($\pm$ standard error (s.e.)) over 128 test instances; residuals are scaled by $\times 10^{-3}$. If the residual norm is reported as $< 10^{-3}$, the corresponding raw value is $< 10^{-6}$. Statistical significance is evaluated via paired t-tests comparing each baseline (single-solver only and HINTS) to the learned greedy router under a one-sided alternative hypothesis that the learned router achieves lower residual and AUC. Reported p-values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\|\mathcal{L}_h^a e_h^{(T)}\| \times 10^3$	$\|\mathcal{L}_h^a e_h^{(T)}\|$ p-value	AUC	AUC p-value	$\|\mathcal{L}_h^a e_h^{(T)}\| \times 10^3$	$\|\mathcal{L}_h^a e_h^{(T)}\|$ p-value	AUC	AUC p-value
Jacobi-related solvers								
Jacobi only	23.939 (66.755)	0.021	43.475 (109.764)	$< 10^{-3}$	27.208 (74.555)	0.012	45.523 (114.774)	$< 10^{-3}$
HINTS-Jacobi	193.327 (37.362)	$< 10^{-3}$	39.763 (70.671)	$< 10^{-3}$	266.421 (34.699)	$< 10^{-3}$	42.851 (70.748)	$< 10^{-3}$
Learned Greedy-Jacobi	20.870 (62.246)	-	32.041 (83.647)	-	20.786 (66.511)	-	34.844 (84.666)	-
GS-related solvers								
GS only	0.761 (2.043)	$< 10^{-3}$	20.776 (52.270)	$< 10^{-3}$	0.002 (0.005)	$< 10^{-3}$	10.720 (26.892)	$< 10^{-3}$
HINTS-GS	183.359 (0.000)	$< 10^{-3}$	22.581 (27.984)	$< 10^{-3}$	246.877 (0.000)	$< 10^{-3}$	18.654 (19.201)	$< 10^{-3}$
Learned Greedy-GS	0.092 (0.145)	-	11.960 (27.863)	-	0.001 (0.003)	-	7.322 (16.968)	-
SymGS-related solvers								
SymGS only	0.004 (0.009)	$< 10^{-3}$	10.343 (25.957)	$< 10^{-3}$	0.002 (0.004)	0.004	8.604 (21.776)	$< 10^{-3}$
HINTS-SymGS	185.601 (0.001)	$< 10^{-3}$	15.489 (18.255)	$< 10^{-3}$	245.629 (0.000)	$< 10^{-3}$	16.515 (15.990)	$< 10^{-3}$
Learned Greedy-SymGS	0.002 (0.005)	-	7.149 (18.364)	-	0.001 (0.002)	-	6.481 (13.190)	-
Jacobi (0.67) -related solvers								
Jacobi (0.67) only	42.246 (112.968)	$< 10^{-3}$	55.250 (138.258)	$< 10^{-3}$	45.451 (121.431)	$< 10^{-3}$	56.747 (141.901)	$< 10^{-3}$
HINTS-Jacobi (0.67)	188.719 (0.008)	$< 10^{-3}$	38.970 (53.035)	$< 10^{-3}$	256.407 (0.005)	$< 10^{-3}$	39.604 (52.578)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	9.290 (26.109)	-	35.905 (86.881)	-	9.010 (13.426)	-	32.516 (83.488)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.003 (0.008)	0.5	9.044 (22.962)	$< 10^{-3}$	0.002 (0.004)	0.189	2.817 (7.499)	$< 10^{-3}$
HINTS-SOR (1.5)	188.187 (0.001)	$< 10^{-3}$	15.894 (19.565)	$< 10^{-3}$	248.226 (0.001)	$< 10^{-3}$	11.420 (7.498)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.003 (0.008)	-	7.784 (19.939)	-	0.002 (0.004)	-	5.530 (19.679)	-

Table 9: Final residual and AUC of squared L^2 residual for varying numbers of solvers. Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. are reported in $\times 10^{-3}$.

Equation	Poisson		ConvDiff	
$\mathcal{W}$	$\|\mathcal{L}_h^a e_h^{(T)}\|$	AUC	$\|\mathcal{L}_h^a e_h^{(T)}\|$	AUC
{Jacobi, GS}	0.135 (0.096)	11.815 (33.768)	0.023 (0.044)	5.765 (14.581)
{Jacobi, GS, SymGS}	0.002 (0.004)	5.118 (12.920)	0.002 (0.002)	4.810 (11.264)
{Jacobi, GS, SymGS, Jacobi (0.67)}	0.013 (0.011)	6.120 (14.651)	0.010 (0.013)	4.859 (11.473)
{Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.004 (0.006)	8.196 (22.516)	0.001 (0.002)	3.209 (8.676)

Table 8 summarizes the performance of single-solver schedules, HINTS, and greedy with respect to the final residuals $r_h^{(T)} = \|f_h - \mathcal{L}_h^a u_h^{(T)}\|$ or $\|\mathcal{L}_h^a e_h^{(T)}\|$ and its AUC $AUC_T = \sum_{t=1}^{T} \|r_h^{(t)}\|_2^2$. Greedy outperforms its HINTS and single-solver counterparts in most equations. We must note that our greedy router is trained to reduce error, not residual. The same error can induce very different residuals depending on the spectrum $\mathcal{L}_h^a$. Table 9 exhibits how residuals are affected by the number of solvers in the solver ensemble. Similar to error, we observe both the final residual and AUC decrease as the number of solvers increase.

D.5 Fourier mode-wise error comparison

We assess frequency-resolved performance by projecting the error onto the discrete Fourier basis. Table 10 report, for modes 1, 5, and 10, the mode-wise final error and mode-wise AUC, comparing single-solver baselines, HINTS, and the greedy router. As a result of including a deep learning model,Greedy consistently achieves the smallest mode-1 error/AUC across equations and solver families. For modes 5 and 10, single-solver schedules sometimes have an edge, reflecting the tendency of classical smoothers to damp high-frequency components more aggressively than ML surrogates (spectral bias). Overall, greedy delivers more uniform convergence across the spectrum: it routes to whichever solver most decreases the full L^2 error, and by Parseval's identity $|e_h^{(t)}|_2^2 = \sum_m |\hat{u}_m^{(t)} - \hat{u}_m|^2$, reductions in the objective correspond to reducing energy across all modes rather than giving preferential treatment to a subset. Additionally, in Table 11, we observe that all mode-wise errors/AUCs reduce with the inclusion of more solvers.

Table 10: Final error and AUC of squared L^2 error for Mode 1, 5, and 10 (lower is better) for Poisson and ConvDiff. Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. of final errors are reported in $\times 10^{-3}$. If the residual norm is reported as $< 10^{-3}$, the corresponding raw value is $< 10^{-6}$.

	Poisson						ConvDiff					
	Mode 1		Mode 5		Mode 10		Mode 1		Mode 5		Mode 10	
Method	Final Error	AUC	Final Error	AUC	Final Error	AUC	Final Error	AUC	Final Error	AUC	Final Error	AUC
Jacobi-related solvers												
Jacobi Only	0.135 (0.295)	3.231 (7.056)	$< 10^{-3}$	0.003 (0.010)	$< 10^{-3}$	0.000 (0.001)	0.079 (0.171)	1.084 (2.369)	$< 10^{-3}$	0.003 (0.011)	$< 10^{-3}$	0.000 (0.001)
HINTS-Jacobi	5.730 (0.240)	2.734 (3.371)	0.028 (0.002)	0.004 (0.012)	0.006 (0.000)	0.000 (0.001)	0.994 (0.220)	0.697 (1.091)	0.061 (0.001)	0.005 (0.012)	0.022 (0.001)	0.001 (0.001)
Learned Greedy-Jacobi	0.032 (0.085)	0.841 (2.081)	$< 10^{-3}$	0.009 (0.028)	$< 10^{-3}$	0.001 (0.003)	0.021 (0.055)	0.324 (0.780)	$< 10^{-3}$	0.009 (0.025)	$< 10^{-3}$	0.001 (0.002)
GS-related solvers												
GS Only	0.002 (0.004)	1.695 (3.702)	$< 10^{-3}$	0.003 (0.011)	$< 10^{-3}$	0.000 (0.001)	$< 10^{-3}$	0.205 (0.452)	$< 10^{-3}$	0.002 (0.006)	$< 10^{-3}$	$< 10^{-3}$
HINTS-GS	5.377 (0.000)	2.113 (2.498)	0.027 (0.000)	0.005 (0.012)	0.007 (0.000)	0.000 (0.001)	0.661 (0.000)	0.271 (0.432)	0.065 (0.000)	0.003 (0.006)	0.019 (0.000)	$< 10^{-3}$
Learned Greedy-GS	0.000 (0.001)	0.465 (1.125)	$< 10^{-3}$	0.009 (0.029)	$< 10^{-3}$	0.001 (0.002)	$< 10^{-3}$	0.075 (0.151)	$< 10^{-3}$	0.005 (0.017)	$< 10^{-3}$	0.001 (0.001)
SymGS-related solvers												
SymGS Only	$< 10^{-3}$	0.883 (1.919)	$< 10^{-3}$	0.002 (0.006)	$< 10^{-3}$	0.000 (0.001)	$< 10^{-3}$	0.180 (0.400)	$< 10^{-3}$	0.001 (0.005)	$< 10^{-3}$	0.000 (0.001)
HINTS-SymGS	5.085 (0.000)	1.432 (1.700)	0.025 (0.000)	0.002 (0.007)	0.007 (0.000)	0.000 (0.001)	0.672 (0.000)	0.240 (0.381)	0.070 (0.000)	0.002 (0.005)	0.019 (0.000)	0.000 (0.001)
Learned Greedy-SymGS	$< 10^{-3}$	0.334 (0.943)	$< 10^{-3}$	0.007 (0.025)	$< 10^{-3}$	0.001 (0.002)	$< 10^{-3}$	0.079 (0.193)	$< 10^{-3}$	0.004 (0.012)	$< 10^{-3}$	0.001 (0.001)
Jacobi (0.67)-related solvers												
Jacobi (0.67) Only	1.063 (2.321)	4.771 (10.421)	$< 10^{-3}$	0.005 (0.017)	$< 10^{-3}$	$< 10^{-3}$	0.428 (0.934)	1.535 (3.352)	$< 10^{-3}$	0.004 (0.016)	$< 10^{-3}$	$< 10^{-3}$
HINTS-Jacobi (0.67)	5.804 (0.001)	2.983 (3.666)	0.029 (0.000)	0.007 (0.019)	0.006 (0.000)	$< 10^{-3}$	1.080 (0.000)	0.776 (1.158)	0.065 (0.000)	0.008 (0.017)	0.021 (0.000)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.390 (1.195)	1.804 (5.363)	$< 10^{-3}$	0.017 (0.058)	$< 10^{-3}$	0.001 (0.002)	0.096 (0.208)	0.435 (0.910)	$< 10^{-3}$	0.013 (0.044)	$< 10^{-3}$	0.001 (0.004)
SOR (1.5) -related solvers												
SOR (1.5) Only	$< 10^{-3}$	0.727 (1.594)	$< 10^{-3}$	0.005 (0.019)	$< 10^{-3}$	0.001 (0.002)	$< 10^{-3}$	0.037 (0.082)	$< 10^{-3}$	0.002 (0.007)	$< 10^{-3}$	0.000 (0.001)
HINTS-SOR (1.5)	4.826 (0.000)	1.239 (1.484)	0.027 (0.000)	0.007 (0.020)	0.008 (0.000)	0.001 (0.002)	0.645 (0.000)	0.080 (0.082)	0.068 (0.000)	0.005 (0.007)	0.019 (0.000)	0.001 (0.001)
Learned Greedy-SOR (1.5)	$< 10^{-3}$	0.302 (0.757)	$< 10^{-3}$	0.010 (0.032)	$< 10^{-3}$	0.001 (0.003)	$< 10^{-3}$	0.049 (0.119)	$< 10^{-3}$	0.007 (0.030)	$< 10^{-3}$	0.001 (0.002)

Table 11: Final error and AUC of squared L^2 error of Mode 1, 5, and 10 for varying numbers of solvers. Values are mean ($\pm$ standard error (s.e.)) over 128 test instances; both mean and s.e. are reported in $\times 10^{-3}$.

Equation	Poisson						ConvDiff					
$\mathcal{W}$ Mode	Mode 1 Error	Mode 1 AUC	Mode 5 Error	Mode 5 AUC	Mode 10 Error	Mode 10 AUC	Mode 1 Error	Mode 1 AUC	Mode 5 Error	Mode 5 AUC	Mode 10 Error	Mode 10 AUC
{Jacobi, GS}	0.000 (0.001)	0.267 (0.571)	$< 10^{-3}$	0.006 (0.023)	$< 10^{-3}$	0.001 (0.003)	$< 10^{-3}$	0.055 (0.112)	$< 10^{-3}$	0.002 (0.008)	$< 10^{-3}$	0.000 (0.002)
{Jacobi, GS, SymGS}	0.000 (0.001)	0.185 (0.413)	$< 10^{-3}$	0.003 (0.009)	$< 10^{-3}$	0.000 (0.002)	$< 10^{-3}$	0.054 (0.114)	$< 10^{-3}$	0.002 (0.007)	$< 10^{-3}$	0.000 (0.001)
{Jacobi, GS, SymGS, Jacobi (0.67)}	0.000 (0.001)	0.172 (0.333)	$< 10^{-3}$	0.003 (0.010)	$< 10^{-3}$	0.000 (0.002)	$< 10^{-3}$	0.056 (0.119)	$< 10^{-3}$	0.002 (0.007)	$< 10^{-3}$	0.000 (0.001)
{Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.000 (0.001)	0.210 (0.399)	$< 10^{-3}$	0.006 (0.023)	$< 10^{-3}$	0.001 (0.004)	$< 10^{-3}$	0.026 (0.061)	$< 10^{-3}$	0.002 (0.005)	$< 10^{-3}$	0.000 (0.002)

D.6 Ensemble Solver Decisions and DeepONet Usage

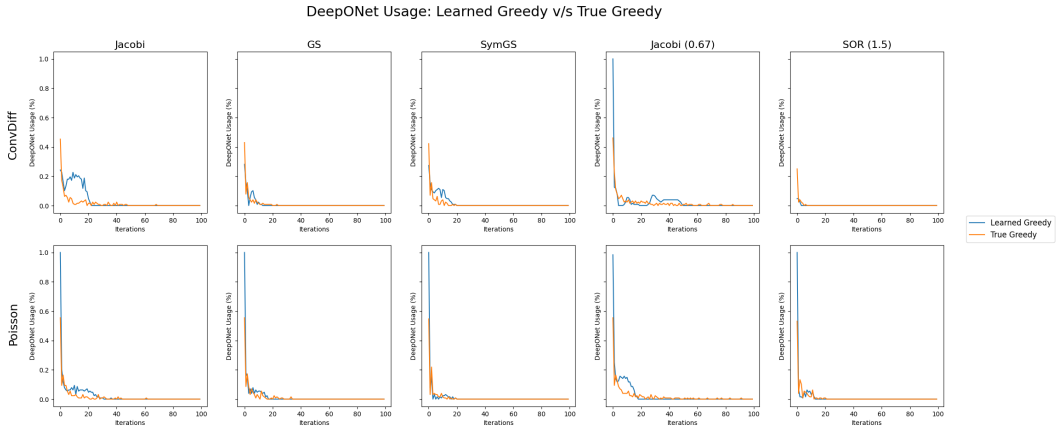

Figure 9: DeepONet usage over iterations for learned and true greedy routing across solver families. Columns correspond to different numerical solvers, and rows correspond to the two equations. The learned router closely follows the true greedy policy, capturing its nonuniform usage of DeepONet across iterations. For readability, we display the first 100 iterations (out of 300 total)

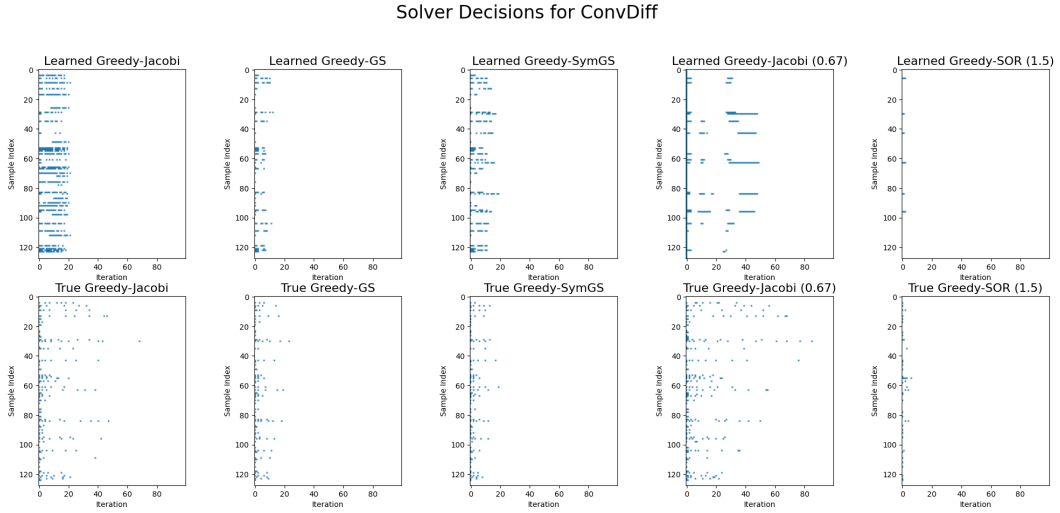

Figure 10: Solver selection patterns for the convection–diffusion equation across different solver pairings and test samples. Each column corresponds to a solver family, while rows show decisions from the learned greedy router (top) and the true greedy policy (bottom). Colored entries indicate iterations where the neural operator is selected, with blank regions corresponding to numerical solver updates. The learned router exhibits instance-dependent routing behavior.

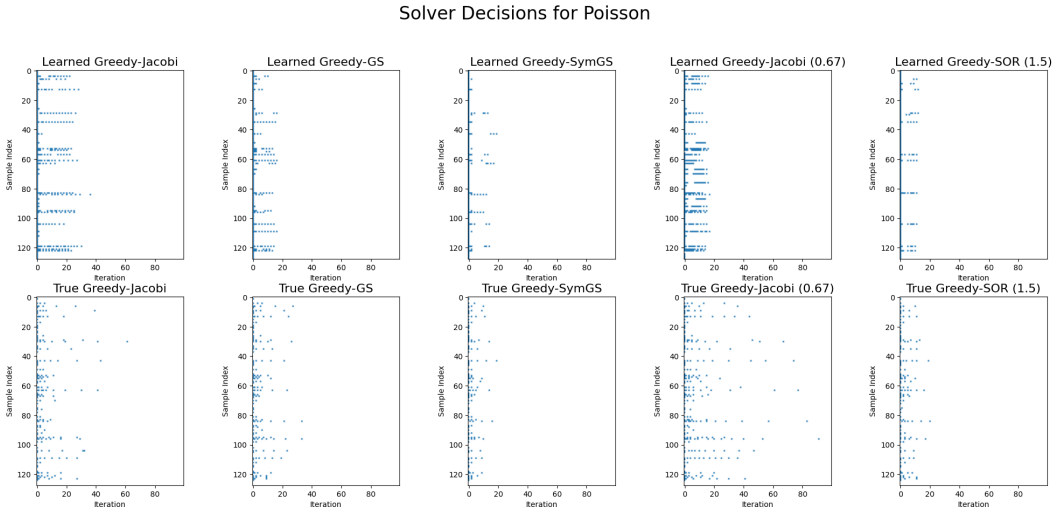

Figure 11: Solver selection patterns for the Poisson equation across different solver pairings and test samples. Each column corresponds to a solver family, while rows show decisions from the learned greedy router (top) and the true greedy policy (bottom). Colored entries indicate iterations where the neural operator is selected, with blank regions corresponding to numerical solver updates. The learned router exhibits instance-dependent routing behavior.

Figure 9 highlights the proportion of DeepONet calls across iterations, while Figures 10 and 11 visualize the routing decisions of the learned and true greedy policies across all test samples and solver families.

Two key observations emerge. First, the routing strategy is highly sample-dependent, indicating that a fixed routing schedule across all instances would be suboptimal and can slow convergence. Second, the learned routing policy closely tracks the behavior of the true greedy policy, providing empirical support for the approximation guarantees discussed in Section 5. In particular, the learned policy can be viewed as a smoothed approximation of the true greedy strategy, capturing its overall structure while exhibiting minor deviations due to learning and finite-sample effects.

Table 12: Average component selection frequencies for the Poisson equation. Rows indicate the router access set. Entries are mean ($\pm$ s.e.) over 128 test instances. Here NO denotes the neural operator; "-" indicates that the component is not available to the router.

$\mathcal{W}$	Jacobi	GS	SSOR	Jacobi (0.67)	SOR (1.5)	DeepONet
{Jacobi, GS}	0.405 (0.143)	0.588 (0.134)	-	-	-	0.007 (0.011)
{Jacobi, GS, SSOR}	0.000 (0.000)	0.000 (0.000)	0.996 (0.005)	-	-	0.004 (0.005)
{Jacobi, GS, SSOR, Jacobi (0.67)}	0.227 (0.112)	0.212 (0.118)	0.434 (0.212)	0.121 (0.102)	-	0.005 (0.007)
{Jacobi, GS, SSOR, Jacobi (0.67), SOR (1.5)}	0.009 (0.033)	0.570 (0.057)	0.260 (0.040)	0.000 (0.000)	0.156 (0.013)	0.005 (0.005)

Table 13: Average component selection frequencies for the Convection-Diffusion equation. Rows indicate the router access set. Entries are mean ($\pm$ s.e.) over 128 test instances. Here NO denotes the neural operator; "-" indicates that the component is not available to the router.

$\mathcal{W}$	Jacobi	GS	SSOR	Jacobi (0.67)	SOR (1.5)	DeepONet
{Jacobi, GS}	0.417 (0.307)	0.579 (0.307)	-	-	-	0.004 (0.006)
{Jacobi, GS, SSOR}	0.109 (0.109)	0.396 (0.110)	0.493 (0.175)	-	-	0.003 (0.005)
{Jacobi, GS, SSOR, Jacobi (0.67)}	0.168 (0.096)	0.190 (0.166)	0.410 (0.227)	0.229 (0.146)	-	0.003 (0.004)
{Jacobi, GS, SSOR, Jacobi (0.67), SOR (1.5)}	0.307 (0.150)	0.061 (0.131)	0.088 (0.170)	0.202 (0.089)	0.342 (0.066)	0.001 (0.001)

Tables 12 and 13 report solver selection frequencies across different solver ensembles. Several interesting observations emerge from these results.

First, the router typically relies on a small subset of solvers, with larger ensembles often leading to the dominance of a single solver (e.g., SOR(1.5) in convection–diffusion). This explains the diminishing returns observed in the multi-solver setting, as the solver ensemble grows.

Additionally, the neural operator is selected only rarely across all configurations. This reflects the strength of the numerical solvers rather than a limitation of our method. Our framework is designed to exploit this structure by relying primarily on numerical updates and invoking the learned component only when beneficial, demonstrating that selective use of learned corrections can yield performance gains without frequent deployment like in HINTS or similar fixed schedules.

Finally, the non-zero standard deviations indicate that solver usage varies across test instances, suggesting that no single global routing strategy emerges.

D.7 Results on a finer grid

We have replicated the Paired Solver and Solver Ensemble experiments for a grid of 63×63.

In Table 14, we notice that, similar to the case with 31×31 grid, the learned greedy router closely matches the performance of the true greedy oracle and outperforms its single solver and HINTS baselines with great statistical significance.

Similar to Table 3, we notice, in Table 15, the learned greedy router seems to outperform the various pairwise configurations mostly with great statistical significance.

Table 14: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances for the grid 63×63; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t-tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p-values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\|e_h^{(T)}\| \times 10^3$	$\|e_h^{(T)}\|$ p-value	AUC	AUC p-value	$\|e_h^{(T)}\| \times 10^3$	$\|e_h^{(T)}\|$ p-value	AUC	AUC p-value
Jacobi-related solvers								
Jacobi Only	4.483 (12.776)	$< 10^{-3}$	2.092 (5.889)	$< 10^{-3}$	1.470 (4.164)	0.001	0.757 (2.116)	0.001
HINTS-Jacobi	0.823 (0.163)	0.043	0.571 (1.205)	0.021	1.512 (0.433)	$< 10^{-3}$	0.565 (0.723)	$< 10^{-3}$
Learned Greedy-Jacobi	0.506 (2.201)	-	0.429 (1.293)	-	0.654 (1.603)	-	0.393 (0.969)	-
True-Greedy-Jacobi	0.199 (0.277)	-	0.289 (0.661)	-	0.317 (0.593)	-	0.277 (0.635)	-
GS-related solvers								
GS only	2.094 (6.009)	$< 10^{-3}$	1.519 (4.296)	$< 10^{-3}$	0.357 (1.025)	0.004	0.430 (1.211)	0.003
HINTS-GS	0.631 (0.003)	$< 10^{-3}$	0.444 (0.939)	$< 10^{-3}$	0.996 (0.004)	$< 10^{-3}$	0.369 (0.397)	0.03
Learned Greedy-GS	0.138 (0.581)	-	0.228 (0.623)	-	0.219 (0.697)	-	0.282 (0.820)	-
True-Greedy-GS	0.072 (0.064)	-	0.180 (0.403)	-	0.053 (0.065)	-	0.132 (0.279)	-
SymGS-related solvers								
SymGS only	1.318 (3.541)	$< 10^{-3}$	1.106 (2.978)	$< 10^{-3}$	0.267 (0.725)	0.012	0.362 (0.974)	0.014
HINTS-SymGS	0.566 (0.002)	$< 10^{-3}$	0.375 (0.782)	$< 10^{-3}$	1.007 (0.000)	$< 10^{-3}$	0.367 (0.329)	$< 10^{-3}$
Learned Greedy-SymGS	0.162 (0.155)	-	0.169 (0.310)	-	0.141 (0.355)	-	0.239 (0.599)	-
True-Greedy-SymGS	0.084 (0.094)	-	0.130 (0.266)	-	0.146 (0.285)	-	0.130 (0.241)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	5.833 (16.544)	$< 10^{-3}$	2.365 (6.639)	$< 10^{-3}$	1.985 (5.595)	0.005	0.876 (2.443)	0.004
HINTS-Jacobi (0.67)	1.177 (0.445)	$< 10^{-3}$	0.687 (1.414)	$< 10^{-3}$	1.970 (0.989)	$< 10^{-3}$	0.652 (0.883)	0.001
Learned Greedy-Jacobi (0.67)	0.693 (1.959)	-	0.489 (1.217)	-	1.060 (2.422)	-	0.517 (1.211)	-
True-Greedy-Jacobi (0.67)	0.396 (0.741)	-	0.406 (0.979)	-	0.528 (1.133)	-	0.352 (0.831)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.115 (0.331)	$< 10^{-3}$	0.652 (1.850)	$< 10^{-3}$	$< 10^{-3}$	0.056	0.078 (0.220)	0.002
HINTS-SOR (1.5)	0.480 (0.000)	$< 10^{-3}$	0.304 (0.680)	$< 10^{-3}$	0.608 (0.000)	$< 10^{-3}$	0.147 (0.165)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.017 (0.084)	-	0.148 (0.516)	-	$< 10^{-3}$	-	0.039 (0.090)	-
True-Greedy-SOR (1.5)	0.004 (0.003)	-	0.081 (0.188)	-	$< 10^{-3}$	-	0.030 (0.067)	-

Table 15: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances for the grid 63×63; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t-tests comparing each solver ensemble against the corresponding pairwise learned router (e.g., NO + Jacobi, NO + GS), using a one-sided alternative that the ensemble achieves lower error/AUC. Reported p-values correspond to these tests.

$\mathcal{W}$	$\|e_h^{(T)}\| \times 10^3$	AUC	Jacobi p-value	GS p-value	SymGS p-value	Jacobi (0.67) p-value	SOR (1.5) p-value
Poisson							
Learned Greedy {Jacobi, GS}	0.077 (0.064)	0.188 (0.410)	0.002	0.08	-	-	-
True Greedy {Jacobi, GS}	0.072 (0.064)	0.180 (0.403)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	0.112 (0.086)	0.151 (0.275)	0.002	0.015	$< 10^{-3}$	-	-
True Greedy {Jacobi, GS, SymGS}	0.054 (0.059)	0.126 (0.264)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.109 (0.103)	0.152 (0.282)	0.002	0.013	$< 10^{-3}$	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.054 (0.059)	0.126 (0.264)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.014 (0.006)	0.092 (0.205)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.054
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	0.004 (0.003)	0.082 (0.189)	-	-	-	-	-
ConvDiff							
Learned Greedy {Jacobi, GS}	0.136 (0.447)	0.170 (0.394)	$< 10^{-3}$	0.015	-	-	-
True Greedy{Jacobi, GS}	0.053 (0.065)	0.132 (0.279)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS}	0.063 (0.098)	0.142 (0.335)	$< 10^{-3}$	0.006	0.001	-	-
True Greedy {Jacobi, GS, SymGS}	0.025 (0.037)	0.105 (0.220)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.114 (0.216)	0.181 (0.413)	$< 10^{-3}$	0.024	0.011	$< 10^{-3}$	-
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67)}	0.025 (0.037)	0.105 (0.220)	-	-	-	-	-
Learned Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.037 (0.087)	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	$< 10^{-3}$	0.069
True Greedy {Jacobi, GS, SymGS, Jacobi (0.67), SOR (1.5)}	$< 10^{-3}$	0.030 (0.067)	-	-	-	-	-

D.8 Results with Dirichlet Boundary Conditions

We have replicated the Paired Solver and Solver Ensemble experiments for solutions with zero dirichlet boundary conditions.

In Table 16, we notice that, similar to PDEs with periodic boundary conditions, the learned greedy router closely matches the performance of the true greedy oracle and outperforms its single solver and HINTS baselines with great statistical significance. Despite the learned router matching the performance of the true greedy oracle, we observe that the learned router doesn't seem to provide much improvement over single-solver baseline in the case of SOR (1.5)-related solvers. This may be a result of the deeponet correction always being weaker relative to the SOR (1.5) correction in terms of error reduction.

Similar to Table 3, we notice, in Table 15, the learned greedy router seems to outperform the various pairwise configurations mostly with great statistical significance.

Table 16: Final error and AUC of squared L^2 error (lower is better). Values are mean ($\pm$ standard error (s.e.)) over 128 test instances with dirichlet boundaries; both mean and s.e. of error are reported in $\times 10^{-3}$. If a standard error is not shown, it is $< 10^{-3}$ in the reported units (raw $< 10^{-6}$). Statistical significance is assessed via paired t-tests comparing each baseline (single-solver only and HINTS) against the learned greedy router, using a one-sided alternative that the learned router achieves lower error/AUC. Reported p-values correspond to these tests.

Equation	Poisson				ConvDiff			
Methods	$\|e_h^{(T)}\| \times 10^3$	$\|e_h^{(T)}\|$ p-value	AUC	AUC p-value	$\|e_h^{(T)}\| \times 10^3$	$\|e_h^{(T)}\|$ p-value	AUC	AUC p-value
Jacobi-related solvers								
Jacobi Only	0.704 (2.151)	0.001	0.703 (1.929)	$< 10^{-3}$	0.003 (0.006)	0.057	0.169 (0.430)	$< 10^{-3}$
HINTS-Jacobi	2.674 (7.315)	$< 10^{-3}$	0.869 (2.396)	$< 10^{-3}$	0.474 (0.001)	$< 10^{-3}$	0.199 (0.254)	$< 10^{-3}$
Learned Greedy-Jacobi	0.243 (0.851)	-	0.236 (0.719)	-	0.003 (0.006)	-	0.076 (0.162)	-
True-Greedy-Jacobi	0.042 (0.047)	-	0.099 (0.252)	-	0.003 (0.007)	-	0.042 (0.089)	-
GS-related solvers								
GS only	0.138 (0.431)	0.001	0.408 (1.139)	$< 10^{-3}$	0.003 (0.006)	0.028	0.057 (0.146)	$< 10^{-3}$
HINTS-GS	2.668 (7.316)	$< 10^{-3}$	0.841 (2.360)	$< 10^{-3}$	0.470 (0.001)	$< 10^{-3}$	0.115 (0.137)	$< 10^{-3}$
Learned Greedy-GS	0.032 (0.158)	-	0.115 (0.433)	-	0.003 (0.006)	-	0.024 (0.048)	-
True-Greedy-GS	0.010 (0.015)	-	0.063 (0.166)	-	0.003 (0.006)	-	0.017 (0.036)	-
SymGS-related solvers								
SymGS only	2.924 (8.865)	0.01	0.926 (2.740)	0.006	1.331 (3.687)	0.004	0.413 (1.133)	0.004
HINTS-SymGS	2.748 (7.375)	0.008	0.856 (2.427)	0.008	0.917 (1.015)	0.006	0.332 (0.697)	0.003
Learned Greedy-SymGS	1.333 (5.715)	-	0.416 (1.722)	-	0.628 (1.468)	-	0.196 (0.449)	-
True-Greedy-SymGS	0.342 (0.906)	-	0.147 (0.375)	-	0.184 (0.323)	-	0.067 (0.118)	-
Jacobi (0.67)-related solvers								
Jacobi (0.67) only	1.265 (3.768)	$< 10^{-3}$	0.910 (2.468)	$< 10^{-3}$	0.008 (0.021)	0.003	0.253 (0.642)	$< 10^{-3}$
HINTS-Jacobi (0.67)	2.686 (7.311)	$< 10^{-3}$	0.886 (2.412)	$< 10^{-3}$	0.487 (0.002)	$< 10^{-3}$	0.220 (0.279)	$< 10^{-3}$
Learned Greedy-Jacobi (0.67)	0.134 (0.275)	-	0.215 (0.660)	-	0.004 (0.010)	-	0.127 (0.393)	-
True-Greedy-Jacobi (0.67)	0.096 (0.220)	-	0.145 (0.408)	-	0.004 (0.013)	-	0.061 (0.131)	-
SOR (1.5)-related solvers								
SOR (1.5) only	0.004 (0.011)	0.012	0.145 (0.412)	$< 10^{-3}$	0.003 (0.006)	1.0	0.006 (0.014)	1.0
HINTS-SOR (1.5)	2.650 (7.321)	$< 10^{-3}$	0.791 (2.295)	$< 10^{-3}$	0.468 (0.001)	$< 10^{-3}$	0.014 (0.014)	$< 10^{-3}$
Learned Greedy-SOR (1.5)	0.004 (0.011)	-	0.039 (0.091)	-	0.003 (0.006)	-	0.006 (0.014)	-
True-Greedy-SOR (1.5)	0.004 (0.011)	-	0.028 (0.070)	-	0.003 (0.006)	-	0.006 (0.014)	-

E Cost-Aware Routing and Wall-Clock Evaluation

The experiments of Section 7 compare methods per iteration, which favors methods whose iterations are expensive. This appendix evaluates the end-to-end *wall-clock time to a target accuracy* of the learned router against HINTS and the classical solver alone, for all five classical solvers and three PDEs, on a 128×128 grid ($N^2 = 128^2$ unknowns), using the cost-aware instantiation of the greedy rule introduced at the end of Section 4. Besides the Poisson and convection–diffusion equations of Section 7 we add *anisotropic diffusion*, $-\epsilon\,\partial^2_{x_1} u - \partial^2_{x_2} u = f$ with $\epsilon = 10^{-2}$ (diffusion tensor $\mathrm{diag}(\epsilon, 1)$), on the same periodic domain with the same forcing distribution. It is the standard model problem on which point-relaxation methods fail to smooth: an error component oscillatory in x_1 and smooth in x_2 is damped by only $(1-\epsilon)/(1+\epsilon) \approx 0.98$ per Jacobi sweep, and geometric multigrid with point smoothing loses its grid-independent convergence (line relaxation or semi-coarsening would be needed). The same pipeline is then applied to solver ensembles, to larger grids and against stronger classical baselines.

E.1 Protocol

Fast, matched implementations. A wall-clock comparison is meaningful only if the classical baseline is not handicapped by implementation. All solvers are therefore re-implemented for the periodic constant-coefficient setting as $O(N^2)$ operations: Jacobi and damped Jacobi as 5-point stencil sweeps, Gauss–Seidel, SOR and symmetric GS via cached sparse triangular factorizations. Their one-step iterates match the dense implementations used in the rest of the paper to 10^{-12} in double precision. Reference solutions are computed by FFT diagonalization of the same discrete operator, i.e., they are exact solutions of the discrete system, so true errors are available for evaluation at no charged cost. All timings are single-threaded CPU measurements (Apple M4 Pro, numpy/scipy/PyTorch, one thread) on an otherwise idle machine.

Corrector. The neural corrector is a DeepONet applied to the residual $r = f_h - \mathcal{L}^a_h u^{(t)}_h$ and evaluated on a 64×64 sensor grid, i.e., on the $4\times$ coarser grid obtained by band-limited (FFT) restriction to the modes $|k_1|, |k_2| \le 31$ (for the anisotropic equation the sensor grid is 128×32, i.e., full resolution along x_1, so that the band contains every mode the point smoothers cannot damp; the sensor grid is the only PDE-specific hyperparameter of the pipeline); its output is prolongated back to the fine grid by exact trigonometric interpolation, so the correction is band-limited by construction and never injects error into the high-frequency band that the classical smoother owns. The trunk net is the fixed orthonormal Fourier basis of that band ($p = 3969$ basis functions, as in POD-DeepONet [Lu et al., 2022]), and the branch net is a linear layer on the 4096 sensor values; it is fit by least squares on $64{,}000$ exact (residual, error) pairs of the discrete operator drawn from a mixture of spectral shapes (forcing-like, residual-like, white and randomly tilted spectra), and reaches a validation relative error of 1.7×10^{-6} on every mode of the band. The corrector is applied scale-equivariantly, $C_{\mathrm{NO}}(r) = \|Rr\|\,\mathcal{G}_\theta(Rr/\|Rr\|)$, which preserves $C_{\mathrm{NO}}(0) = 0$ (Proposition 4.2). One corrector call costs $\approx$ 0.6–0.7 ms at 128^2 (two FFTs and a 4096×4096 matrix–vector product), i.e., about 12 Jacobi sweeps, 3 GS/SOR sweeps, or 2 symmetric-GS sweeps (Table 20). The same corrector is used by HINTS and by all routing policies.

Cost-equalised macro-actions. Per-iteration costs c_j (residual evaluation plus the update) are measured once per pairing on the benchmark machine (minimum over seven repeated blocks of the block median, cached), the unit of cost u is one corrector call, and every operation is applied $m_j = \max\{1, \mathrm{round}(u/c_j)\}$ times per decision (Table 20); for the five stationary solvers this equalises costs to within 8%. An operation dearer than the unit (a multigrid cycle) is applied once and compared per unit of cost, $\|e_j\|^{u/(m_j c_j)}$; the surrogate weights $\tilde{c}_j$ use the same exponent. The cost-aware oracle runs Algorithm 1 on these macro-actions with the true error; the *greedy oracle* row runs the cost-agnostic Algorithm 1 on single operations, as in the main text.

Lightweight router. The learned router selects macro-actions from $6 + K$ scalar features, all $O(1)$ given the residual norm that any stopping test computes: the log relative residual, its per-iteration change over the last macro-action and an exponential moving average thereof, the log epoch index, a one-hot encoding of the previous macro-action, and the log counts of corrector calls so far and of iterations since the last call. It is a two-hidden-layer MLP (64 units) evaluated in numpy, costing

$\approx 5\,\mu$s per decision independently of N. It is trained with the surrogate of Section 5.1, with the per-state weights $\tilde{c}_k$ normalised by $\sum_k \tilde{c}_k$ (which does not change the Bayes decision), on states visited by oracle rollouts with ε-exploration (probability 0.1), followed by two DAgger rounds [Ross et al., 2011] in which the current router's own rollouts are relabelled by the oracle—the batch analogue of the scheduled sampling used in Section C. Training uses 128 instances per round drawn from the same GRF distribution as the test set but with a different seed; the router of every pairing and ensemble is trained in under two minutes.

Baselines and metric. HINTS applies the corrector every τ-th iteration; we report $\tau = 25$ (the schedule used in the main text and in Zhang et al. [2024]) and the best of $\tau \in \{5, 10, 25, 50\}$ selected *per cell* on the test instances themselves (an optimistic baseline). For each of 64 test instances we record, after every iteration, the true relative error $\|e_h^{(t)}\|/\|u_h\|$ (untimed, benchmark-only) and the cumulative charged wall-clock time of a replay that executes only the chosen operations (residual, update, and—for the learned router—every feature and decision computation; oracle decisions are not charged). We report the median first time at which the error drops below a tolerance ε, and paired per-instance median speedups. We highlight $\varepsilon = h^2 = 6.1 \times 10^{-5}$: the discretization error of the second-order scheme scales as $O(h^2)$, so driving the algebraic error far below h^2 does not improve the computed solution. We also report the iteration-based metrics of the main text (final relative error and AUC $= \sum_{t=1}^{T} \|e_h^{(t)}\|/\|u_h\|$ over $T = 300$ iterations).

E.2 Pairwise results

Table 2 (main text) reports the time to truncation-level accuracy for all ten pairings; the tables below give the comparison against HINTS at three tolerances (Table 17), the iteration-based metrics of the main text (Table 18), corrector-call and iteration counts (Table 19), the measured costs (Table 20), and the full tolerance sweeps (Tables 21 and 22).

Uniform gains. In every one of the 10 solver–equation pairings the learned router is the fastest deployable method at $\varepsilon = h^2$: 238×–1983×faster than the solver alone, 2.68×–10.3×faster than HINTS with τ=25, and 1.21×–3.22×faster than the best fixed period chosen per pairing on the test set (all paired Wilcoxon $p < 0.01$). The gains are not confined to the truncation level: at $\varepsilon = 10^{-8}$ the router is still 1.33×–5.26×faster than HINTS (τ=25) and 1.07×–1.81×faster than the per-tolerance best period (Table 17), and the best period itself varies across pairings and tolerances (τ=5 at truncation level, $\tau \in \{5, 10, 50\}$ at 10^{-8}), so no single fixed schedule is competitive everywhere. The learned router reproduces the cost-aware oracle's decisions almost exactly (the same median iteration and call counts in Table 19; median times within 0.98×–1.06×of the oracle, the difference being its own decision cost and timer noise).

Where the time goes. Table 19 explains the numbers: with a corrector that removes the smooth band to 10^{-6} in one call, the cost-aware oracle and the router call it first and reach h^2 after one or two further sweeps, i.e., in about two iterations, whereas HINTS reaches the same accuracy only at its first scheduled call (iteration τ), having spent $\tau-1$ sweeps on an error that is almost entirely smooth. Below h^2 the remaining error is the high-frequency band that the corrector cannot see; the router therefore stops calling it (one to three calls in total down to 10^{-8}; Figure 13), whereas a fixed schedule keeps paying for a call every τ iterations—HINTS (τ=5) executes hundreds of useless calls on the Jacobi pairings (Table 21). The cost-agnostic greedy oracle (Algorithm 1 on single operations) makes the same decisions as its cost-aware counterpart here, because after the first call the corrector is never better than a single sweep on the remaining high-frequency error; the two rules differ only through the granularity of their decisions, and the learned router pays for neither.

Iteration-based metrics. Table 18 confirms that the iteration-count conclusions of Section 7 hold on the finer grid: the router's AUC over $T = 300$ iterations is three to four orders of magnitude below HINTS' and four to five below the solver-only baseline (all $p < 10^{-3}$), and its error after T iterations is at the stopping threshold of the runs (10^{-9}) except for undamped Jacobi, where all hybrids stall at the near-Nyquist band.

Table 17: Paired median speedup of the learned router over HINTS at three tolerances (64 instances; bold: $\geq 1.1\times$ with one-sided Wilcoxon $p < 0.01$). The right block compares against the fixed period that is best *for that tolerance and pairing*, chosen on the test set.

		vs. HINTS (τ=25)			vs. best fixed τ		
Equation	Solver	ε=10^{-3}	ε=h^2	ε=10^{-8}	ε=10^{-3}	ε=h^2	ε=10^{-8}
Poisson	Jacobi	**2.89**×	**2.68**×	**1.55**×	**1.19**× (τ=5)	**1.21**× (τ=5)	**1.22**× (τ=50)
	Jacobi (0.67)	**3.20**×	**2.90**×	**1.78**×	**1.31**× (τ=5)	**1.28**× (τ=5)	1.09× (τ=10)
	GS	**7.46**×	**6.75**×	**3.25**×	**1.99**× (τ=5)	**1.94**× (τ=5)	**1.25**× (τ=5)
	SymGS	**11.8**×	**9.21**×	**5.26**×	**2.71**× (τ=5)	**2.34**× (τ=5)	**1.81**× (τ=5)
	SOR (1.5)	**7.63**×	**10.3**×	**2.65**×	**1.97**× (τ=5)	**3.22**× (τ=5)	**1.25**× (τ=10)
ConvDiff	Jacobi	**3.34**×	**2.95**×	**1.33**×	**1.27**× (τ=5)	**1.34**× (τ=5)	**1.17**× (τ=50)
	Jacobi (0.67)	**3.64**×	**3.14**×	**1.77**×	**1.39**× (τ=5)	**1.32**× (τ=5)	1.07× (τ=10)
	GS	**7.71**×	**6.76**×	**3.78**×	**2.08**× (τ=5)	**2.60**× (τ=5)	**1.17**× (τ=5)
	SymGS	**11.6**×	**8.70**×	**4.75**×	**2.69**× (τ=5)	**2.40**× (τ=5)	**1.62**× (τ=5)
	SOR (1.5)	**7.36**×	**8.89**×	**2.58**×	**2.03**× (τ=5)	**2.40**× (τ=5)	**1.31**× (τ=5)

Table 18: Iteration-based metrics of the main text on the 128×128 grid: relative error after $T = 300$ iterations and AUC of the relative error over T iterations, mean (standard error) over 64 instances; p-values are one-sided paired t-tests of the baseline's AUC against the learned router's.

	Poisson			ConvDiff		
Method	$\|e_h^{(T)}\|/\|u_h\|$	AUC	p	$\|e_h^{(T)}\|/\|u_h\|$	AUC	p
Jacobi-related solvers						
Solver only	7.99e-01 (4.5e-03)	2.68e+02 (8.9e-01)	$< 10^{-3}$	7.43e-01 (9.1e-03)	2.57e+02 (1.9e+00)	$< 10^{-3}$
HINTS (τ=25)	7.10e-06 (1.9e-06)	2.37e+01 (1.1e-02)	$< 10^{-3}$	1.55e-05 (4.2e-06)	2.36e+01 (2.7e-02)	$< 10^{-3}$
Learned router (ours)	**6.86e-06 (1.8e-06)**	**7.15e-03 (1.9e-03)**	-	**1.50e-05 (4.1e-06)**	**1.55e-02 (4.1e-03)**	-
Cost-aware oracle	*6.86e-06 (1.8e-06)*	*7.15e-03 (1.9e-03)*	-	*1.50e-05 (4.1e-06)*	*1.55e-02 (4.1e-03)*	-
Jacobi (0.67)-related solvers						
Solver only	8.56e-01 (3.8e-03)	2.77e+02 (7.1e-01)	$< 10^{-3}$	8.11e-01 (7.9e-03)	2.69e+02 (1.5e+00)	$< 10^{-3}$
HINTS (τ=25)	1.15e-12 (1.0e-13)	2.38e+01 (8.3e-03)	$< 10^{-3}$	1.46e-12 (2.7e-13)	2.37e+01 (2.1e-02)	$< 10^{-3}$
Learned router (ours)	**7.57e-10 (2.9e-11)**	**2.49e-03 (6.3e-04)**	-	**7.96e-10 (1.6e-11)**	**5.39e-03 (1.4e-03)**	-
Cost-aware oracle	*8.19e-10 (1.2e-11)*	*2.49e-03 (6.3e-04)*	-	*8.18e-10 (1.3e-11)*	*5.39e-03 (1.4e-03)*	-
GS-related solvers						
Solver only	6.53e-01 (5.2e-03)	2.42e+02 (1.2e+00)	$< 10^{-3}$	5.46e-01 (9.9e-03)	2.20e+02 (2.4e+00)	$< 10^{-3}$
HINTS (τ=25)	2.84e-11 (1.9e-12)	2.35e+01 (2.1e-02)	$< 10^{-3}$	8.78e-11 (4.8e-12)	2.32e+01 (5.4e-02)	$< 10^{-3}$
Learned router (ours)	**7.66e-10 (1.8e-11)**	**1.94e-03 (4.9e-04)**	-	**6.84e-10 (2.8e-11)**	**4.03e-03 (1.0e-03)**	-
Cost-aware oracle	*7.08e-10 (2.5e-11)*	*1.94e-03 (4.9e-04)*	-	*6.73e-10 (2.8e-11)*	*4.03e-03 (1.0e-03)*	-
SymGS-related solvers						
Solver only	4.49e-01 (4.6e-03)	2.03e+02 (1.3e+00)	$< 10^{-3}$	3.77e-01 (9.1e-03)	1.84e+02 (2.7e+00)	$< 10^{-3}$
HINTS (τ=25)	7.14e-10 (5.4e-11)	2.30e+01 (3.5e-02)	$< 10^{-3}$	3.64e-12 (2.3e-13)	2.26e+01 (8.1e-02)	$< 10^{-3}$
Learned router (ours)	**6.50e-10 (2.4e-11)**	**1.46e-03 (3.7e-04)**	-	**5.19e-10 (3.4e-11)**	**3.16e-03 (8.0e-04)**	-
Cost-aware oracle	*6.32e-10 (2.5e-11)*	*1.46e-03 (3.7e-04)*	-	*5.00e-10 (3.2e-11)*	*3.16e-03 (8.0e-04)*	-
SOR (1.5)-related solvers						
Solver only	3.09e-01 (3.5e-03)	1.72e+02 (1.3e+00)	$< 10^{-3}$	1.23e-01 (3.4e-03)	1.17e+02 (1.9e+00)	$< 10^{-3}$
HINTS (τ=25)	2.16e-10 (1.3e-11)	2.26e+01 (4.6e-02)	$< 10^{-3}$	8.06e-10 (2.3e-11)	2.12e+01 (1.2e-01)	$< 10^{-3}$
Learned router (ours)	**8.74e-10 (9.7e-12)**	**3.30e-03 (8.4e-04)**	-	**8.08e-10 (1.3e-11)**	**6.49e-03 (1.7e-03)**	-
Cost-aware oracle	*8.36e-10 (1.5e-11)*	*3.29e-03 (8.4e-04)*	-	*7.77e-10 (1.8e-11)*	*6.49e-03 (1.7e-03)*	-

Table 19: Median number of iterations and corrector calls needed to reach $\varepsilon = h^2$.

		HINTS (τ=25)		Cost-aware oracle		Learned router	
Equation	Solver	iters	NO calls	iters	NO calls	iters	NO calls
Poisson	Jacobi	25	1	2	1	2	1
	Jacobi (0.67)	25	1	2	1	2	1
	GS	26	1	2	1	2	1
	SymGS	26	1	2	1	2	1
	SOR (1.5)	50	2	2	1	2	1
ConvDiff	Jacobi	25	1	2	1	2	1
	Jacobi (0.67)	25	1	4	1	4	1
	GS	30	1	2	1	2	1
	SymGS	26	1	2	1	2	1
	SOR (1.5)	50	2	4	1	4	1

Table 20: Measured single-thread per-iteration costs (residual + update) and the resulting macro-action sizes $m_j = \mathrm{round}(c_{\max}/c_j)$.

Equation	Solver	classical iteration	corrector iteration	m_{solver}	m_{NO}
Poisson	Jacobi	52 μs	635 μs	12	1
	Jacobi (0.67)	52 μs	639 μs	12	1
	GS	200 μs	643 μs	3	1
	SymGS	366 μs	635 μs	2	1
	SOR (1.5)	197 μs	635 μs	3	1
ConvDiff	Jacobi	72 μs	658 μs	9	1
	Jacobi (0.67)	71 μs	659 μs	9	1
	GS	234 μs	673 μs	3	1
	SymGS	378 μs	665 μs	2	1
	SOR (1.5)	223 μs	660 μs	3	1

Table 21: Poisson: median wall-clock time to relative error ε for every policy and pairing (paired median speedup over solver-only in parentheses; $\dagger n$: n instances did not reach the tolerance within the iteration cap).

Solver	Method	$\varepsilon{=}10^{-2}$	$\varepsilon{=}10^{-3}$	$\varepsilon{=}h^2$	$\varepsilon{=}10^{-5}$	$\varepsilon{=}10^{-6}$	$\varepsilon{=}10^{-8}$
Jacobi	Solver only	536.0 ms	806.1 ms	1.15 s	1.36 s	1.62 s	2.14 s
	HINTS ($\tau{=}25$)	2.61 ms (193×)	2.61 ms (294×)	2.65 ms (412×)	2.89 ms (478×)	3.56 ms (447×)	226.1 ms (10.0×)
	HINTS ($\tau{=}10$)	1.42 ms (355×)	1.42 ms (543×)	1.46 ms (738×)	1.50 ms (864×)	3.89 ms (518×)	329.2 ms (6.94×)
	HINTS ($\tau{=}5$)	1.06 ms (463×)	1.06 ms (719×)	1.10 ms (984×)	1.15 ms (1153×)	6.30 ms (432×)	545.2 ms (4.19×)
	HINTS ($\tau{=}50$)	3.80 ms (122×)	3.80 ms (187×)	4.16 ms (253×)	4.22 ms (300×)	5.29 ms (322×)	182.8 ms (12.5×)
	Greedy oracle (Alg. 1)	773 μs (668×)	803 μs (996×)	854 μs (1283×)	964 μs (1350×)	2.94 ms (680×)	155.1 ms (14.4×)
	Cost-aware oracle	*862 μs (605×)*	*880 μs (901×)*	*942 μs (1199×)*	*1.02 ms (1268×)*	*2.94 ms (623×)*	*154.1 ms (13.9×)*
	Learned router (ours)	884 μs (597×)	918 μs (868×)	987 μs (1154×)	1.09 ms (1202×)	2.98 ms (655×)	151.3 ms (14.4×)
Jacobi (0.67)	Solver only	777.3 ms	1.16 s	1.62 s	1.93 s	2.33 s	3.16 s
	HINTS ($\tau{=}25$)	2.71 ms (290×)	2.71 ms (437×)	2.71 ms (621×)	2.71 ms (749×)	2.74 ms (874×)	5.16 ms (631×)
	HINTS ($\tau{=}10$)	1.46 ms (540×)	1.46 ms (819×)	1.46 ms (1161×)	1.50 ms (1335×)	1.61 ms (1467×)	3.03 ms (1058×)
	HINTS ($\tau{=}5$)	1.11 ms (713×)	1.11 ms (1074×)	1.13 ms (1464×)	1.17 ms (1681×)	1.91 ms (1332×)	4.09 ms (747×)
	HINTS ($\tau{=}50$)	4.42 ms (183×)	4.42 ms (278×)	4.42 ms (391×)	4.42 ms (468×)	4.43 ms (557×)	8.66 ms (377×)
	Greedy oracle (Alg. 1)	796 μs (988×)	818 μs (1455×)	915 μs (1904×)	1.09 ms (1970×)	1.50 ms (1575×)	2.83 ms (1112×)
	Cost-aware oracle	*805 μs (981×)*	*820 μs (1424×)*	*922 μs (1836×)*	*1.09 ms (1892×)*	*1.53 ms (1684×)*	*2.93 ms (1106×)*
	Learned router (ours)	822 μs (958×)	839 μs (1402×)	912 μs (1863×)	1.10 ms (1869×)	1.51 ms (1650×)	2.93 ms (1093×)
GS	Solver only	854.7 ms	1.29 s	1.83 s	2.17 s	2.60 s	3.45 s
	HINTS ($\tau{=}25$)	6.43 ms (134×)	6.43 ms (203×)	6.65 ms (277×)	12.4 ms (177×)	12.6 ms (208×)	13.9 ms (250×)
	HINTS ($\tau{=}10$)	2.88 ms (296×)	2.88 ms (450×)	3.14 ms (574×)	5.65 ms (383×)	5.65 ms (458×)	8.38 ms (412×)
	HINTS ($\tau{=}5$)	1.72 ms (502×)	1.72 ms (759×)	1.99 ms (906×)	3.36 ms (645×)	3.63 ms (711×)	5.55 ms (621×)
	HINTS ($\tau{=}50$)	11.8 ms (73.1×)	11.8 ms (110×)	12.0 ms (152×)	20.2 ms (108×)	23.4 ms (111×)	23.7 ms (146×)
	Greedy oracle (Alg. 1)	814 μs (1066×)	835 μs (1563×)	960 μs (1872×)	1.29 ms (1622×)	2.53 ms (999×)	4.34 ms (778×)
	Cost-aware oracle	*818 μs (1061×)*	*827 μs (1571×)*	*953 μs (1892×)*	*1.31 ms (1603×)*	*2.55 ms (990×)*	*4.37 ms (773×)*
	Learned router (ours)	832 μs (1041×)	850 μs (1544×)	1.01 ms (1809×)	1.34 ms (1563×)	2.61 ms (969×)	4.45 ms (758×)
SymGS	Solver only	720.5 ms	1.08 s	1.52 s	1.81 s	2.17 s	2.91 s
	HINTS ($\tau{=}25$)	10.2 ms (71.3×)	10.2 ms (108×)	10.3 ms (149×)	11.8 ms (155×)	20.2 ms (109×)	20.2 ms (146×)
	HINTS ($\tau{=}10$)	4.33 ms (167×)	4.33 ms (252×)	4.53 ms (341×)	6.89 ms (266×)	8.48 ms (259×)	9.10 ms (325×)
	HINTS ($\tau{=}5$)	2.35 ms (308×)	2.35 ms (465×)	2.60 ms (585×)	4.38 ms (412×)	4.63 ms (477×)	6.94 ms (425×)
	HINTS ($\tau{=}50$)	19.6 ms (36.9×)	19.6 ms (55.9×)	19.6 ms (78.4×)	20.8 ms (88.2×)	39.1 ms (56.4×)	39.1 ms (75.4×)
	Greedy oracle (Alg. 1)	814 μs (882×)	829 μs (1312×)	1.09 ms (1395×)	1.44 ms (1285×)	2.20 ms (1029×)	3.87 ms (772×)
	Cost-aware oracle	*817 μs (883×)*	*829 μs (1319×)*	*1.13 ms (1330×)*	*1.49 ms (1205×)*	*2.22 ms (1004×)*	*3.86 ms (766×)*
	Learned router (ours)	843 μs (856×)	853 μs (1289×)	1.13 ms (1331×)	1.49 ms (1198×)	2.28 ms (981×)	3.88 ms (754×)
SOR (1.5)	Solver only	281.1 ms	424.4 ms	600.9 ms	713.1 ms	855.7 ms	1.15 s
	HINTS ($\tau{=}25$)	6.37 ms (44.3×)	6.37 ms (67.6×)	12.2 ms (49.4×)	12.5 ms (57.5×)	12.5 ms (69.3×)	18.5 ms (62.4×)
	HINTS ($\tau{=}10$)	2.85 ms (98.0×)	2.85 ms (148×)	5.59 ms (107×)	5.59 ms (127×)	5.83 ms (149×)	8.56 ms (135×)
	HINTS ($\tau{=}5$)	1.70 ms (167×)	1.72 ms (251×)	3.37 ms (180×)	3.41 ms (208×)	5.08 ms (168×)	9.38 ms (122×)
	HINTS ($\tau{=}50$)	11.9 ms (23.3×)	11.9 ms (35.6×)	17.0 ms (35.3×)	23.8 ms (30.1×)	23.8 ms (36.2×)	35.0 ms (32.7×)
	Greedy oracle (Alg. 1)	783 μs (354×)	814 μs (528×)	1.03 ms (583×)	1.74 ms (419×)	3.77 ms (236×)	7.48 ms (158×)
	Cost-aware oracle	*784 μs (357×)*	*800 μs (527×)*	*1.02 ms (582×)*	*1.87 ms (380×)*	*3.94 ms (219×)*	*6.81 ms (168×)*
	Learned router (ours)	804 μs (343×)	823 μs (511×)	1.02 ms (576×)	1.82 ms (390×)	3.88 ms (223×)	7.05 ms (160×)

Table 22: Convection–diffusion: median wall-clock time to relative error ε for every policy and pairing.

Solver	Method	$\varepsilon{=}10^{-2}$	$\varepsilon{=}10^{-3}$	$\varepsilon{=}h^2$	$\varepsilon{=}10^{-5}$	$\varepsilon{=}10^{-6}$	$\varepsilon{=}10^{-8}$
Jacobi	Solver only	692.2 ms	1.07 s	1.51 s	1.80 s	2.16 s	2.90 s
	HINTS ($\tau{=}25$)	3.41 ms (205×)	3.41 ms (312×)	3.44 ms (436×)	3.46 ms (516×)	9.31 ms (306×)	384.4 ms (7.40×)
	HINTS ($\tau{=}10$)	1.76 ms (392×)	1.76 ms (598×)	1.80 ms (829×)	1.88 ms (972×)	12.8 ms (229×)	536.2 ms (5.30×)
	HINTS ($\tau{=}5$)	1.26 ms (539×)	1.26 ms (821×)	1.28 ms (1145×)	1.47 ms (1236×)	20.4 ms (140×)	838.4 ms (3.39×)
	HINTS ($\tau{=}50$)	5.76 ms (121×)	5.76 ms (184×)	5.76 ms (261×)	5.85 ms (307×)	9.40 ms (250×)	321.5 ms (8.89×)
	Greedy oracle (Alg. 1)	851 μs (792×)	874 μs (1198×)	992 μs (1532×)	1.31 ms (1317×)	7.99 ms (332×)	281.0 ms (10.2×)
	Cost-aware oracle	*961 μs (706×)*	*979 μs (1073×)*	*1.15 ms (1319×)*	*1.43 ms (1206×)*	*8.26 ms (337×)*	*281.8 ms (10.1×)*
	Learned router (ours)	1.00 ms (694×)	1.01 ms (1062×)	1.14 ms (1317×)	1.52 ms (1146×)	8.15 ms (313×)	292.7 ms (9.72×)
Jacobi (0.67)	Solver only	1.01 s	1.55 s	2.22 s	2.65 s	3.21 s	4.32 s
	HINTS ($\tau{=}25$)	3.40 ms (296×)	3.40 ms (461×)	3.40 ms (656×)	3.40 ms (784×)	3.47 ms (926×)	6.53 ms (663×)
	HINTS ($\tau{=}10$)	1.77 ms (572×)	1.77 ms (880×)	1.77 ms (1234×)	1.79 ms (1445×)	2.06 ms (1575×)	3.79 ms (1139×)
	HINTS ($\tau{=}5$)	1.28 ms (782×)	1.28 ms (1208×)	1.31 ms (1670×)	1.49 ms (1760×)	2.62 ms (1221×)	5.62 ms (764×)
	HINTS ($\tau{=}50$)	5.76 ms (174×)	5.76 ms (269×)	5.76 ms (386×)	5.76 ms (461×)	5.81 ms (551×)	11.3 ms (381×)
	Greedy oracle (Alg. 1)	849 μs (1185×)	885 μs (1789×)	1.10 ms (2036×)	1.38 ms (1877×)	1.91 ms (1668×)	3.56 ms (1177×)
	Cost-aware oracle	*856 μs (1182×)*	*878 μs (1766×)*	*1.08 ms (1974×)*	*1.37 ms (1885×)*	*1.88 ms (1624×)*	*3.61 ms (1164×)*
	Learned router (ours)	893 μs (1145×)	912 μs (1741×)	1.10 ms (1983×)	1.43 ms (1817×)	1.94 ms (1620×)	3.70 ms (1139×)
GS	Solver only	749.7 ms	1.15 s	1.63 s	1.94 s	2.34 s	3.14 s
	HINTS ($\tau{=}25$)	7.03 ms (107×)	7.03 ms (164×)	8.27 ms (198×)	13.7 ms (141×)	13.7 ms (170×)	20.5 ms (152×)
	HINTS ($\tau{=}10$)	3.21 ms (233×)	3.21 ms (356×)	5.02 ms (330×)	6.34 ms (312×)	6.35 ms (370×)	9.41 ms (337×)
	HINTS ($\tau{=}5$)	1.90 ms (391×)	1.90 ms (604×)	3.60 ms (453×)	3.66 ms (529×)	4.28 ms (547×)	6.24 ms (502×)
	HINTS ($\tau{=}50$)	13.3 ms (56.5×)	13.3 ms (86.7×)	14.2 ms (115×)	26.4 ms (74.0×)	26.4 ms (89.1×)	28.2 ms (112×)
	Greedy oracle (Alg. 1)	863 μs (855×)	887 μs (1283×)	1.23 ms (1341×)	1.75 ms (1108×)	3.28 ms (712×)	5.32 ms (587×)
	Cost-aware oracle	*869 μs (838×)*	*909 μs (1262×)*	*1.32 ms (1222×)*	*1.84 ms (1048×)*	*3.26 ms (717×)*	*5.46 ms (582×)*
	Learned router (ours)	892 μs (819×)	906 μs (1239×)	1.35 ms (1243×)	1.89 ms (1060×)	3.29 ms (704×)	5.59 ms (570×)
SymGS	Solver only	716.5 ms	1.10 s	1.56 s	1.85 s	2.23 s	2.99 s
	HINTS ($\tau{=}25$)	10.7 ms (67.4×)	10.7 ms (103×)	11.1 ms (141×)	21.1 ms (88.1×)	21.2 ms (106×)	21.6 ms (139×)
	HINTS ($\tau{=}10$)	4.55 ms (157×)	4.55 ms (240×)	5.02 ms (311×)	8.94 ms (207×)	8.94 ms (249×)	13.2 ms (226×)
	HINTS ($\tau{=}5$)	2.52 ms (287×)	2.52 ms (436×)	2.96 ms (529×)	4.92 ms (378×)	4.93 ms (455×)	7.35 ms (408×)
	HINTS ($\tau{=}50$)	20.7 ms (34.5×)	20.7 ms (53.0×)	21.1 ms (73.6×)	40.9 ms (45.5×)	41.4 ms (54.1×)	41.4 ms (72.4×)
	Greedy oracle (Alg. 1)	895 μs (785×)	911 μs (1198×)	1.25 ms (1254×)	1.66 ms (1130×)	2.97 ms (752×)	4.91 ms (618×)
	Cost-aware oracle	*886 μs (795×)*	*905 μs (1212×)*	*1.27 ms (1236×)*	*1.73 ms (1084×)*	*3.14 ms (693×)*	*4.48 ms (657×)*
	Learned router (ours)	908 μs (772×)	924 μs (1182×)	1.28 ms (1211×)	1.72 ms (1086×)	3.20 ms (698×)	4.54 ms (647×)
SOR (1.5)	Solver only	177.4 ms	270.0 ms	383.6 ms	457.3 ms	550.1 ms	737.6 ms
	HINTS ($\tau{=}25$)	6.91 ms (25.3×)	6.92 ms (38.5×)	13.6 ms (28.0×)	13.6 ms (33.4×)	13.6 ms (40.1×)	20.2 ms (36.1×)
	HINTS ($\tau{=}10$)	3.16 ms (54.9×)	3.27 ms (82.9×)	6.21 ms (61.4×)	6.21 ms (73.1×)	9.20 ms (59.6×)	10.7 ms (68.5×)
	HINTS ($\tau{=}5$)	1.85 ms (92.3×)	1.98 ms (137×)	3.64 ms (104×)	4.36 ms (107×)	5.88 ms (93.1×)	9.62 ms (76.0×)
	HINTS ($\tau{=}50$)	13.4 ms (13.2×)	13.4 ms (20.0×)	26.6 ms (14.5×)	26.6 ms (17.2×)	26.6 ms (20.7×)	39.8 ms (18.7×)
	Greedy oracle (Alg. 1)	876 μs (199×)	919 μs (300×)	1.39 ms (273×)	2.43 ms (188×)	4.34 ms (130×)	7.66 ms (98.8×)
	Cost-aware oracle	*900 μs (198×)*	*922 μs (294×)*	*1.63 ms (240×)*	*2.72 ms (167×)*	*4.55 ms (121×)*	*7.97 ms (93.9×)*
	Learned router (ours)	910 μs (195×)	934 μs (294×)	1.59 ms (238×)	2.67 ms (168×)	4.50 ms (120×)	8.18 ms (91.7×)

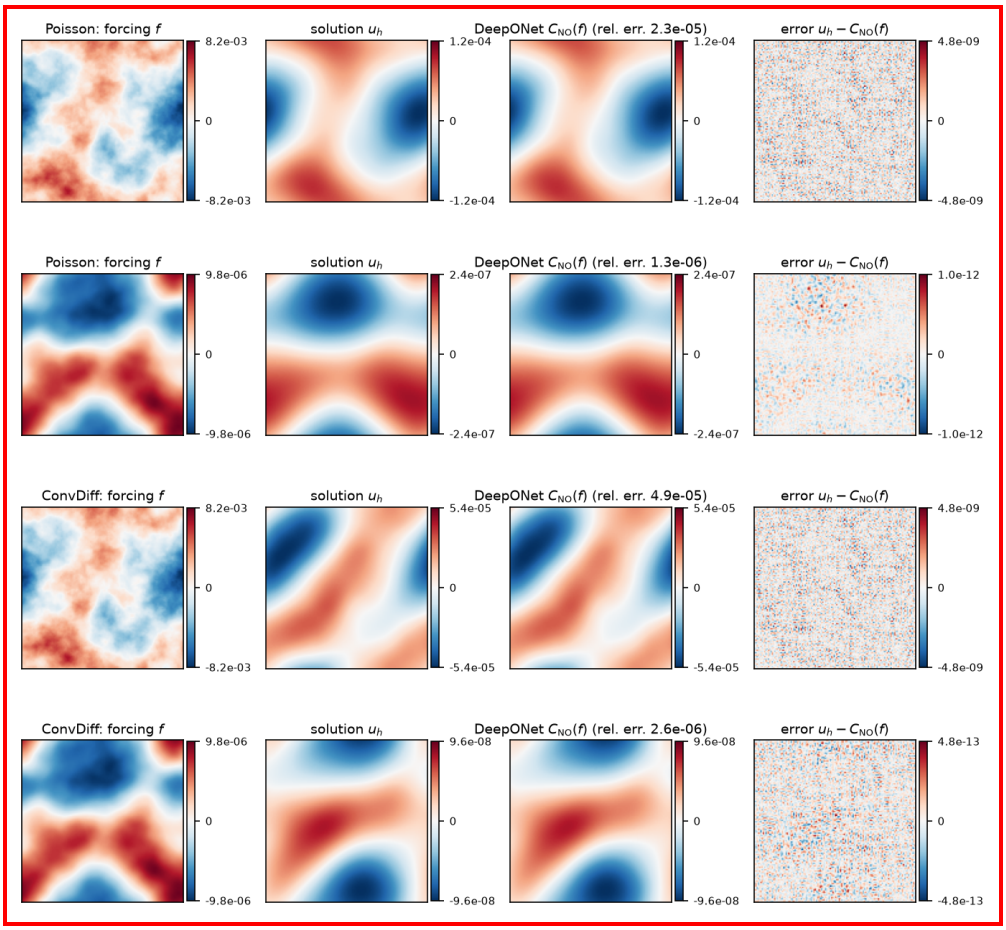

Figure 12: One-shot predictions of the corrector on two test forcings per equation (rows): forcing, exact discrete solution, DeepONet output $C_{\mathrm{NO}}(f)$ and its error. The corrector solves the band $|k| \le 31$ to $\approx 10^{-6}$; the remaining error is the high-frequency content of the solution outside the band, which the classical smoother removes in a few sweeps.

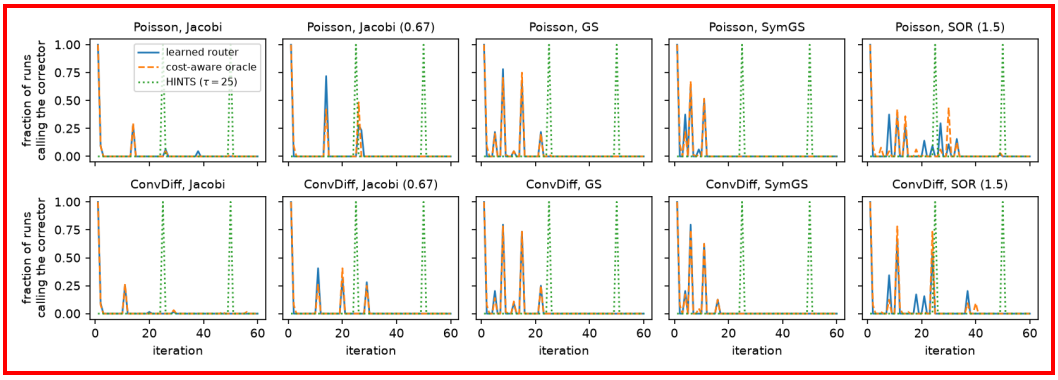

Figure 13: Corrector usage: fraction of test runs that execute a corrector call at each iteration, for the learned router, the cost-aware oracle and HINTS ($\tau = 25$). The router calls the corrector once at the start, spends a solver-dependent number of sweeps on the high-frequency band, and calls it again only when the smooth error has re-emerged; HINTS waits 24 sweeps before its first call.

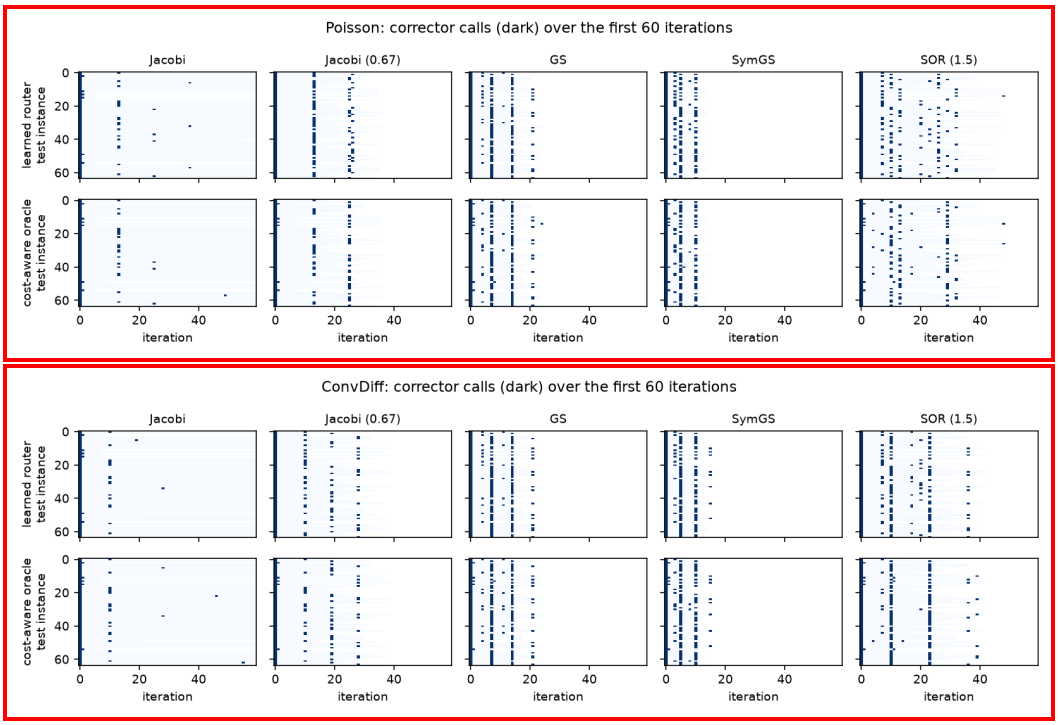

Figure 14: Routing decisions (dark: corrector call) of the learned router (top rows) and the cost-aware oracle (bottom rows) over the first 60 iterations of the stored test instances, for every pairing.

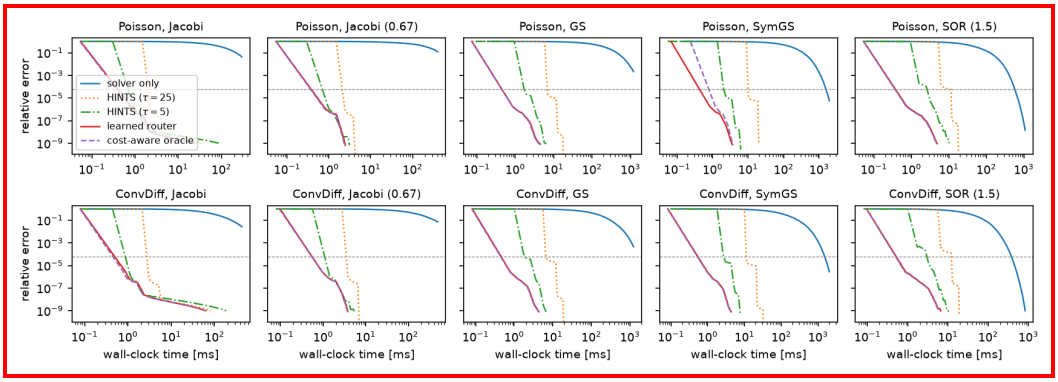

Figure 15: Relative error against charged wall-clock time for one test instance per pairing. The dashed horizontal line is $\varepsilon = h^2$.

E.3 Solver ensembles

The ensemble experiments of Section 7 are repeated in wall-clock terms with the cost-aware router over $\mathrm{NO} \cup \mathcal{W}$ for the same four ensembles $\mathcal{W}$ (macro-action sizes are set from the measured costs of all members, $m_j = \mathrm{round}(c_{\max}/c_j)$). HINTS does not apply to ensembles, so the comparison is against every member solver alone and against the best pairwise router $\mathrm{Router}(\mathrm{NO} \cup \{\mathrm{solver}\})$, solver $\in \mathcal{W}$, from Table 2.

Results. Table 23 shows that the ensemble router is competitive with, but does not systematically improve on, the best pairwise router of its members: across the 8 ensemble–equation cells the ratio of median times to h^2 (best pairwise router over ensemble router) ranges from $0.82\times$ to $1.14\times$, i.e.,

within $\pm 15\%$, while the ensemble router is $380\times$–$1512\times$ faster than the fastest member solver alone, and it does not need to know in advance which member is fastest (the best pairwise member differs between Jacobi and damped Jacobi across cells). The cost-aware *oracle* over the full ensemble is faster than every pairwise router in every cell (Table 23), so the ensembles do contain exploitable diversity; the gap between the ensemble router and its oracle (significant in the larger ensembles, Table 30) is imitation error. It has a clear cause: with several cost-equalised smoothers in the ensemble the oracle's choice among them depends on the spectral content of the current error, which the router's residual-norm history features only reflect indirectly, and the surrogate's weights are small for such near-ties, so the router agrees with the oracle on only 55–83% of its own decisions (against 90–99% for the pairwise routers) and occasionally picks a smoother that is a few percent worse per unit cost. Deciding *when* to call the corrector—the decision that dominates the wall-clock gains—is learned equally well in the ensembles. Table 24 lists how the iterations up to the h^2 crossing are distributed: the corrector is called once (or twice) and the remaining iterations go to whichever smoother the router selects for the high-frequency band, with an instance-dependent mix, as in Section D.6.

Table 23: Ensembles: median wall-clock time to $\varepsilon = h^2$ on the 128×128 grid (64 instances). The ensemble router is compared with the fastest member solver alone, with the fastest pairwise router of its members (paired median speedup in parentheses), and with the cost-aware oracle over the same ensemble.

Equation	$\mathcal{W}$	Best solver only	Best pairwise router	Router(NO $\cup$ $\mathcal{W}$)	Oracle(NO $\cup$ $\mathcal{W}$)
Poisson	$\{Jacobi, GS\}$	1.15 s (Jacobi)	987 μs (Jacobi)	964 μs (1.09$\times$ vs pairwise)	*834 μs*
	$\{Jacobi, GS, SymGS\}$	1.15 s (Jacobi)	987 μs (Jacobi)	951 μs (1.07$\times$ vs pairwise)	*854 μs*
	$\{Jacobi, GS, SymGS, Jacobi(0.67)\}$	1.15 s (Jacobi)	912 μs (Jacobi (0.67))	868 μs (0.98$\times$ vs pairwise)	*842 μs*
	$\{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)\}$	600.9 ms (SOR (1.5))	912 μs (Jacobi (0.67))	1.12 ms (0.86$\times$ vs pairwise)	*900 μs*
ConvDiff	$\{Jacobi, GS\}$	1.51 s (Jacobi)	1.14 ms (Jacobi)	1.25 ms (1.09$\times$ vs pairwise)	*974 μs*
	$\{Jacobi, GS, SymGS\}$	1.51 s (Jacobi)	1.14 ms (Jacobi)	1.01 ms (1.16$\times$ vs pairwise)	*996 μs*
	$\{Jacobi, GS, SymGS, Jacobi(0.67)\}$	1.51 s (Jacobi)	1.10 ms (Jacobi (0.67))	996 μs (1.00$\times$ vs pairwise)	*1.07 ms*
	$\{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)\}$	383.6 ms (SOR (1.5))	1.10 ms (Jacobi (0.67))	1.01 ms (1.06$\times$ vs pairwise)	*1.08 ms*

Table 24: Ensembles: average fraction of iterations (up to the h^2 crossing) spent in each component by the learned router, mean (standard deviation) over 64 instances; "-" marks components not in the ensemble.

Equation	$\mathcal{W}$	Jacobi	GS	SymGS	Jacobi (0.67)	SOR (1.5)	DeepONet
Poisson	$\{Jacobi, GS\}$	0.228 (0.365)	0.144 (0.216)	-	-	-	0.628 (0.386)
	$\{Jacobi, GS, SymGS\}$	0.237 (0.337)	0.043 (0.134)	0.086 (0.165)	-	-	0.634 (0.378)
	$\{Jacobi, GS, SymGS, Jacobi(0.67)\}$	0.235 (0.297)	0.008 (0.062)	0.101 (0.207)	0.000 (0.000)	-	0.656 (0.352)
	$\{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)\}$	0.094 (0.195)	0.020 (0.051)	0.182 (0.265)	0.046 (0.125)	0.007 (0.023)	0.650 (0.366)
ConvDiff	$\{Jacobi, GS\}$	0.089 (0.255)	0.310 (0.337)	-	-	-	0.600 (0.373)
	$\{Jacobi, GS, SymGS\}$	0.279 (0.354)	0.067 (0.156)	0.055 (0.103)	-	-	0.599 (0.374)
	$\{Jacobi, GS, SymGS, Jacobi(0.67)\}$	0.305 (0.340)	0.000 (0.000)	0.022 (0.038)	0.110 (0.214)	-	0.563 (0.410)
	$\{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)\}$	0.166 (0.302)	0.028 (0.127)	0.000 (0.000)	0.233 (0.375)	0.000 (0.000)	0.573 (0.399)

E.4 Stronger classical baselines, larger grids and overheads

The comparisons above are against the stationary solvers of Section 7. This section adds the classical methods a practitioner would actually reach for on these problems, scales the grid up to 512×512, and accounts explicitly for every overhead.

Baselines. All baselines run on the same $O(N^2)$ stencil with the same accounting (first iteration at which the true error is below ε; timed replay of exactly that many iterations). (i) *FFT direct solve*: for the constant-coefficient periodic problems used here the discrete operator is diagonalised by the FFT, so the system can be solved directly with two FFTs; this is the reference solver we use to compute ground truth and it is included as the lower bound on the time any iterative method can achieve on this problem class. (ii) *Geometric multigrid*: V(2,2)-cycles with lexicographic Gauss–Seidel smoothing, full-weighting restriction, bilinear prolongation, rediscretised coarse operators and a direct solve on the 32×32 coarsest grid (error contraction ≈ 0.03 per cycle). A V-cycle is a stationary iteration, so multigrid is also used as an ensemble member below. (iii) *Krylov methods* (SciPy implementations driven by the stencil matvec): conjugate gradients, CG preconditioned by symmetric GS and by a symmetric V-cycle for Poisson; BiCGSTAB, multigrid-preconditioned BiCGSTAB and restarted GMRES(20) for convection–diffusion (GMRES is accounted at the granularity of its restart cycles).

Table 25: Strong classical baselines on the 128×128 grid: median time to relative error ε over 64 instances (paired median speedup of the learned router with its best stationary pairing over each method in parentheses; > 1 means the router is faster) and median iterations to h^2. The FFT solve is a direct method for this problem class and bounds every iterative solver from below.

Equation	Method	$\varepsilon{=}10^{-3}$	$\varepsilon{=}h^2$	$\varepsilon{=}10^{-6}$	$\varepsilon{=}10^{-8}$	iters to h^2
Poisson	FFT direct solve	317 μs (0.36×)	317 μs (0.36×)	317 μs (0.36×)	317 μs (0.36×)	1
	Multigrid V(2,2) alone	2.75 ms (2.96×)	3.99 ms (3.62×)	5.47 ms (5.42×)	7.97 ms (8.11×)	3
	CG	9.37 ms (9.44×)	11.5 ms (11.8×)	14.2 ms (14.9×)	16.3 ms (17.6×)	162
	PCG (SymGS)	20.3 ms (20.5×)	30.8 ms (32.4×)	40.3 ms (44.0×)	52.5 ms (56.4×)	75
	PCG (multigrid)	2.97 ms (2.99×)	4.35 ms (3.85×)	5.84 ms (5.48×)	7.36 ms (7.44×)	3
	HINTS (τ=25), Jacobi (0.67)	2.71 ms (2.90×)	2.71 ms (2.90×)	2.74 ms (2.94×)	5.16 ms (5.57×)	25
	Learned router, {NO, Jacobi (0.67)}	839 μs	912 μs	1.51 ms	2.93 ms	2
	Learned router, {NO, Multigrid}	783 μs	2.05 ms	3.33 ms	4.72 ms	2
	Cost-aware oracle, {NO, Multigrid}	773 μs	2.02 ms	3.29 ms	4.66 ms	2
ConvDiff	FFT direct solve	376 μs (0.37×)	376 μs (0.37×)	376 μs (0.37×)	376 μs (0.37×)	1
	Multigrid V(2,2) alone	3.06 ms (2.92×)	4.52 ms (3.71×)	6.15 ms (5.56×)	8.92 ms (7.78×)	3
	BiCGSTAB	56.0 ms (49.3×)	67.6 ms (61.2×)	82.2 ms (74.9×)	93.1 ms (86.3×)	338
	BiCGSTAB (multigrid)	3.12 ms (3.01×)	6.17 ms (5.31×)	6.17 ms (6.11×)	9.21 ms (8.87×)	2
	GMRES(20)	301.7 ms (272×)	434.1 ms (430×)	615.9 ms (574×)	882.8 ms (817×)	2100
	HINTS (τ=25), Jacobi (0.67)	3.40 ms (3.14×)	3.40 ms (3.14×)	3.47 ms (3.22×)	6.53 ms (6.05×)	25
	Learned router, {NO, Jacobi (0.67)}	912 μs	1.10 ms	1.94 ms	3.70 ms	4
	Learned router, {NO, Multigrid}	873 μs	2.32 ms	3.83 ms	5.50 ms	2
	Cost-aware oracle, {NO, Multigrid}	851 μs	2.33 ms	3.83 ms	5.49 ms	2

Results at 128^2. With the same implementation technology (numpy stencils and scipy sparse triangular solves for every classical operation; a BLAS matrix–vector product for the corrector), the learned router with its best stationary pairing reaches truncation-level accuracy 3.62×–3.71× faster than multigrid alone (three V-cycles), 3.85×–5.31× faster than the best multigrid-preconditioned Krylov method and an order of magnitude faster than the unpreconditioned or SymGS-preconditioned Krylov solvers; the only faster method is the FFT direct solve, by 2.87×–2.91×, which exists only because the test problems have constant coefficients and periodic boundaries (it is not an iterative method and is the reference we solve against). The reason the hybrid beats multigrid here is the corrector's accuracy on the smooth band: one call removes the modes $|k| \le 31$ to 10^{-6} at the price of a single 4096×4096 matrix–vector product, whereas a V(2,2)-cycle contracts the whole spectrum by ≈ 0.03 at the price of four Gauss–Seidel sweeps on the fine grid plus the coarse-grid work. Table 25 also reports the router over {NO, multigrid}: the cost-aware oracle and the learned router call the corrector once and then let the V-cycle remove the rest, and are 1.93×–2.03× faster than multigrid alone, i.e., adding a strong classical member to the ensemble is handled by the same construction without any change (the multigrid cycle is dearer than the unit, so it is the case in which the per-unit-cost comparison of Section 4 matters).

Scaling with the grid. Table 26 repeats the Jacobi and Gauss–Seidel pairings on 256×256 and 512×512 grids (32 and 16 instances; correctors retrained per grid with the same recipe, keeping the sensor grid at 64×64, i.e., the corrector always handles the modes $|k| \le 31$, and the router retrained per pairing). The corrector's cost is dominated by a 4096×4096 matrix–vector product and is therefore nearly grid-independent, while every classical iteration grows as N^2, so the corrector becomes relatively cheaper as N grows; the classical member, however, has to cover a growing fraction of the spectrum. The solver-only baselines at 512^2 are capped at 8000 iterations and reported as lower bounds.

Table 26: Scaling: median time to $\varepsilon = h^2$ for the Jacobi and GS pairings against the strongest classical alternatives on each grid (paired median speedup of the learned router over each method in parentheses; $\dagger n$: n instances did not reach the tolerance within the iteration cap).

Equation	N	Solver only	HINTS (τ=25)	HINTS (τ=5)	Learned router	Cost-aware oracle	Multigrid	PCG/BiCGSTAB (MG)	FFT direct
Poisson, Jacobi	128^2	1.15 s	2.65 ms (2.68×)	1.10 ms (1.21×)	987 μs	*942 μs*	3.99 ms (3.43×)	4.35 ms (3.57×)	317 μs (0.33×)
	256^2	>7.61 s$^{\dagger 32}$	5.95 ms (2.52×)	2.47 ms (1.56×)	2.38 ms	*2.32 ms*	14.1 ms (6.70×)	15.2 ms (6.30×)	1.32 ms (0.58×)
	512^2	>9.85 s$^{\dagger 16}$	87.0 ms (1.11×)	126.3 ms (1.56×)	78.8 ms$^{\dagger 2}$	*74.2 ms*	81.9 ms (1.14×)	85.6 ms (1.26×)	7.65 ms (0.10×)
Poisson, GS	128^2	1.83 s	6.65 ms (6.75×)	1.99 ms (1.94×)	1.01 ms	*953 μs*	3.99 ms (3.39×)	4.35 ms (3.52×)	317 μs (0.33×)
	256^2	28.83 s	22.0 ms (5.77×)	6.07 ms (1.70×)	3.92 ms	*3.78 ms*	14.1 ms (4.15×)	15.2 ms (3.85×)	1.32 ms (0.36×)
	512^2	>29.96 s$^{\dagger 16}$	142.8 ms (1.11×)	162.5 ms (1.21×)	131.5 ms	*130.0 ms*	81.9 ms (0.69×)	85.6 ms (0.76×)	7.65 ms (0.06×)
ConvDiff, Jacobi	128^2	1.51 s	3.44 ms (2.95×)	1.28 ms (1.34×)	1.14 ms	*1.15 ms*	4.52 ms (3.54×)	6.17 ms (5.18×)	376 μs (0.36×)
	256^2	>12.73 s$^{\dagger 32}$	9.37 ms (1.92×)	4.97 ms (1.57×)	4.76 ms	*4.81 ms*	18.8 ms (4.60×)	24.0 ms (5.06×)	1.56 ms (0.39×)
ConvDiff, GS	128^2	1.63 s	8.27 ms (6.76×)	3.60 ms (2.60×)	1.35 ms	*1.32 ms*	4.52 ms (3.29×)	6.17 ms (4.42×)	376 μs (0.34×)
	256^2	30.05 s	31.0 ms (5.52×)	10.5 ms (1.53×)	7.05 ms	*7.15 ms*	18.8 ms (3.08×)	24.0 ms (3.47×)	1.56 ms (0.26×)

Overheads and amortisation. Table 27 lists the measured cost of every operation as a function of N, including one decision of the LSTM router of Section 7 (full-state input of size $4N^2$, hidden size 256, 4 layers): at 128^2 one LSTM decision costs – ms, about – Jacobi sweeps, which is why the wall-clock study uses the scalar-feature router ($\approx 5\,\mu$s, independent of N). The learned router's decisions therefore account for well under 1% of its solve time, and its corrector calls for 50–70% of the time to h^2 (one or two calls). Table 28 accounts for training: the corrector (shared with HINTS) is fit in minutes, the router in seconds to a few minutes per pairing, and the break-even number of solves after which the router's own training cost is recovered relative to HINTS (τ=25) with the same corrector.

Table 27: Measured single-thread cost of one operation as a function of the grid size (Poisson; median of repeated calls). Multigrid uses a direct solve below 64^2 and is not listed there.

(results pending)

Table 28: Training-cost amortisation for the GS pairing: corrector training (data generation and least-squares fit, CPU), router training (oracle rollouts and two DAgger rounds), median wall-clock saved per solve at $\varepsilon = h^2$ relative to HINTS (τ=25) and to the solver alone, and the number of solves after which the router's training cost is recovered relative to HINTS.

(results pending)

E.5 Statistical significance and multi-trial robustness

All wall-clock comparisons are paired: every policy solves the same 64 test instances, so we test the per-instance log time ratios with a one-sided Wilcoxon signed-rank test (alternative: the router is faster) and, as a parametric complement, a one-sided paired t-test on log times; instances that a baseline fails to bring below the tolerance within the iteration cap are counted against the router (ratio 0). Table 29 reports the p-values of the pairwise comparisons of Tables 2 and 17, together with a multi-trial analysis: the router of every pairing was retrained from scratch with five independent seeds (different training instances, exploration and initialisation; the corrector and the test set are fixed) and re-benchmarked, and we report the mean and standard deviation over seeds of its median time to h^2, the range over seeds of its paired speedup over HINTS (τ=25), and the number of seeds for which it is significantly faster ($p < 0.01$) than both HINTS (τ=25) and the best fixed period. Table 30 gives the corresponding tests for the ensembles (including the tests in the opposite direction, to show where an ensemble router is significantly *slower* than the best pairwise router or than its oracle), and Table 31 for the strong classical baselines of Section E.4. The iteration-based table (Table 18) already carries paired t-tests on the AUC.

Table 29: Significance of the pairwise comparisons on the 128×128 grid (64 paired instances; one-sided Wilcoxon / paired t-test p-values, "10^{-k}" denotes $p < 10^{-k}$) and robustness over five independent router-training seeds.

		$\varepsilon = h^2$: p (Wilcoxon / t)			$\varepsilon = 10^{-8}$: p		Router over 5 training seeds		
Equation	Solver	vs. solver only	vs. HINTS-25	vs. best τ	vs. HINTS-25	vs. best τ	time to h^2	speedup vs. HINTS-25	seeds with p<0.01
Poisson	Jacobi	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-8} / 10^{-7}	$< 10^{-10}$ / $< 10^{-10}$	10^{-9} / 10^{-8}	–	–	–
	Jacobi (0.67)	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-9} / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-4} / 10^{-5}	–	–	–
	GS	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-10} / $< 10^{-10}$	–	–	–
	SymGS	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	–	–	–
	SOR (1.5)	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-9} / 10^{-10}	–	–	–
ConvDiff	Jacobi	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / 10^{-7}	–	–	–
	Jacobi (0.67)	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-10} / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-5} / 10^{-6}	–	–	–
	GS	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	10^{-10} / $< 10^{-10}$	–	–	–
	SymGS	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	–	–	–
	SOR (1.5)	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	$< 10^{-10}$ / $< 10^{-10}$	–	–	–

Table 30: Ensembles: one-sided Wilcoxon p-values at $\varepsilon = h^2$ (64 paired instances) for the ensemble router against the fastest member solver alone, against the best pairwise router in both directions, and against the cost-aware oracle over the same ensemble (alternative: the ensemble router is slower).

Equation	$\mathcal{W}$	vs. best solver only	vs. best pairwise router (faster)	vs. best pairwise router (slower)	vs. oracle($\mathrm{NO} \cup \mathcal{W}$) (slower)
Poisson	$\{Jacobi, GS\}$	$< 10^{-10}$	10^{-7}	1.000	0.040
	$\{Jacobi, GS, SymGS\}$	$< 10^{-10}$	0.001	0.999	0.001
	$\{Jacobi, GS, SymGS, Jacobi(0.67)\}$	$< 10^{-10}$	0.856	0.144	10^{-4}
	$\{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)\}$	$< 10^{-10}$	1.000	10^{-8}	10^{-8}
ConvDiff	$\{Jacobi, GS\}$	$< 10^{-10}$	10^{-4}	1.000	10^{-5}
	$\{Jacobi, GS, SymGS\}$	$< 10^{-10}$	10^{-10}	1.000	0.492
	$\{Jacobi, GS, SymGS, Jacobi(0.67)\}$	$< 10^{-10}$	0.783	0.217	0.001
	$\{Jacobi, GS, SymGS, Jacobi(0.67), SOR(1.5)\}$	$< 10^{-10}$	0.047	0.953	0.566

Table 31: Strong baselines: one-sided Wilcoxon p-values that the learned router (best stationary pairing) is faster than each method at each tolerance; where the baseline's median is lower the entry gives the p-value that the router is slower.

Equation	Method	ε=10^{-3}	ε=h^2	ε=10^{-6}	ε=10^{-8}
ConvDiff 128^2	FFT direct solve	slower: $< 10^{-10}$	slower: $< 10^{-10}$	slower: $< 10^{-10}$	slower: $< 10^{-10}$
	Multigrid V(2,2) alone	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
	BiCGSTAB	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
	BiCGSTAB (multigrid)	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
	GMRES(20)	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
ConvDiff 256^2	FFT direct solve	slower: 10^{-4}	slower: 10^{-6}	slower: 10^{-10}	slower: 10^{-10}
	Multigrid V(2,2) alone	10^{-10}	0.006	0.533	slower: 10^{-4}
	BiCGSTAB	10^{-10}	10^{-9}	10^{-6}	0.105
	BiCGSTAB (multigrid)	10^{-10}	0.005	0.562	slower: 10^{-4}
	GMRES(20)	10^{-10}	10^{-10}	10^{-10}	10^{-5}
Poisson 128^2	FFT direct solve	slower: $< 10^{-10}$	slower: $< 10^{-10}$	slower: $< 10^{-10}$	slower: $< 10^{-10}$
	Multigrid V(2,2) alone	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
	CG	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
	PCG (SymGS)	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
	PCG (multigrid)	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$	$< 10^{-10}$
Poisson 256^2	FFT direct solve	slower: 10^{-5}	slower: 10^{-6}	slower: 10^{-9}	slower: 10^{-10}
	Multigrid V(2,2) alone	10^{-10}	10^{-5}	0.165	slower: 0.009
	CG	10^{-10}	10^{-9}	10^{-5}	0.789
	PCG (SymGS)	10^{-10}	10^{-10}	10^{-7}	0.014
	PCG (multigrid)	10^{-10}	10^{-5}	0.195	slower: 0.007
Poisson 512^2	FFT direct solve	0.353	slower: 10^{-4}	slower: 10^{-5}	slower: 10^{-5}
	Multigrid V(2,2) alone	10^{-5}	0.052	0.217	slower: 0.005
	CG	10^{-5}	10^{-5}	10^{-5}	10^{-5}
	PCG (SymGS)	10^{-5}	10^{-5}	10^{-5}	10^{-5}
	PCG (multigrid)	10^{-5}	0.037	slower: 0.702	slower: 0.005

F Limitations

A primary limitation of our approach is the higher on-time computational cost requires to learn the greedy routing strategy. In contrast, the simpler baselines such as HINTS incur no training overhead. While this cost is amortized during inference time and leads to improved solution quality, it may be prohibitive in settings with limited computational resources or when rapid deployment is required. On the deployment side, Section E shows that the gains survive cost accounting: with a corrector that covers the smooth half of the spectrum and a classical smoother that covers the rest, one or two corrector calls suffice and the router's advantage over fixed schedules is uniform across solvers, equations and tolerances. The corrector we use is specific to the constant-coefficient periodic setting (a fixed Fourier trunk basis on a coarse sensor grid), its cost grows with the fourth power of the sensor-grid size, and undamped Jacobi remains limited far below truncation level by the near-Nyquist modes that neither component damps. Our wall-clock study is restricted to single-thread CPU execution and stationary smoothers; comparisons against optimal-complexity solvers such as multigrid, and hybrid ensembles that include them, are left to future work.

Additionally, the effectiveness of the learned router depends on the neural operator exhibiting heterogeneous performance across samples. When the neural operators produces predictions of relatively uniform quality, the benefit of adaptive routing diminishes and it is more practical to set a fixed schedule based on the quality of the neural operator. In practice, achieving such heterogeneity can be sensitive to training choices, and neural operators themselves can be finicky to train.

G LLM Usage

LLMs, specifically ChatGPT and Gemini, supported the writing process in an iterative manner. We drafted paragraphs and asked the models for feedback on grammar and clarity. We then incorporated selected suggestions into the writing and repeated this process until we were satisfied with the writing.

The code developed for the experiments was written by the authors with the help of occasional code completions. The central components (e.g., the hybrid solver implementation and the greedy-router training pipelines) were implemented exclusively by the authors.

All substantive intellectual contributions, which include ideas, theorems, and analyses, are our own. LLMs were occasionally used to verify the correctness of proofs, but all proof strategies originated from the authors and relevant literature.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract clearly delineates the paper's main contributions and scope, and the introduction reiterates these claims in a structured, bullet-point form. All stated claims are directly supported by the theoretical and experimental results presented in the paper.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

Justification: The paper discusses key limitations in Appendix E.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: All theoretical results include clearly stated assumptions in the statement of theorems. Complete proofs are provided in Appendices A and B with appropriate cross-referencing from the main text.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

Justification: The paper provides sufficient detail to reproduce all experiments, including training procedures and implementation details described in the main text and Appendix C. Additionally, we release code with a comprehensive README to support the reproducibility of our results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: We provide open access to code with detailed instructions in a comprehensive README, including environment setup and commands to train routers and reproduce the main experimental results. These files will automatically generate the data needed for training. Data generation has also been documented in Appendix C.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies all key training and evaluation details in the Section 6 and Appendix C.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: The papr reports standard deviations in the main results tables, and further provides statistical significance via t-test p values in Appendix D.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper provides details on compute resources in Appendix C.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: This paper does not involve any human subjects and all data has been simulated. This research focuses on numerical methods for physical systems and does not raise concerns related to safety, security, discrimination, surveillance, deception & harrassment, environment, human rights, and bias & fairness.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses broader impacts in the introduction section. No direct negative societal impacts are anticipated.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.

- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release any models or datasets at all. It focuses on methodological contributions for numerical solvers using simulated data. Therefore, safeguards are not applicable.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [N/A]

    Justification: The paper does not use any external datasets or pretrained models. All experiments are conducted using simulated data and models trained in-house.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We release code for our method which is documented with a detailed README.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper does not involve crowdsourcing or any research with human subjects. All experiments are conducted using simulated data.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: This paper does not involve human subjects. All experiments are conducted using simulated data. Therefore, IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are primarily used for writing and editing. Further details are provided in Appendix F.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.